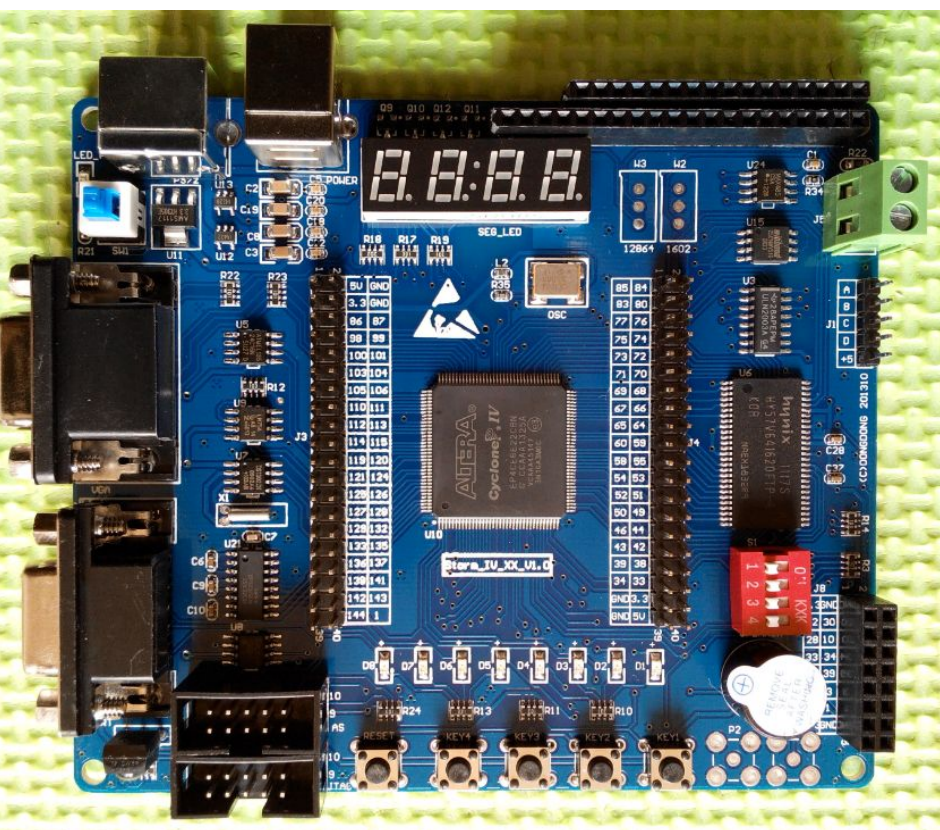


手把手教您学习 FPGA



阿东
恒创科技
2016/02

手把手教您学习 FPGA

恒创科技

阿东团队

2016-02

手把手教您学习 FPGA

第一章 FPGA 概述.....	8
1.1 为什么要学习 FPGA.....	8
1.2 学习 FPGA 几个疑点.....	8
1.2.1 VHDL 还是 Verilog.....	8
1.2.2 NIOS 重要还是 Verilog 重要?	8
1.3 FPGA 简介.....	9
1.4 Verilog 简介.....	11
1.4.1 端口定义.....	13
1.4.2 信号类型定义.....	13
1.4.3 数字定义.....	13
1.4.4 阻塞赋值和非阻塞赋值.....	14
1.5 FPGA 开发流程.....	15
第二章 FPGA 开发板介绍.....	17
2.1 STORM IV_E6 开发板简介.....	17
2.2 开发板详细配置.....	18
2.3 开发板硬件原理图.....	19
2.3.1 LED 发光二极管.....	19
2.3.2 通用按键.....	20
2.3.3 复位按键.....	20
2.3.4 拨码开关.....	21
2.3.5 晶振.....	21
2.3.6 七段数码管.....	22
2.3.7 蜂鸣器.....	23
2.3.8 PS/2 接口.....	23
2.3.9 UART 串口.....	24
2.3.10 VGA 接口.....	24
2.3.11 E2PROM.....	25
2.3.12 温度传感器 LM75.....	25
2.3.13 MAX485 接口.....	25
2.3.14 DS1302 时钟.....	26
2.3.15 串行 FLASH.....	26
2.3.16 电机驱动.....	26
2.3.17 时钟扩展口.....	27
2.3.18 JTAG&AS 接口.....	27

手把手教您学习 FPGA

2.3.19 1602 液晶接口.....	28
2.3.20 12864 液晶接口.....	29
2.3.21 SDCARD 接口.....	30
2.3.22 SDRAM.....	31
2.3.23 配置芯片.....	31
2.3.24 扩展口.....	32
第三章 设计实例篇.....	34
3.2 LED 流水灯.....	34
3.2.1 LED 简介.....	34
LED 流水灯原理简介:	34
3.2.2 实验任务.....	34
3.2.3 硬件设计.....	34
3.2.4 程序设计.....	35
3.2.5 实验现象.....	37
3.3 按键控制 LED.....	38
3.3.1 按键控制 LED 简介.....	38
3.3.2 实验任务.....	38
3.3.3 硬件设计.....	39
3.3.4 程序设计.....	40
3.3.5 实验现象.....	40
3.4 七段数码管 静态显示.....	41
3.4.1 数码管简介.....	41
3.4.2 实验任务.....	43
3.4.3 硬件设计.....	44
3.4.4 程序设计.....	44
3.4.5 实验现象.....	49
3.5 七段数码管 动态扫描.....	50
3.5.1 动态扫描简介.....	50
3.5.2 实验任务.....	50
3.5.3 硬件设计.....	51
3.5.4 程序设计.....	51
3.5.5 实验现象.....	57
3.6 串口发送.....	58
3.6.1 串口简介.....	58

3.6.2 实验任务.....	60
3.6.3 硬件设计.....	60
3.6.4 程序设计.....	60
3.6.5 实验现象.....	65
3.7 串口接收.....	65
3.7.1 串口简介.....	错误！未定义书签。
3.7.2 实验任务.....	错误！未定义书签。
3.7.3 硬件设计.....	错误！未定义书签。
3.7.4 程序设计.....	错误！未定义书签。
3.7.5 实验现象.....	65
3.8 同步 FIFO.....	66
3.8.1 同步 FIFO 简介.....	66
3.8.2 实验任务.....	70
3.8.3 硬件设计.....	70
3.8.4 程序设计.....	70
3.8.5 实验现象.....	74
3.8 异步 FIFO.....	74
3.8.1 异步 FIFO 简介.....	错误！未定义书签。
3.8.2 实验任务：	错误！未定义书签。
3.8.3 硬件设计：	错误！未定义书签。
3.8.4 程序设计：	错误！未定义书签。
3.8.5 实验现象：	74
3.9 状态机.....	75
3.9.1 状态机简介.....	75
3.9.2 实验任务.....	76
3.9.3 硬件设计.....	76
3.9.4 程序设计.....	76
3.9.5 实验现象.....	79
3.10 E2PROM 写操作.....	79
3.10.1 E2PROM 简介.....	79
3.10.2 实验任务.....	80
3.10.3 硬件设计.....	80
3.10.4 程序设计.....	81
3.10.5 实验现象.....	92

3.11 E2PROM 读操作.....	93
3.11.1 E2PROM 简介.....	错误！未定义书签。
3.11.2 实验任务.....	错误！未定义书签。
3.11.3 硬件设计.....	错误！未定义书签。
3.11.4 程序设计.....	错误！未定义书签。
3.11.5 实验现象.....	93
3.12 PS/2 键盘读操作.....	93
3.12.1 PS/2 接口简介.....	93
3.12.2 实验任务.....	94
3.12.3 硬件设计.....	95
3.12.4 程序设计.....	95
3.12.5 实验现象.....	99
3.13 VGA.....	99
3.13.1 VGA 简介.....	错误！未定义书签。
3.13.2 实验任务.....	错误！未定义书签。
3.13.3 硬件设计.....	错误！未定义书签。
3.13.4 程序设计.....	错误！未定义书签。
3.13.5 实验现象.....	99
3.14 LCD1602.....	99
3.14.1 LCD1602 简介.....	错误！未定义书签。
3.14.2 实验任务.....	错误！未定义书签。
3.14.3 硬件设计.....	错误！未定义书签。
3.14.4 程序设计.....	错误！未定义书签。
3.14.5 实验现象.....	错误！未定义书签。
3.15 红外遥控.....	错误！未定义书签。
3.15.1 红外遥控简介.....	错误！未定义书签。
3.15.2 实现任务.....	错误！未定义书签。
3.15.3 硬件设计.....	错误！未定义书签。
3.15.4 程序设计.....	错误！未定义书签。
3.15.5 实验现象.....	99
3.16 SDRAM 控制器.....	100
3.16.1 SDRAM 简介.....	错误！未定义书签。
3.15.2 实现任务.....	错误！未定义书签。
3.15.3 硬件设计.....	错误！未定义书签。

3. 15. 4 程序设计.....	错误！未定义书签。
3. 15. 5 实验现象.....	100
第四章 设计思想和感悟.....	100
4. 1 代码简单化.....	101
4. 2 注释层次化.....	102
4. 3 交互界面清晰化.....	102
4. 4 模块划分最优化.....	102
4. 5 方案精细化.....	103
4. 6 时序流水化.....	103

第一章 FPGA 概述

1.1 为什么要学习 FPGA

很多在校的学生对于学单片机、ARM、FPGA 和 DSP 非常困惑，不清楚自己要学习什么，首先学习单片机和 ARM 的人是最多的，学习 FPGA 和 DSP 的人相对少些，单片机和 ARM 的市场应用也是最多的，基本上涉及各个领域，DSP 一般应用在算法领域，FPGA 一般应用在高性能处理、实时要求高的领域，比如高速接口、报文转发、图像处理、视频传输等等领域，还可以应用在芯片前期验证。

如果您对算法不是很敏感，那么可以排除 DSP，像我就对算法不是很感冒。如果您只是想找个工作，对于单片机还是 FPGA 没有要求，那么可以好好学习下单片机和 ARM，当然 ARM 深入学习也还是比较困难的，特别是到了操作系统级别。FPGA 本身的特性决定了适合做高速和大容量处理，而且 FPGA 工程的方案设计相对比较复杂和系统化，所以学好了之后的系统观念更强，几年之后下来，和做单片机的差异也就越来越大了，很可能几年下来，您就是系统工程师了。这个也是学习 FPGA 可能给您带来的一个好处，当您系统观念越来越强的时候，那么承担的工作也就越来越重要，待遇当然也不用多说了。

学习 FPGA 还有一个好处就是可以转型做芯片设计，可能做芯片设计是很多人心中的一个梦想，想当年我就是怀揣着这个梦想走下来的。

1.2 学习 FPGA 几个疑点

1.2.1 VHDL 还是 Verilog

很多初学者对于 FPGA 学习是使用 VHDL 还是 Verilog 没有方向，很多学校里面还是在教 VHDL，真是误人子弟。之前一个初学者学习了 VHDL 一个月时间，然后才问我该学习什么 VHDL 还是 Verilog，这个地方特别强调下，现在基本所有的芯片和设计全部采用 Verilog 设计，我本身也经历了几个大型项目，全部采用 Verilog，包括 ARM 的芯片和 IP 都是采用 Verilog，使用 VHDL 的人非常非常少，只有部分学校在教学和少数的研究所在用，Verilog 已经占据了完全主导地位。

虽然说设计思想都是一样的，但是为什么还要学习大公司都不用的东西呢？

1.2.2 NIOS 重要还是 Verilog 重要？

可能很多初学者觉得 NIOS 很重要，但是很遗憾的告诉大家，FPGA 的主流和优势设计是使用 Verilog，为什么不是 NIOS 呢？因为 NIOS 使用 C 语言开发，需要 FPGA 使用宝贵的逻辑资源内建一个 CPU，速度可能还比不上一般的 ARM 芯片，另外功耗、成本都比 ARM 高很多，易用性也比 ARM 差很多，在一些 ARM 满足不了的场景使用 NIOS 倒是不错，比如系统需要 6 个以上串口、单板上已经有大容量的 FPGA（不使用也浪费）。

那么 FPGA 为什么还那么流行呢？主要是 FPGA 是逻辑器件，内部都是由可以编程的寄存器、组合逻辑构成，使用硬件语言编程实现，可以一个时钟周期执行一次动作，甚至 0 个时钟周期都可以执行操作（完全组合逻辑实现），单片机/ARM 需要 CPU 取指、译码、执行，那么自然 FPGA 速度非常快，这一点是 ARM 替代不了的。

1.3 FPGA 简介

FPGA (Field—Programmable Gate Array)，即现场可编程门阵列，它是在 PAL、GAL、CPLD 等可编程器件的基础上进一步发展的产物。它是作为专用集成电路 (ASIC) 领域中的一种半定制电路而出现的，既解决了定制电路的不足，又克服了原有可编程器件门电路数有限的缺点。

FPGA，可以说是划时代的发明，解决了硬件逻辑可以任意编程而不需要改动硬件的问题，提供了硬件高速处理的能力，同时给 ASIC 前端验证提供了一个非常重要的手段。

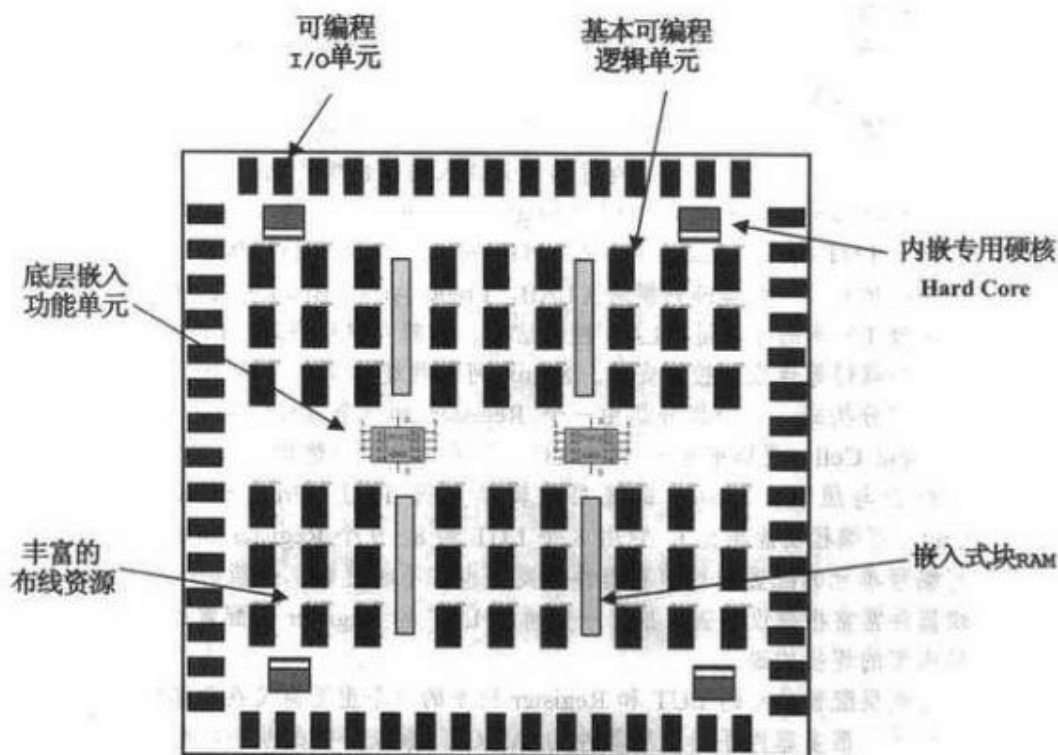


图 1-1 可编程逻辑器件的结构原理图

图 1-1 可编程逻辑器件的结构原理图

每个单元的基本概念介绍如下。

(1) 可编程输入/输出单元

输入/输出单元简称 I/O 单元，它们是芯片与外界电路的接口部分，完成不同电气特性下对输入/输出信号的驱动与匹配需求，为了使 FPGA 有更灵活的应用，目前大多数 FPGA 的 I/O 单元被设计为可编程模式，即通过软件的灵活配置，可以适配不同的电气标准与 I/O 物理特性；可以调整匹配阻抗特性，上下拉电阻；可以调整驱动电流的大小等。

可编程 I/O 单元支持的电气标准因工艺而异，不同器件商不同器件族的 FPGA 支持的 I/O 标准不同，一般来说，常见的电气标准 LVTTTL, LVCMOS, SSTL, HSTL, LVDS, LVPECL 和 PCI 等。值得一提的是，随着 ASIC 工艺的飞速发展，目前可编程 I/O 支持的最高频率越来越高，一些高编 FPGA 通过 DDR 寄存器技术，甚至可以支持高达 2Gbit/s 的数据速率。

(2) 基本可编程逻辑单元

基本可编程逻辑单元是可编程逻辑的主体，可以根据设计灵活地改变其内部连接与配置，完成不同的逻辑功能。FPGA 一般是基于 SRAM 工艺的，其基本可编程逻辑单元几乎都是由查找表和寄存器组成的。FPGA 内部查找表一般分为 4 个输入，查找表一般完成纯组合逻辑功能。FPGA 内部寄存器

结构相当灵活，可以配置为带同步/异步复位或置位，时钟使能的触发器，也可以配置成为锁存器。FPGA 一般依赖寄存器完成同步时序逻辑设计。一般来说，比较经典的基本可编程逻辑单元的配置是一个寄存器加一个查找表，但是不同的厂商的寄存器与查找表也有一定的差异，而且寄存器与查找表的组合模式也不同。例如，Altera 可编程逻辑单元通常被称为 LE，由一个寄存器加一个 LUT 构成。Altera 大多数 FPGA 将 10 个 LE 有机地组合在一起，构成更大的逻辑单元—逻辑阵列模块，LAB 中除了 LE 还包含 LE 之间的进位链，LAB 控制信号，局部互连线资源，LUT 级联链，寄存器级联链等连线与控制资源。Xilinx 可编程逻辑单元叫 Slice，它是由上下两个部分组成，每个部分都由一个寄存器加一个 LUT 组成，被称为 LC，两个 LC 之间有一些公用逻辑，可以完成 LC 之间的配合与级联。Lattice 的底部逻辑单元叫 PFU，它是由 8 个 LUT 和 8~9 个寄存器构成，当然这些可编程逻辑单元的配置结构随着器件的不断发展也在不断更新，最新的一些可编程逻辑器件常常根据设计需求新的 LUT 和寄存器的配置比率，并优化其内部的连接构造。

学习底层配置单元的 LUT 和寄存器比率的一个重要意义在于器件选型和规模估算。很多器件手册上用器件的 ASIC 门数或等效的系统门数表示器件的规模。但是由于目前 FPGA 内部除了基本可编程逻辑单元外，还包含丰富的嵌入式 RAM、PLL 或 DLL，专用 Hard IP Core（如 PCIe、Serdes 硬核）等，这些功能模块也会等效出一定规模的系统门，所以用系统门权衡基本可编程逻辑单元的数量是不准确的，常常混淆设计者。比较简单科学的方法是用器件的寄存器或 LUT 的数量衡量。例如，Xilinx 的 Spartan 系列的 XC3S1000 有 15360 个 LUT，而 Lattice 的 EC 系列 LFEC15E 也有 15360 个 LUT，所以这两款 FPGA 的可编程逻辑单元数量基本相当，属于同一规模的产品。同样道理，Altera 的 Cyclone IV 器件族的 EP4CE6 的 LUT 数量是 6000 个，就比前面提到的两款 FPGA 规模略小。需要说明的是，器件选型是一个综合性的问题，需要将设计的需求，降低成本，规模，速度等级，时钟资源，I/O 特性，封装，专用功能模块等诸多因素综合考虑。

(3) 嵌入式块 RAM

目前大多数 FPGA 都有内嵌的块 RAM，FPGA 内部嵌入可编程 RAM 模块，大大地拓展了 FPGA 的应用范围和使用灵活性。FPGA 内嵌的块 RAM 一般可配置为单口 RAM，双口 RAM，伪双口 RAM，CAM，FIFO 等常用存储结构。RAM 的概念和功能读者应该非常熟悉，在此不再赘述。FPGA 中其实并没有专用的 ROM 硬件资源，实现 ROM 的思路是对 RAM 赋予初值。所谓 CAM，即内容地址存储器，CAM 这种存储器在其每个存储单元都包含了一个比较的逻辑，写入 CAM 的数据会和其内部存储的每一个数据进行比较，并返回与端口数据相同的所有内部数据的地址。概括的讲，RAM 是一种根据地址读，写数据的存储单元；而 CAM 和 RAM 恰恰相反，它返回的是端口数据相同的所有内部地址。CAM 的应用也十分广泛，比如在路由器中的交换表等。FIFO 是先进先出队列的存储结构。FPGA 内部实现 RAM、ROM、CAM、FIFO 等存储结构都可以基于嵌入式块 RAM 单元，并根据需求自动生成相应的粘合逻辑以完成地址和片选等控制逻辑。

不同器件商或不同器件族的内嵌块 RAM 的结构不同，Xilinx 常见的块 RAM 大小是 4kbit 和 18kbit 两种结构，Lattice 常用的块 RAM 大小是 9KBIT，Altera 的块 RAM 最灵活，一些高端器件内部同时含有 3 种块 RAM 结构，分别是 M512 RAM、M4K RAM、M9K RAM。

需要补充的一点是，除了块 RAM，Xilinx 和 Lattice 的 FPGA 还可以灵活地将 LUT 配置成 RAM、ROM、FIFO 等存储结构，这种技术被称为分布式 RAM。根据设计需求，块 RAM 的数量和配置方式也是器件选型的一个重要标准。

(4) 丰富的布线资源

布线资源连通 FPGA 内部的所有单元，而连线的长度和工艺决定着信号在连线上的驱动能力和传输速度。FPGA 芯片内部有着丰富的布线资源，根据工艺、长度、宽度和分布位置的不同而划分为 4 类不同的类别。第一类是全局布线资源，用于芯片内部全局时钟和全局复位/置位的布线；第二类是长线资源，用以完成芯片 Bank 间的高速信号和第二全局时钟信号的布线；第三类是短线资源，

用于完成基本逻辑单元之间的逻辑互连和布线；第四类是分布式的布线资源，用于专有时钟、复位等控制信号线。

在实际中设计者不需要直接选择布线资源，布局布线器可自动地根据输入逻辑网表的拓扑结构和约束条件选择布线资源来连通各个模块单元。从本质上讲，布线资源的使用方法和设计的结果有密切、直接的关系。

(5) 底层嵌入功能单元

底层嵌入功能单元的概念比较笼统，这里我们指的是那些通用程度较高的嵌入式功能模块，比如 PLL、DLL、DSP、CPU 等。随着 FPGA 的发展，这些模块被越来越多地嵌入到 FPGA 的内部，以满足不同场合的需求。

目前大多数 FPGA 厂商都在 FPGA 内部集成了 DLL 或者 PLL 硬件电路，用以完成时钟的高精度、低抖动的倍频、分频、占空比调整、相移等功能。目前，高端 FPGA 产品集成的 DLL 和 PLL 资源越来越丰富，功能越来越复杂，精度越来越高。Altera 芯片集成的是 PLL，Xilinx 集成的是 DLL，Lattice 的新型 FPGA 同时集成了 PLL 与 DLL 以适应不同的需求。Altera 芯片的 PLL 模块分为增强型 PLL 和快速 PLL 等。Xilinx 芯片 DLL 的模块名称为 CLKDLL，在高端 FPGA 中，CLKDLL 的增强型模块为 DCM。这些时钟模块的生成和配置方法一般分为两种，一种是在 HDL 代码和原理图中直接例化，另一种是在 IP 核生成器中配置相关参数，自动生成 IP。Altera 的 IP 生成器叫做 Mega wizard，Xilinx 的 IP 核生成器叫做 Core Generator，Lattice 的 IP 核生成器叫做 Module/IP manager。另外可以通过在综合、实现步骤的约束文件中编写约束属性来完成时钟模块的约束。

越来越多的高端 FPGA 产品将包含 DSP 或 CPU 等软处理核，从而 FPGA 将由传统的硬件设计手段逐步过渡到系统级设计平台。例如 Altera 的 Stratix、Stratix II、Stratix IV 等器件族内部集成了 DSP core，配合同样逻辑资源，还可实现 ARM、MIPS、NIOS II 等嵌入式处理系统；Xilinx 的 Virtex II 和 Virtex II pro 系列 FPGA 内部集成了 Power PC450 的 CPU Core 和 MicroBlaze RISC 处理器 Core；Lattice 的 ECP 系列 FPGA 内部集成了系统 DSP Core 模块，这些 CPU 或 DSP 处理模块的硬件主要由一些加、乘、快速进位链、Pipelining 和 Mux 等结构组成，加上用逻辑资源和块 RAM 实现的软核部分，就组成了功能强大的软运算中心。这种 CPU 或 DSP 比较适合实现 FIR 滤波器、编码解码、FFT 等运算密集型运用。FPGA 内部嵌入 CPU 或 DSP 等处理器，使 FPGA 在一定程度上具备了实现软硬件联合系统的能力，FPGA 正逐步成为 SPOC 的高效设计平台。Altera 的系统级开发工具是 SOPC Builder 和 DSP Builder，专用硬件结构与软硬件协同处理模块等。Xilinx 的系统设计工具是 EDK 和 Platform Studio，Lattice 的嵌入式 DSP 开发工具是 Matlab 的 Simulink。

(6) 内嵌专用硬核

这里的内嵌专用硬核与前面的底层嵌入单元是有区分的，这里讲的内嵌专用硬核主要指那些通用性相对较弱，不是所有 FPGA 器件都包含硬核。我们称 FPGA 和 CPLD 为同样逻辑器件，是区分于专用集成电路而言的。其实 FPGA 内部也有两个阵营：一方面是通用性强，目标市场范围很广，价格适中的 FPGA；另一方面是针对性较强，目标市场明确，价格较高的 FPGA。前者主要指低成本 FPGA，后者主要指某些高端通信市场的可编程逻辑器件。

1.4 Verilog 简介

Verilog 语法标准贵的内容还是很多的，可以单独写一本书了，但是项目里面真正用到的不多，本节不准备描述所有的语法，只选择描述实际项目中用到几个语法。

阿东也不建议大家一上来就拼命学习各种语法，知道常用的几个语法酒可以了，关键还是在于学习实践 Verilog 设计思想。项目里面如果需要用到复杂的语法，打开语法书看下就知道了。

手把手教您学习 FPGA

下面通过一个流水灯来讲解基本的语法：

```
module LED (
    //input
    input      sys_clk           ,    //system clock;
    input      sys_rst_n         ,    //system reset, low is active;

    //output
    output reg [7:0] LED
);

//Parameter define
parameter WIDTH = 8           ;
parameter SIZE  = 8           ;
parameter WIDTH2 = 18          ;
parameter Para = 100000       |;

//Reg define
reg  [SIZE-1:0]    counter      ;
reg  [WIDTH2-1:0]  count        ;

//Wire define

//*****
//**                               Main Program
//**
//*****

// count for add counter
always @(posedge sys_clk or negedge sys_rst_n) begin
    if (sys_rst_n == 1'b0)
        count <= 18'b0;
    else
        count <= count + 18'b1;
end

// counter for delay time to LED display
always @(posedge sys_clk or negedge sys_rst_n) begin
    if (sys_rst_n == 1'b0)
        counter <= 8'b0;
    else if (count == Para)
        counter <= counter + 8'b1;
    else ;
end

// ctrl LED pipeline display when counter is equal 10 or 20 ....
always @(posedge sys_clk or negedge sys_rst_n) begin
    if (sys_rst_n == 1'b0)
        LED <= 8'b0;
    else begin
        case (counter)
            8'd10    : LED <= 8'b10000000 ;
            8'd20    : LED <= 8'b01000000 ;
            8'd30    : LED <= 8'b00100000 ;
            8'd40    : LED <= 8'b00010000 ;
            8'd50    : LED <= 8'b00001000 ;
            8'd60    : LED <= 8'b00000100 ;
            8'd70    : LED <= 8'b00000010 ;
            8'd80    : LED <= 8'b00000001 ;
            default  : LED <= 8'b00000000 ;
        endcase
    end
end

endmodule
//end of RTL code
```

1.4.1 端口定义

模块的端口可以是输入端口、输出端口或双向端口。缺省的端口类型为线网类型（即 wire 类型）。输入端口默认为 wire 类型，不需要定义，输出或双向端口能够声明为 wire/reg 型，使用 reg 必须显式声明，使用 wire 也强烈建议显式声明。

Verilog 2001 语法中的输入输出包括端口名、输入输出、wire/reg 类型、位宽定义，极大的减少了端口声明占用的代码行数。当前已经非常普及。

例子：

```
module LED (  
    //input  
    input sys_clk , //system clock;  
    input sys_rst_n , //system reset, low is active;  
    //output  
    output reg [7:0] LED );
```

该例子包括输入、输出、姓名名称、位宽、输出的信号类型。

1.4.2 信号类型定义

信号类型主要有 wire 和 reg 两种。线网类型用于对结构化器件之间的物理连线的建模。由于线网类型代表的是物理连接线，因此它不存储逻辑值。必须由器件所驱动。通常由 assign 进行赋值。

如 assign A = B ^ C;

reg 是最常用的寄存器类型，寄存器类型通常用于对存储单元的描述，如 D 型触发器、ROM 等。存储器类型的信号当在某种触发机制下分配了一个值，在分配下一个值之时保留原值。

reg 类型定义语法如下：

```
reg [msb: lsb] reg1, reg2, . . . reg N;
```

说明：reg 类型的信号不一定是寄存器，Verilog 语法规定的 Always 语句都要使用 reg 定义，而 Always 语句分为带有时钟和不带时钟两种，不带时钟的综合出来就是组合逻辑，带有时钟的综合出来才是寄存器。

1.4.3 数字定义

[size] 'base value

size 定义以位计的常量的位长；

base 为 o 或 O（表示八进制），b 或 B（表示二进制），d 或 D（表示十进制），h 或 H（表示十六进制）之一；

value 是基于 base 的值的数字序列。值 x 和 z 以及十六进制中的 a 到 f 不区分大小写。

下面是一些具体实例：

5 'o37 5 位八进制数（二进制 11111）

4 'd2 4 位十进制数（二进制 0011）

4 'b1x_01 4 位二进制数

7 'hx 7 位 x(扩展的 x)，即 xxxxxxx

4 'hz 4 位 z(扩展的 z)，即 zzzz

4 'd-4 非法：数值不能为负

3' b 001 非法：和基数 b 之间不允许出现空格

(2+3)'b10 非法：位长不能够为表达式

说明：

实际项目中使用十进制、二进制和十六进制较多，八进制使用较少。

1.4.4 阻塞赋值和非阻塞赋值

阻塞赋值：

“=” 用于阻塞的赋值，凡是在组合逻辑（如在 assign 语句中）赋值的请用阻塞赋值。阻塞赋值 “=” 在 begin 和 end 之间的语句是顺序执行，属于串行语句。

一个组合逻辑的例子：

```
always @(*) begin
    if ( new_vld_after == 1'b1 )
        port_win = new_port_after ;
    else if ( new_vld_before == 1'b1 )
        port_win = new_port_before ;
    else
        port_win = last_sel_port ;
end
```

注意：

1、always 语句的敏感变量如果不含有时钟，即 always (*) 这样描述，那么也属于组合逻辑，需要使用阻塞赋值。

非阻塞赋值：

“<=” 用于非阻塞的赋值，凡是在时序逻辑（如在 always 语句中）赋值的请用非阻塞赋值，非阻塞赋值 “<=” 在 begin 和 end 之间的语句是并行执行，属于并行执行语句。

一个时序逻辑的例子：

```
always @(posedge sys_clk or negedge sys_rst_n) begin
    if (sys_rst_n == 1'b0)
        clk_cnt <= 26'b0;
    else
        clk_cnt <= clk_cnt + 26'b1;
end
```

注意：

时序逻辑值的是带有时钟的 always 块逻辑，只有 always 带有时钟，这个逻辑才能综合为寄存器。

1.5 FPGA 开发流程

FPGA 的典型开发流程如下图所示：

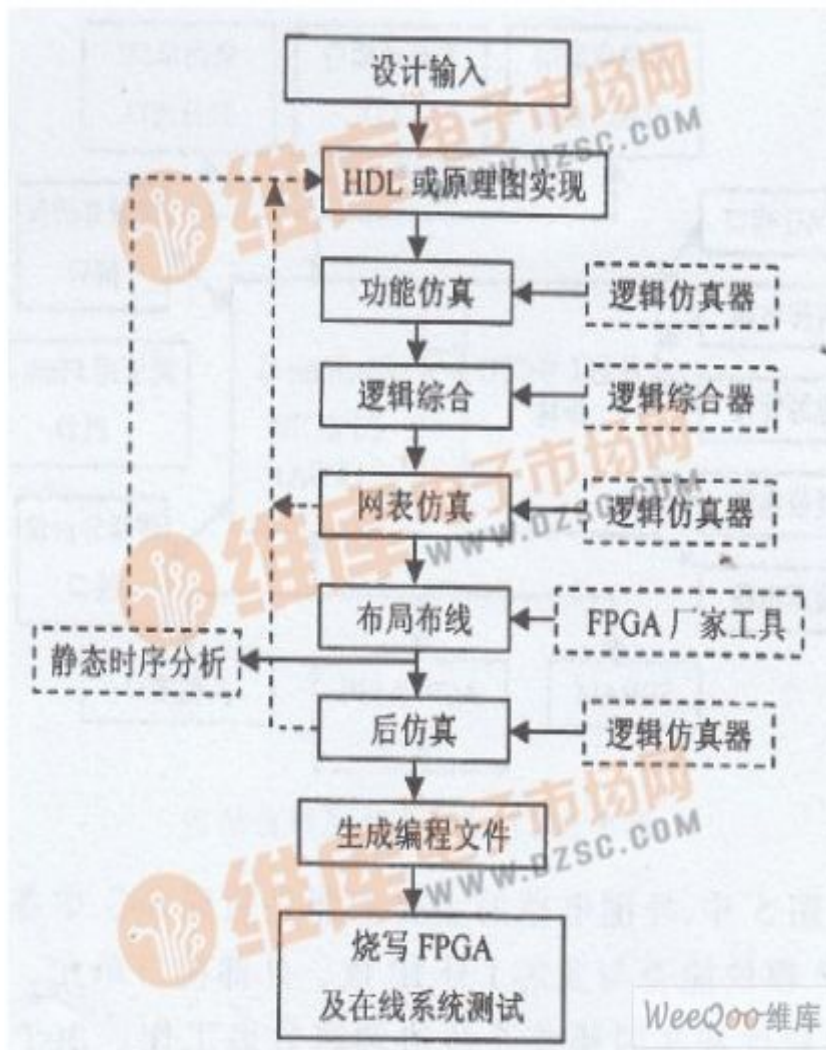


图 1-2 FPGA 的典型开发流程

在图中，逻辑仿真器主要有 Modelsim 等，逻辑综合器主要有 Quartus II、Synplify Pro、ISE 等，FPGA 厂家集成开发环境有 Altera 公司的 Quartus II，Xilinx 公司的 ISE。

设计输入主要有原理图输入和 HDL 输入两种方式，绝大部分设计，FPGA 和 ASIC 的工程师都使用 HDL 输入。

设计仿真主要包括功能仿真和网表仿真，设计仿真需要 RTL 代码或综合后的 HDL 网表和验证程序，有时候还需要测试数据，测试数据可能是代码编译后的二进制文件或使用专门的工具采集的数据。

布局布线工具利用综合生成的网表、调用模块的网表，根据布局布线目标，把设计翻译成原始的目标工艺，最后得到生成编程比特流所需的数据文件。布局布线一般需要的输入输出与调用关系如下图所示。布局布线目标包括所使用的 FPGA 具体型号等，约束条件包括管脚位置、管脚电平逻辑（LVCMOS 等）需要达到的时钟频率，有时包括部分模块的布局、块 RAM 的位置等。

静态时序分析主要是通过分析每个时序路径的延时，计算出设计的各项时序性能指标，如最高时钟频率、建立保持设计等，发现时序违规。它仅仅涉及时序性能的分析，不涉及设计的逻辑功能。

在一般设计中，只需要注意管脚位置和需要达到的时钟频率，逻辑端口与 FPGA 管脚的对应取决于 PCB 板的设计。

第二章 FPGA 开发板介绍

2.1 STORM IV_E6 开发板简介

STORM IV_E6 开发板使用 ALTERA Cyclone IV 四代 FPGA 芯片，四代的 FPGA 速度、功耗都比之前的 Cyclone I/II/III 系列 FPGA 优化很多。板子配套外设非常丰富，有 SDRAM、串行 FLASH、1602、12864、LED、按键、蜂鸣器、串口、VGA 接口、扩展口、MAX485、电机驱动接口等等。

此开发板非常适合初学者入门学习，板子价格也非常便宜，仅 100 多元，还赠送 USB Blaster 下载器，淘宝链接如下：<http://item.taobao.com/item.htm?id=35911884243>。

开发板图片如下：

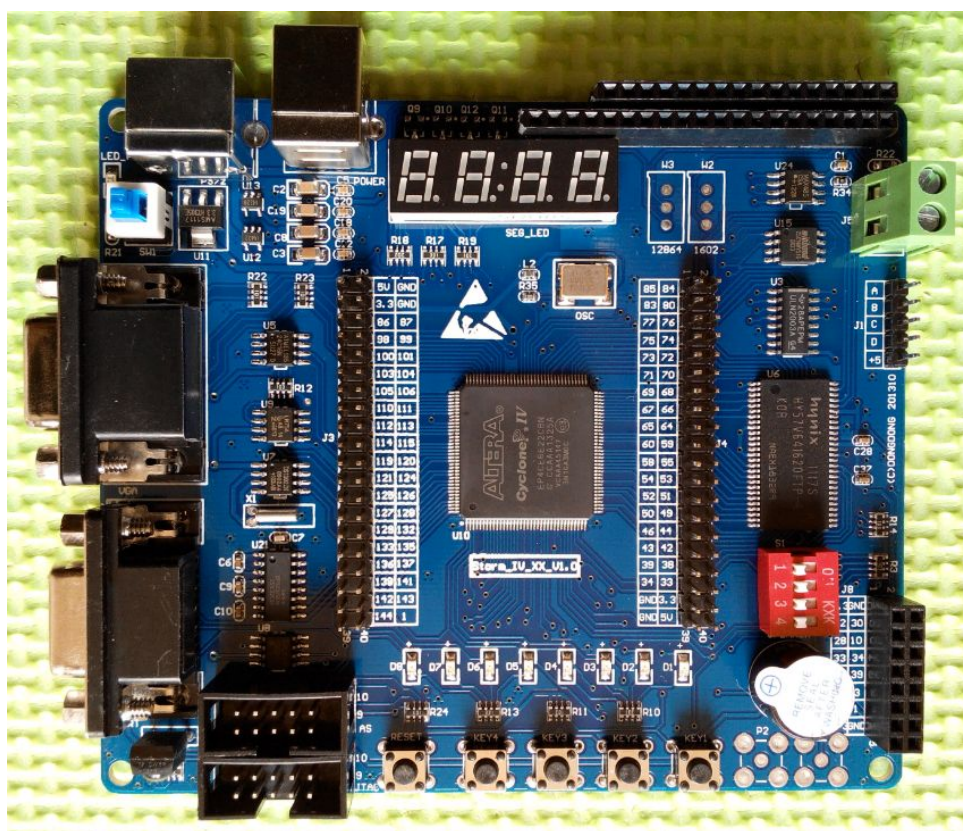
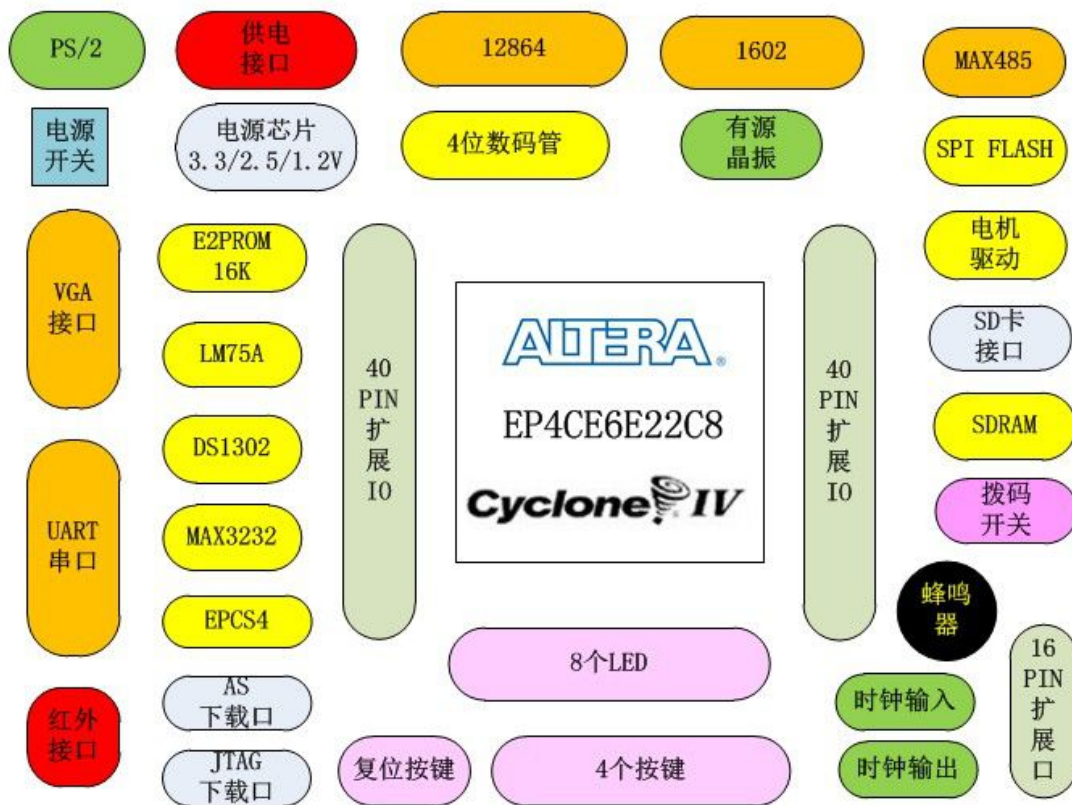


图 2-1 开发板图片

开发板功能示意图(FPGA 为 EP4CE6)：



2.2 开发板详细配置

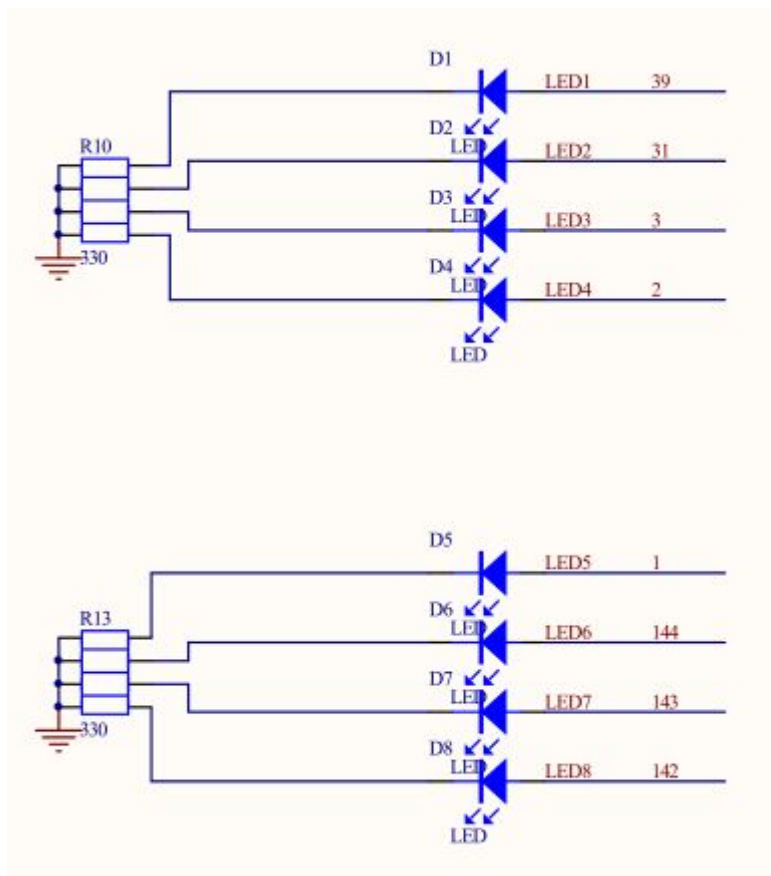
- 1 片 Cyclone IV 四代 FPGA: EP4CE6E22C8;
- 1 片 EPCS4 配置芯片, 4Mbit 容量 Flash;
- USB 接口提供电源;
- 1 个 50M 有源晶振;
- 8 个 LED 发光二极管;
- 4 个通用按键;
- 1 个复位按键;
- 1 片 SDRAM, 容量: 64Mbit SDRAM;
- 1 个 4 位拨码开关;
- 1 个 4 位七段数码管;
- 1 个 DS1302 时钟芯片;
- 1 个红外遥控接口;
- 1 个 MAX485 接口;
- 1 个电机接口: 支持 1 路步进电机, 1 路直流电机;
- 1 片 I2C 接口的 E2PROM, AT24C16, 16Kbit 容量;
- 1 片串行 FLASH: 16Mbit 容量;
- 1 个蜂鸣器;
- 1 个 RS232 UART 串口;
- 1 个 PS/2 接口, 可以接键盘或者鼠标;

手把手教您学习 FPGA

- 1 个 VGA 接口，可以接电脑显示器；
- 1 个温度传感器 LM75A；
- 1 个 1602 液晶接口，管脚兼容 1.8 寸 TFT 彩屏液晶；
- 1 个 12864 液晶接口；
- 1 个专用时钟输入接口；
- 1 个专用时钟输出接口；
- 2 个 40 脚扩展接口（排针）；
- 1 个 16 脚扩展接口（排母），精心设计的 16PIN 扩展口，可以直接插网络模块、无线模块等独立模块；
- 1 个 JTAG 接口；
- 1 个 AS 接口；
- 1 个 SDCARD 接口

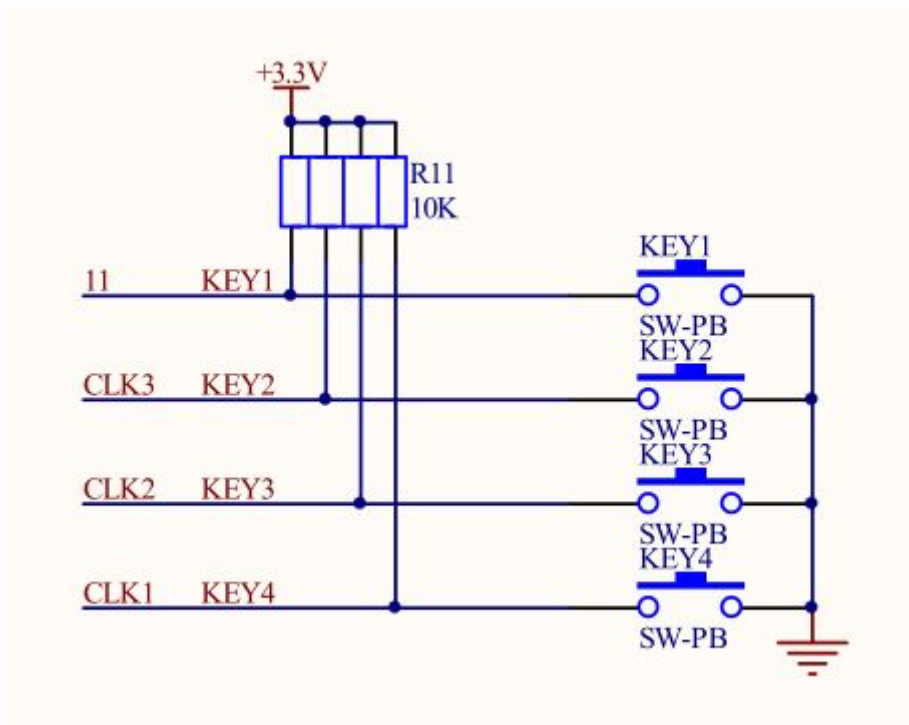
2.3 开发板硬件原理图

2.3.1 LED 发光二极管



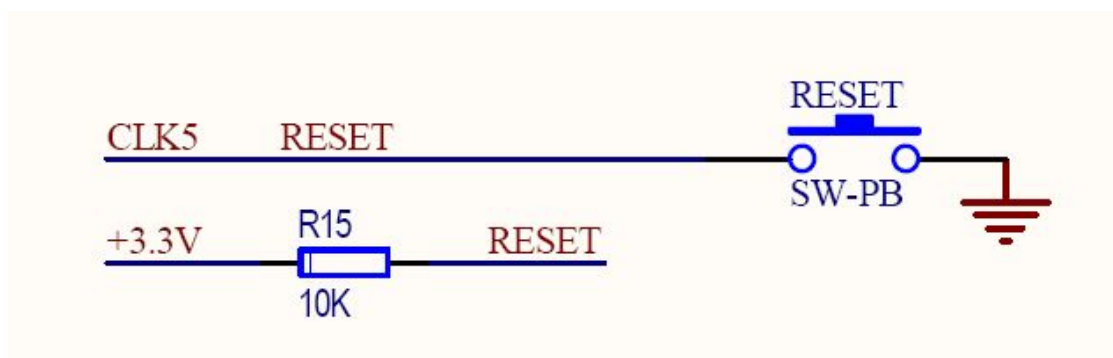
8 个 LED 指示灯，FPGA 输出为 1 时候 LED 发光，可以用来做 8bit 的数据显示。

2.3.2 通用按键



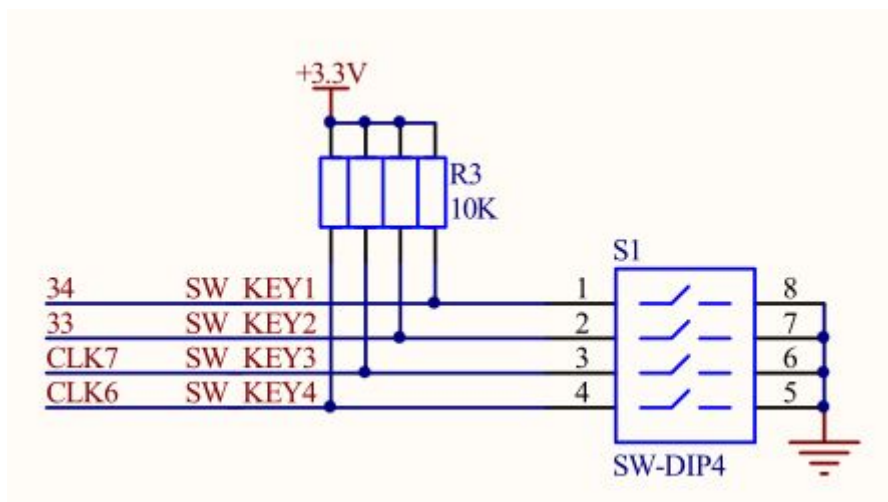
4 个通用按键，没有按下时候输入为 1，按下时候输入为 0。

2.3.3 复位按键



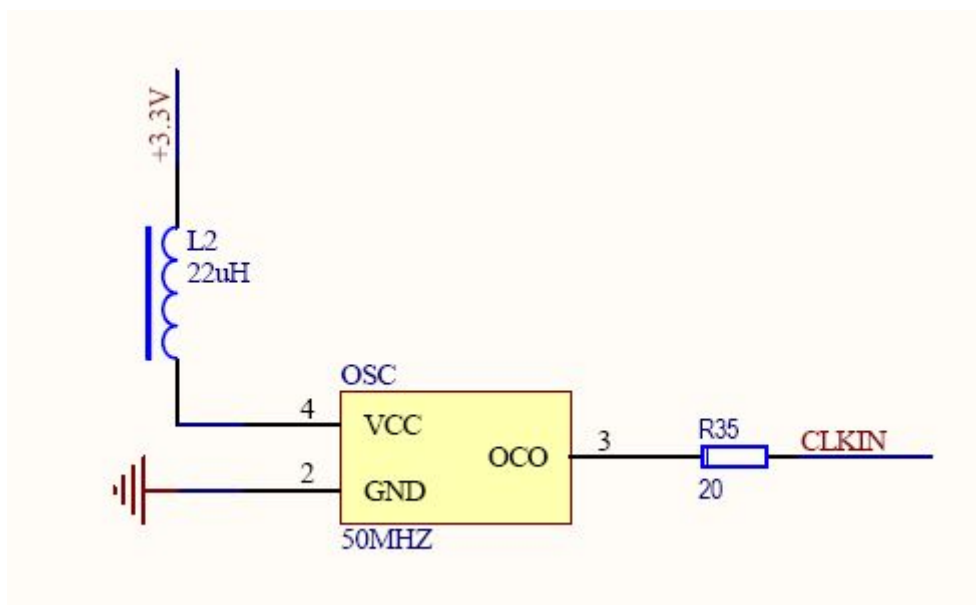
1 个复位按键，没有按下时候输入为 1，按下时候输入为 0，复位时候是 0 有效，代表复位。

2.3.4 拨码开关

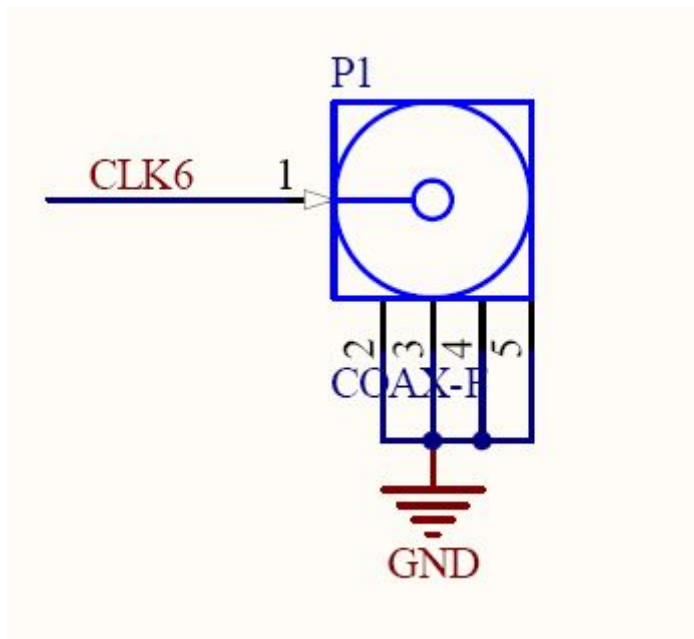


4 个拨码按键，没有拨下时候输入为 1，拨下时候输入为 0。

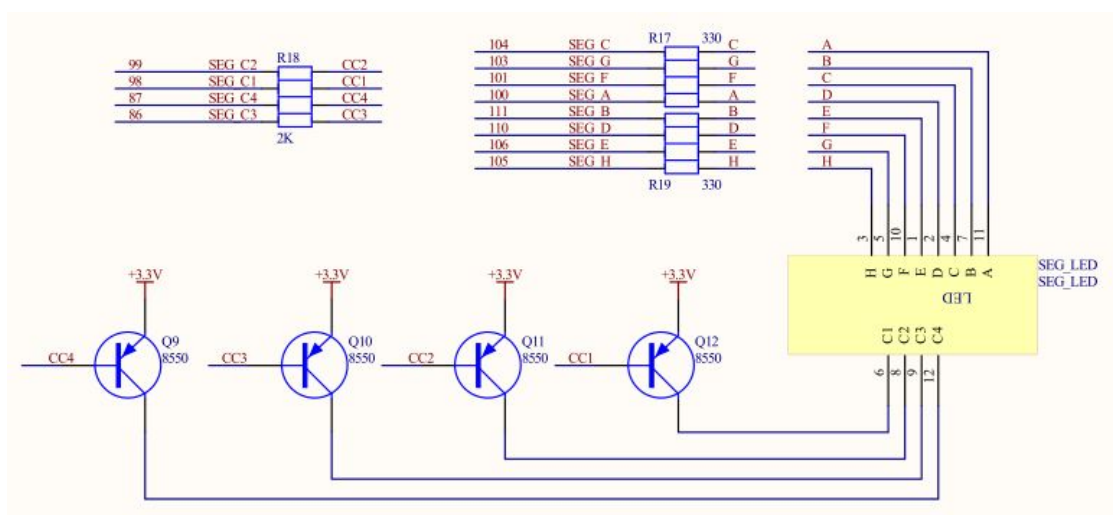
2.3.5 晶振



1 个 50Mhz 有源晶振，系统所使用的时钟就是这个提供的，如果需要多个时钟，高频时钟可以使用 PLL 进行倍频，或者使用时钟输入口进行输入。低频时钟可以使用 PLL 进行降频，或者使用时钟输入口进行输入，还可以自己设计分频器。

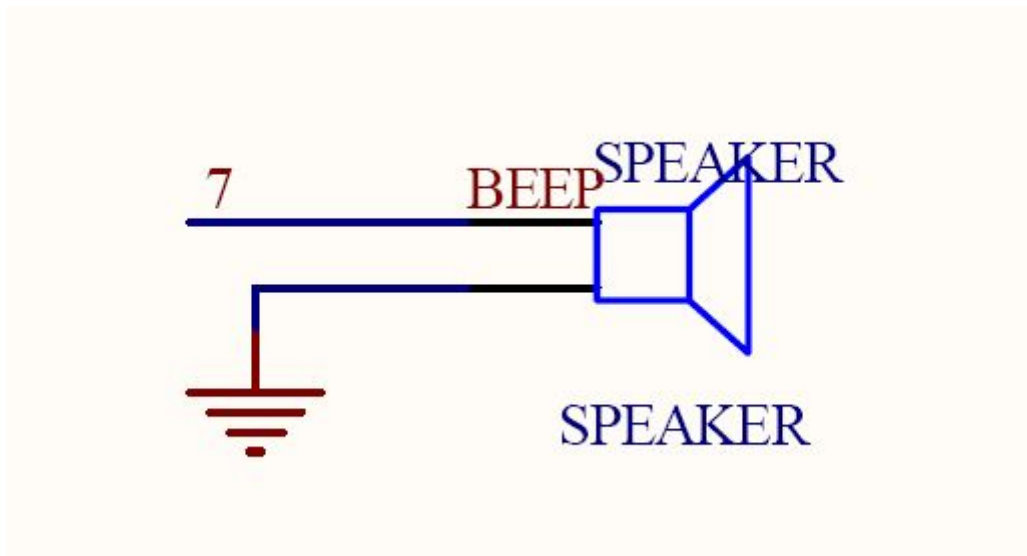


2.3.6 七段数码管



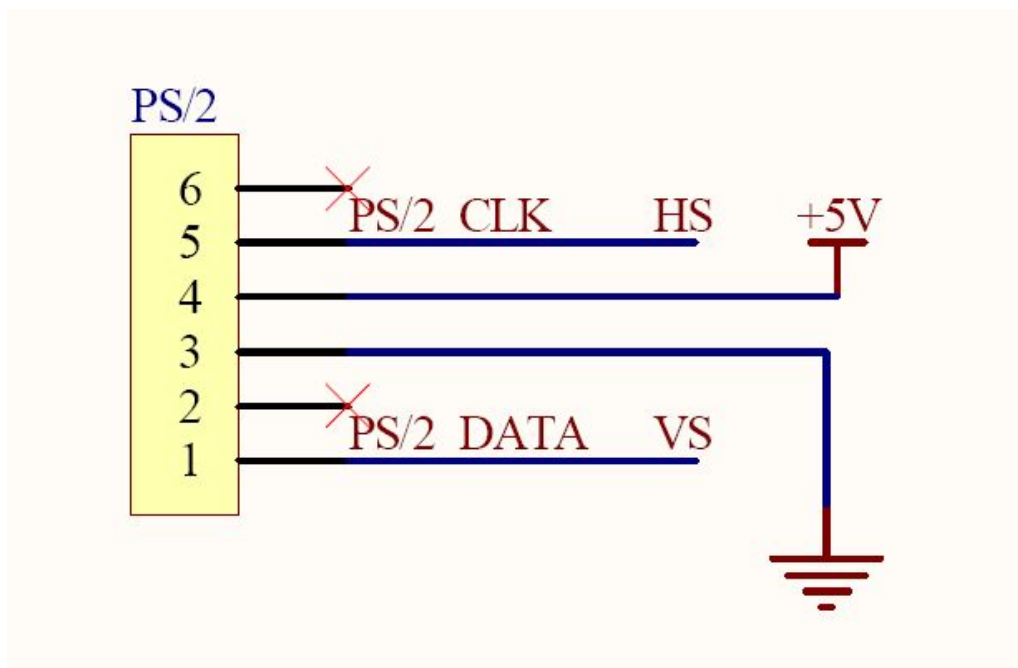
七段四位数码管，共阳，可以使用静态驱动或者动态扫描方式进行驱动。

2.3.7 蜂鸣器



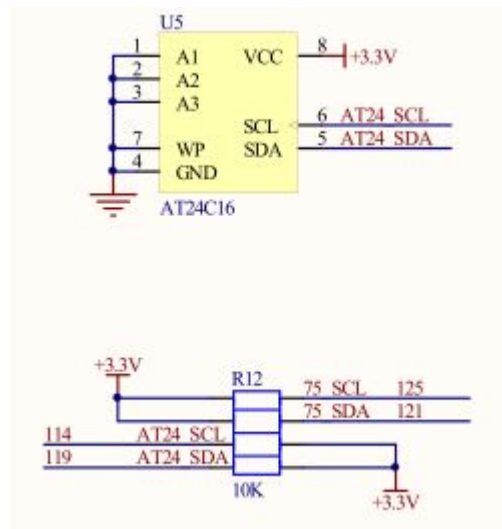
使用 FPGA IO 直接驱动，FPGA IO 输出高电平，蜂鸣器即可发声，低电平，蜂鸣器不发声。该蜂鸣器是有源蜂鸣器。

2.3.8 PS/2 接口



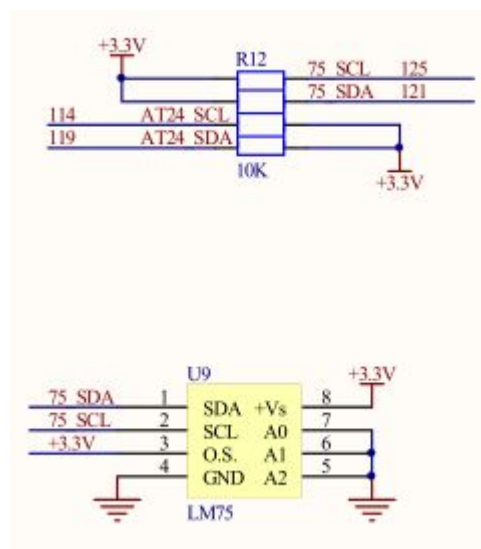
PS/2 的两个管脚通过 VGA 的串接电阻接到 FPGA 的 IO 上面，起到隔离的作用，和 VGA 的 HS/VS 复用管脚。

2.3.11 E2PROM



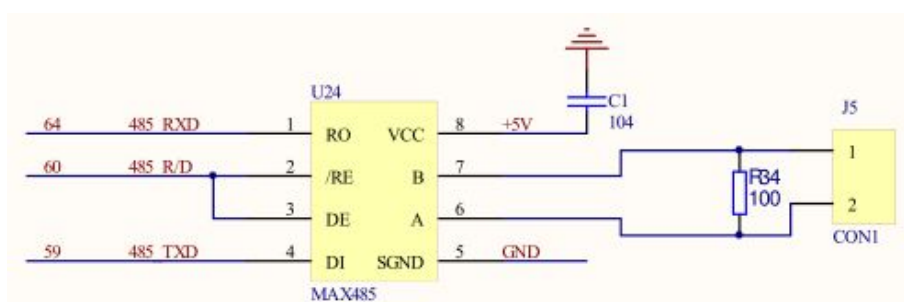
使用 16Kbit 的 I2C 接口的 E2PROM，可以存储数据和图片等。

2.3.12 温度传感器 LM75

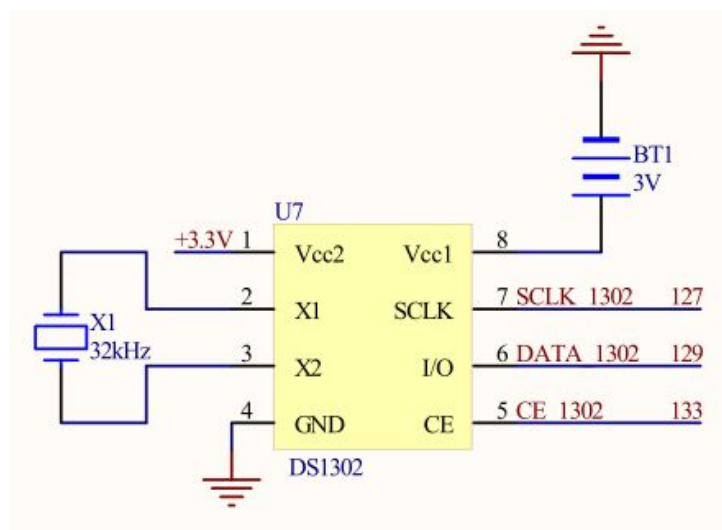


使用 I2C 接口的温度传感器 LM75A，该芯片内部自带 AD 转换，可以直接通过 I2C 接口读取内部温度数据。

2.3.13 MAX485 接口

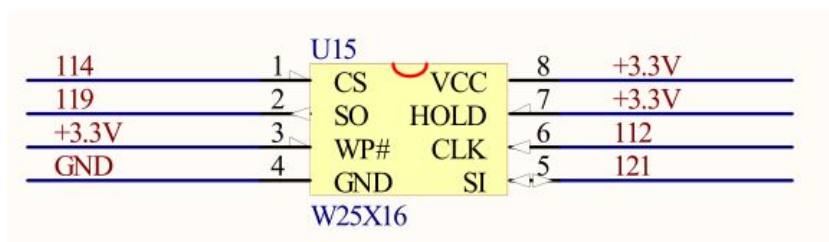


2.3.14 DS1302 时钟



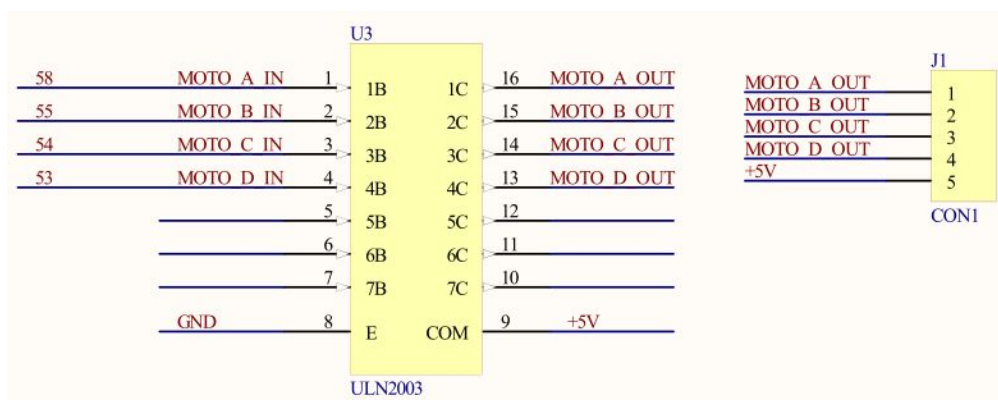
DS1302 数字时钟芯片可以产生时间和日期，可以做时钟和万年历等功能。

2.3.15 串行 FLASH



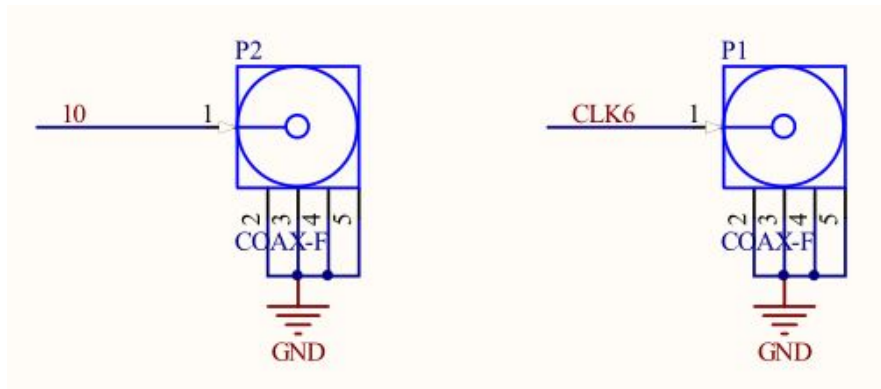
16Mbit 的 SPI 接口的串行 FLASH，可以存储程序、数据等。

2.3.16 电机驱动



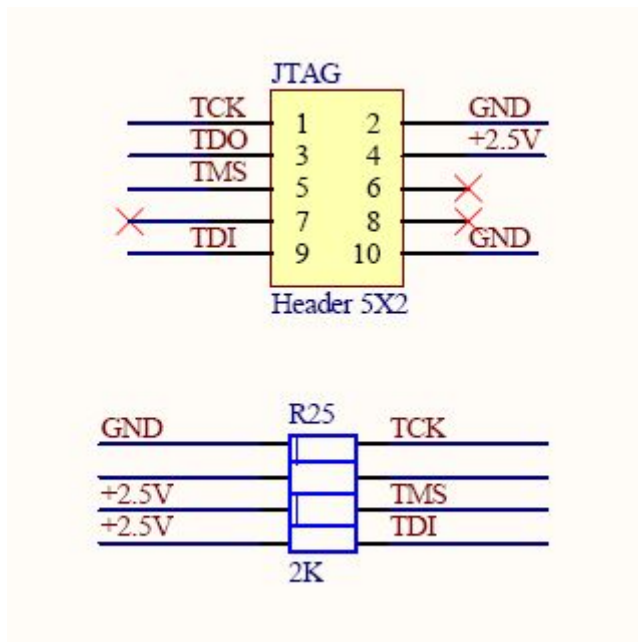
电机驱动电路，可以驱动一个步进电机或者一个直流电机。

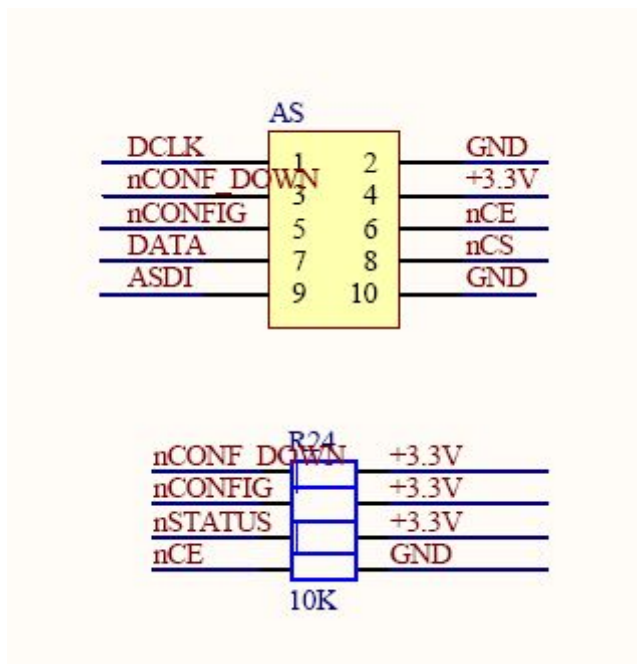
2.3.17 时钟扩展口



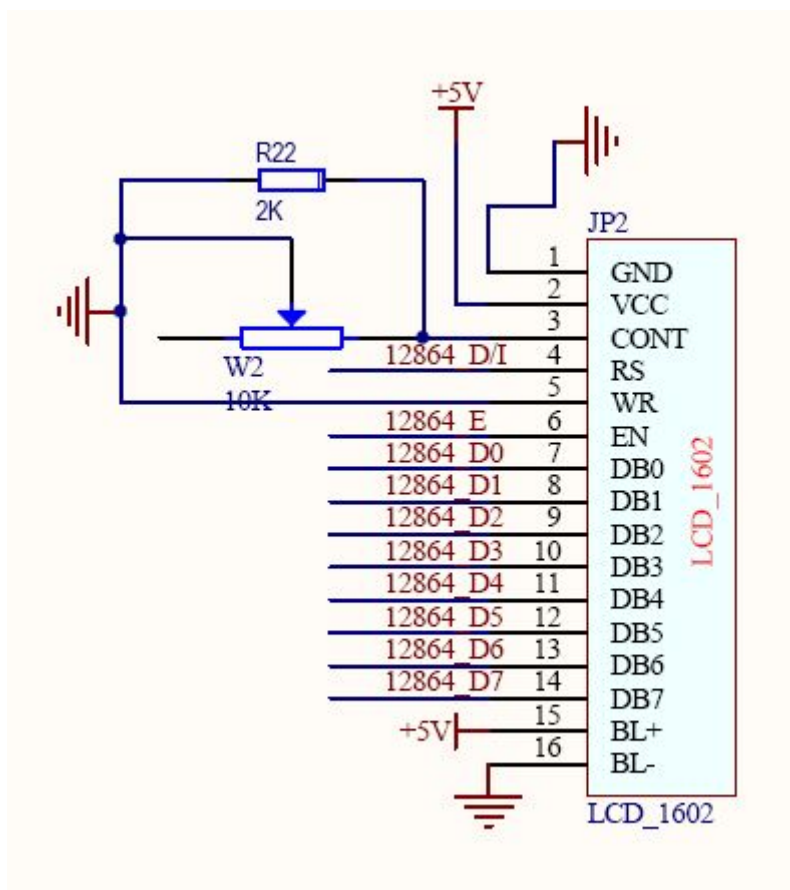
P2 是时钟扩展输出，P1 是时钟扩展输入。

2.3.18 JTAG&AS 接口





2.3.19 1602 液晶接口

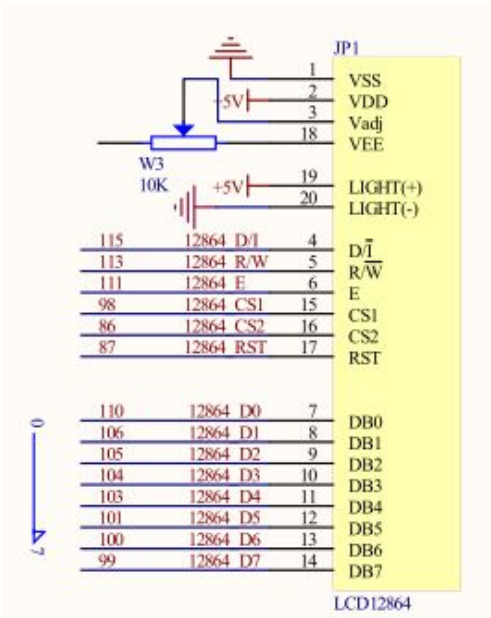


R22 和 W2 全部为调节对比度电阻，默认 W2 可调电阻没有焊接，使用 R22 作为调节对比度电阻，如果您的液晶在 R22 的调节下还是显示不清楚，可以自己焊接一个 W2 可调电阻调节（此时可以将 R22 取下），我们配套的液晶在 R22 的调节下是非常清楚的。

另外 LCD1602 液晶接口和 LCD12864 共用管脚，请避免同时使用两个液晶，另外由于液晶需要使用 5V 电源，所以在测试液晶的湿时候必须将开发板的 USB 电源接上提供 5V 电源，否则液晶不工作。

1.8 寸的 TFT 液晶也是使用 1602 液晶接口作为驱动，接口完成兼容。

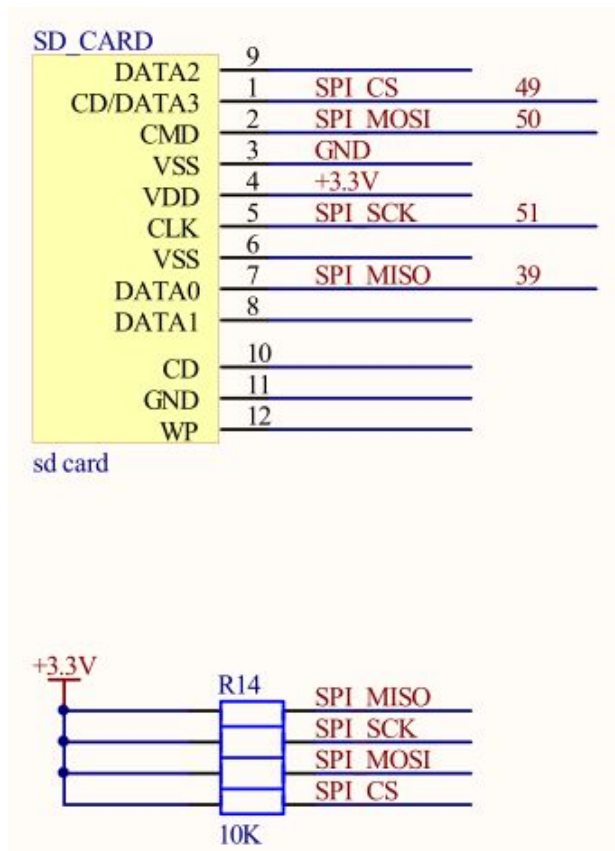
2.3.20 12864 液晶接口



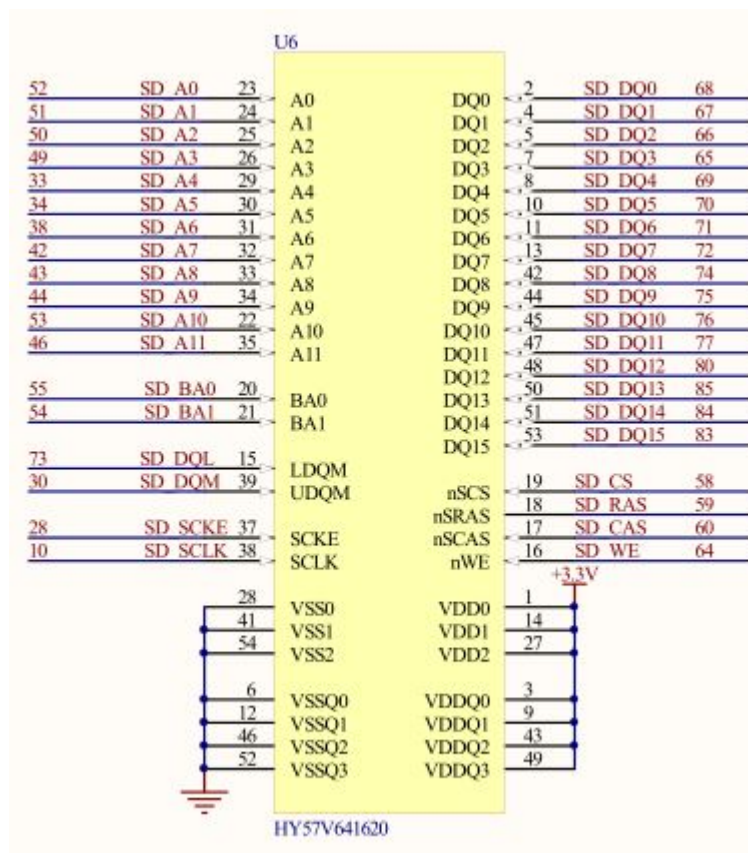
W3 为调节对比度电阻，默认 W3 可调电阻没有焊接，如果您的液晶显示不清楚，可以自己焊接一个 W3 可调电阻调节，我们配套的液晶在无 W3 的调节下是非常清楚的。

另外 LCD1602 液晶接口和 LCD12864 共用管脚，请避免同时使用两个液晶，另外由于液晶需要使用 5V 电源，所以在测试液晶的湿时候必须将开发板的 USB 电源接上提供 5V 电源，否则液晶不工作。

2.3.21 SDCARD 接口

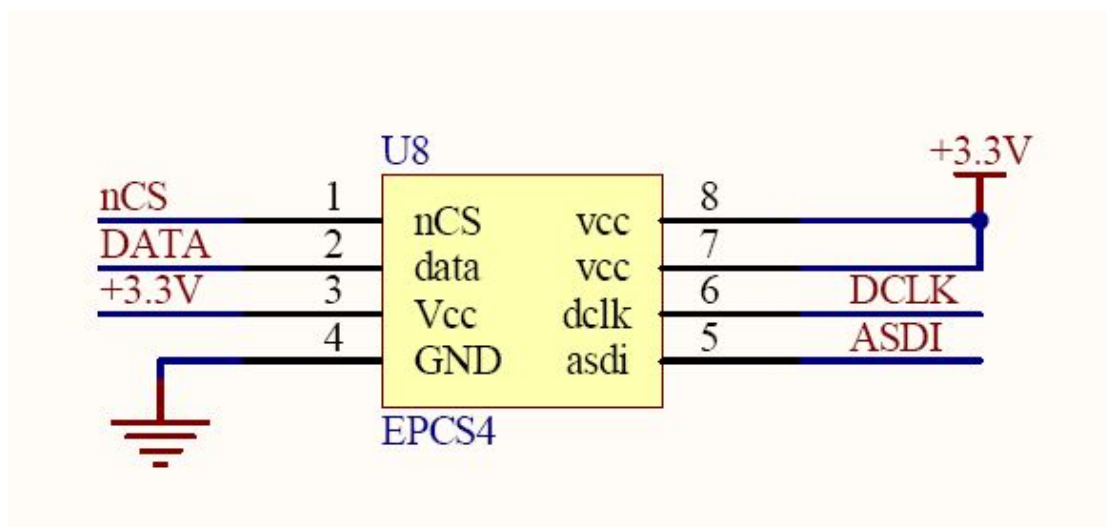


2.3.22 SDRAM



SDRAM 使用 16bit 接口, 64Mbit, 可以作为数据和程序的存储器。Qsys/Nios II 系统也可以使用 SDRAM 作为程序存储器。

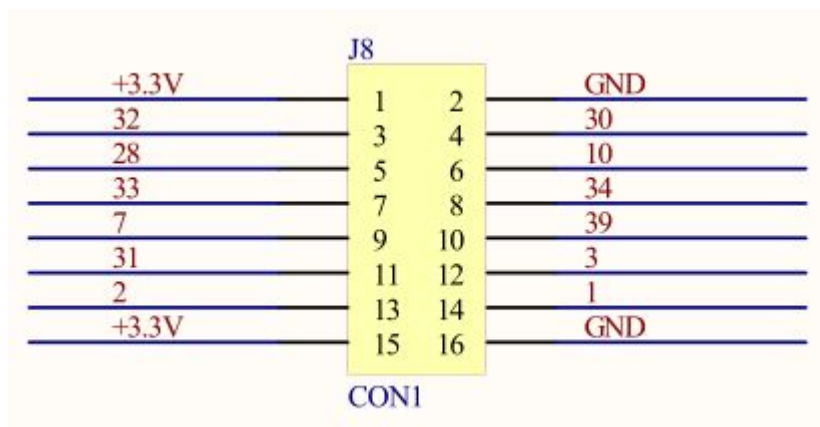
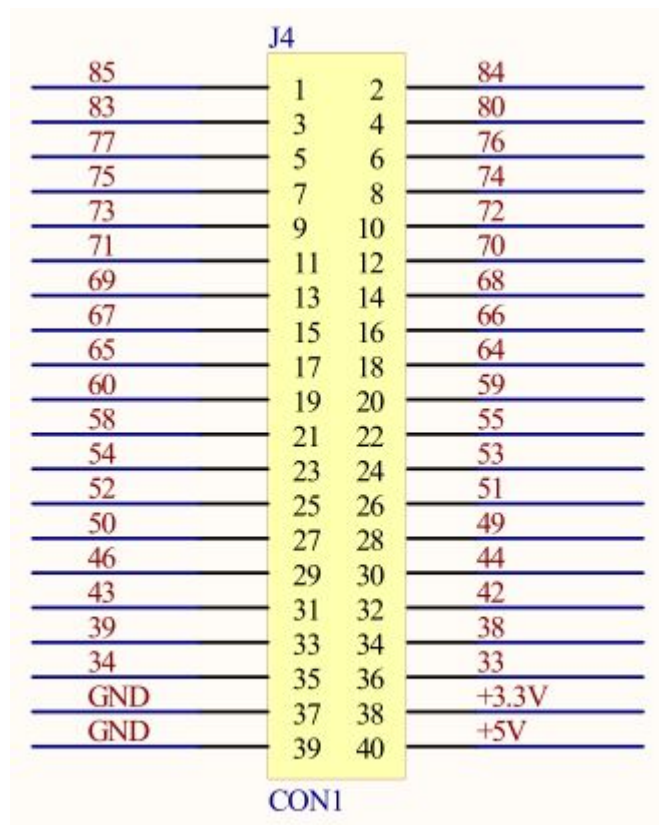
2.3.23 配置芯片



EPCS4 可以存储 FPGA 的配置数据，保证 FPGA 掉电后程序不消失，使用 AS 下载方法可以把程序下载到 EPCS4 里面。

2.3.24 扩展口

J3			
+5V	1	2	GND
+3.3V	3	4	GND
86	5	6	87
98	7	8	99
100	9	10	101
103	11	12	104
105	13	14	106
110	15	16	111
112	17	18	113
114	19	20	115
119	21	22	120
121	23	24	124
125	25	26	126
127	27	28	128
129	29	30	132
133	31	32	135
136	33	34	137
138	35	36	141
142	37	38	143
144	39	40	1
CON1			



FPGA 有两个 40PIN 扩展口，分别是 J3、J4，可以做通用的扩展，也可以插我们的开发的外设，另外一个扩展口是 J8，可以插摄像头模块、网络模块等；每个管脚在板上都有数字描述，方便扩展开发，在扩展开发的时候建议首先选用没有使用的 FPGA IO 管脚。

第三章 设计实例篇

3.2 LED 流水灯

3.2.1 LED 简介

LED (Light Emitting Diode)，发光二极管，是一种固态的半导体器件，它可以直接把电转化为光。此种元件早在 1962 年出现，早起只能发出低光度的红光，被 HP 买下专利后当作指示灯利用。及后发展出其它单色光的版本，时至今日，能够发出的光已经遍及可见光、红外线及紫外线，光度亦提高到相当高的程度。用途由初时的指示灯及显示板等。由于白光发光二极管的出现，近年逐渐发展至被普遍用作照明用途。

LED 的心脏是一个半导体的晶片，晶片的一端附在一个支架上，一端是负极，另一端连接电源的正极，整个晶片被环氧树脂封装起来。

LED 流水灯原理简介：

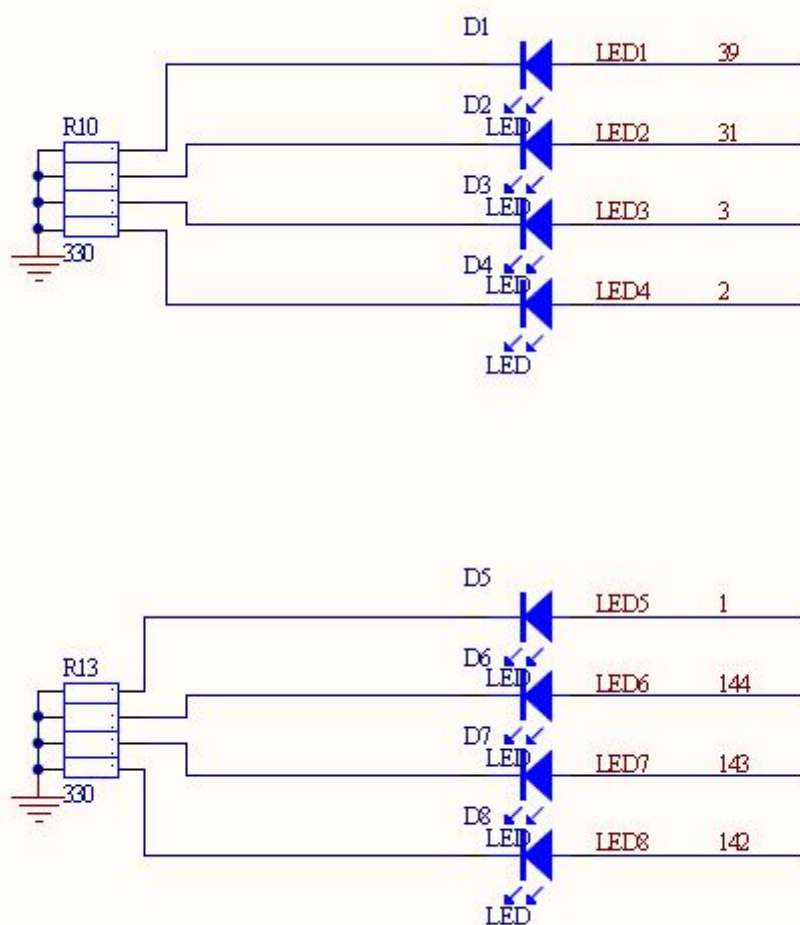
让二极管形成流水，即先点亮一个 LED1 灯点亮，等待一小段时间后熄灭，再让第二个 LED2 灯点亮，过一小段时间后再熄灭，以此类推，当最后一个 LED 灯点亮后，过一小段时间后熄灭，然后点亮 LED1，由此循环。让 LED 发光比较简单，只需要 IO 口电平发生变化即可。那么如何产生一个时间间隔呢？最常用的就是使用计数器延时来控制发光二极管。

3.2.2 实验任务

8 个 LED 依次顺序显示，产生一个流水效果，相邻 LED 发光时间间隔大约为 0.5 秒，人眼可以明显感知到这个间隔。

3.2.3 硬件设计

如下图所示, LED1~LED8 连接到 FPGA 对应的 IO 口上，通过控制 IO 口的电平变化，实现二极管的导通从而使发光二极管发光。以 D1 为例，当 39 端口为高电平（3.3V）时，LED1 发光，低电平熄灭。排阻 R10 和 R13 为限流电阻。排阻具有方向性，与色环电阻相比具有整齐、所占空间少的优势。



3.2.4 程序设计

设计思路：

前面已经提到过，通过控制 IO 口的高低电平可以实现 LED 发光或者熄灭，但是该怎样使用计数器实现延时呢？

实验要求相邻 LED 时间间隔为 0.5S，FPGA 开发板的晶振为 50MHz， $0.5S/20nS=25000000$ 。 $2^{24}<25000000<2^{25}$ ，需要 25 位计数位宽。当有效位计数达到最大值时，实现翻转（全 1 变全 0，往高位进 1）。

本实验使用两个计数器，第一个计数器实现延迟间隔，计数位宽 25bit，计数器命名 counter，第二个计数器控制哪个 LED 亮，计数器位宽 3bit，计数器命名 count。

counter 会一直持续计数，计数到最大值后，会翻转为 0；当 counter 每计数至 0 的时候，count 加 1，当 count 计数到最大值后，也会翻转为 0。

源代码：

```

/*****Copyright (c)*****
**                               Adong   Studio

```

手把手教您学习 FPGA

```
**
**-----File
Info-----
** File name:          LED
** Last modified Date: 2015-6-1
** Last Version:      1.1
** Descriptions:      LED
**-----

-----
** Created by:        Adong
** Created date:      2015-6-1
** Version:           1.0
** Descriptions:      The original version
**
**-----

-----
** Modified by:
** Modified date:
** Version:
** Descriptions:
**
**-----

-----
*****/
module LED(
    //input
    input      sys_clk          ,//system clock;
    input      sys_rst_n        ,//system reset,low is active;
    //output
    output reg[7:0] LED
);

//reg define
reg      [ 2:0]      count      ;
reg      [24:0]      counter    ;

//*****
//**
//**                      Main Program
//*****

//count for add counter,0.5s/20ns=25000000,need 25 bit cnt
always @(posedge sys_clk or negedge sys_rst_n) begin
```

手把手教您学习 FPGA

```
        if ( sys_rst_n == 1'b0 )
            counter <= 25'b0;
        else
            counter <= counter + 25'b1;
    end

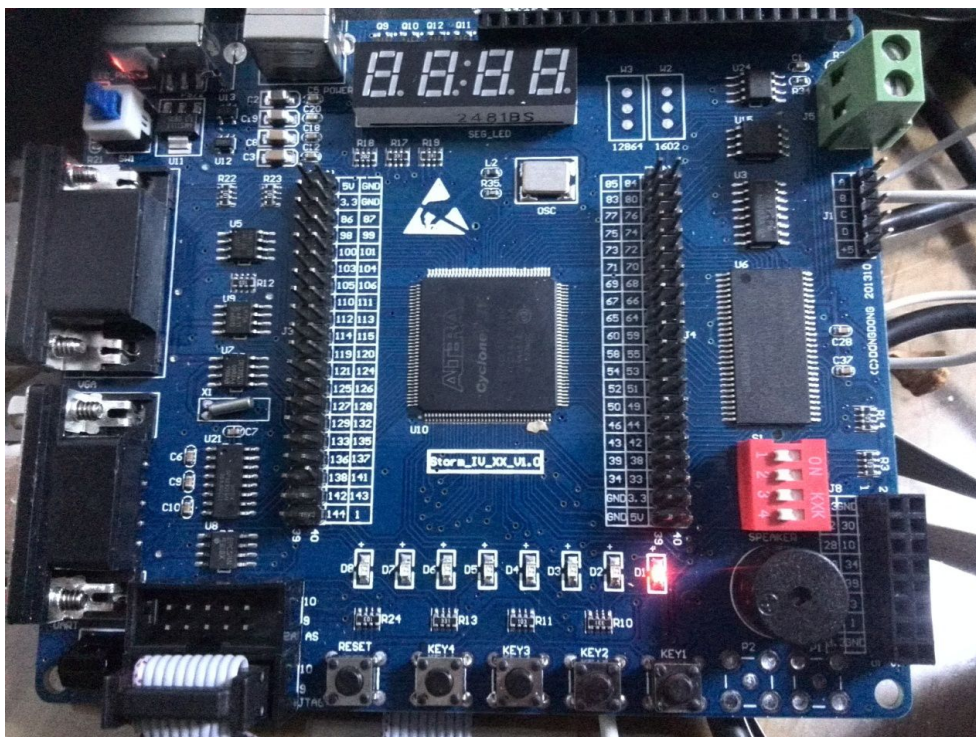
always @(posedge sys_clk or negedge sys_rst_n) begin
    if ( sys_rst_n == 1'b0 )
        count <= 3'b0;
    else if( counter == 25'd0 )
        count <= count + 3'b1;
    else ;
end

//ctr the LED pipeline display when count is equal to 0,1..
always @(posedge sys_clk or negedge sys_rst_n) begin
    if ( sys_rst_n == 1'b0 )
        LED <= 8'b0;
    else begin
        case ( count )    //通过控制 IO 口的高低电平实现发光二极管的亮灭
            0      : LED = 8'b00000001    ;
            1      : LED = 8'b00000010    ;
            2      : LED = 8'b00000100    ;
            3      : LED = 8'b00001000    ;
            4      : LED = 8'b00010000    ;
            5      : LED = 8'b00100000    ;
            6      : LED = 8'b01000000    ;
            7      : LED = 8'b10000000    ;
            default: LED = 8'b00000000    ;
        endcase
    end
end

endmodule
//end of rtl code
```

3.2.5 实验现象

将程序下载到开发板后，可观察到 D8~D1 从左至右轮流点亮，形成流水效果。需要注意的是当 D1 点亮后，会经过较长的时间 D8 才重新开始循环。

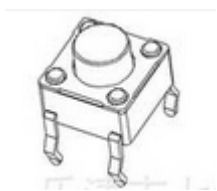


3.3 按键控制 LED

3.3.1 按键控制 LED 简介

按键是最常用的输入设备，广泛用于单片机/FPGA/DSP 的控制输入，操作人员可以通过按键输入数据或者命令，实现简单的人机对话。FPGA 开发板使用的按键是一种常开型的开关，通常按键的两个触点不按下时处于断开状态，按下时按键处于闭合状态。

下图为按键实物图。

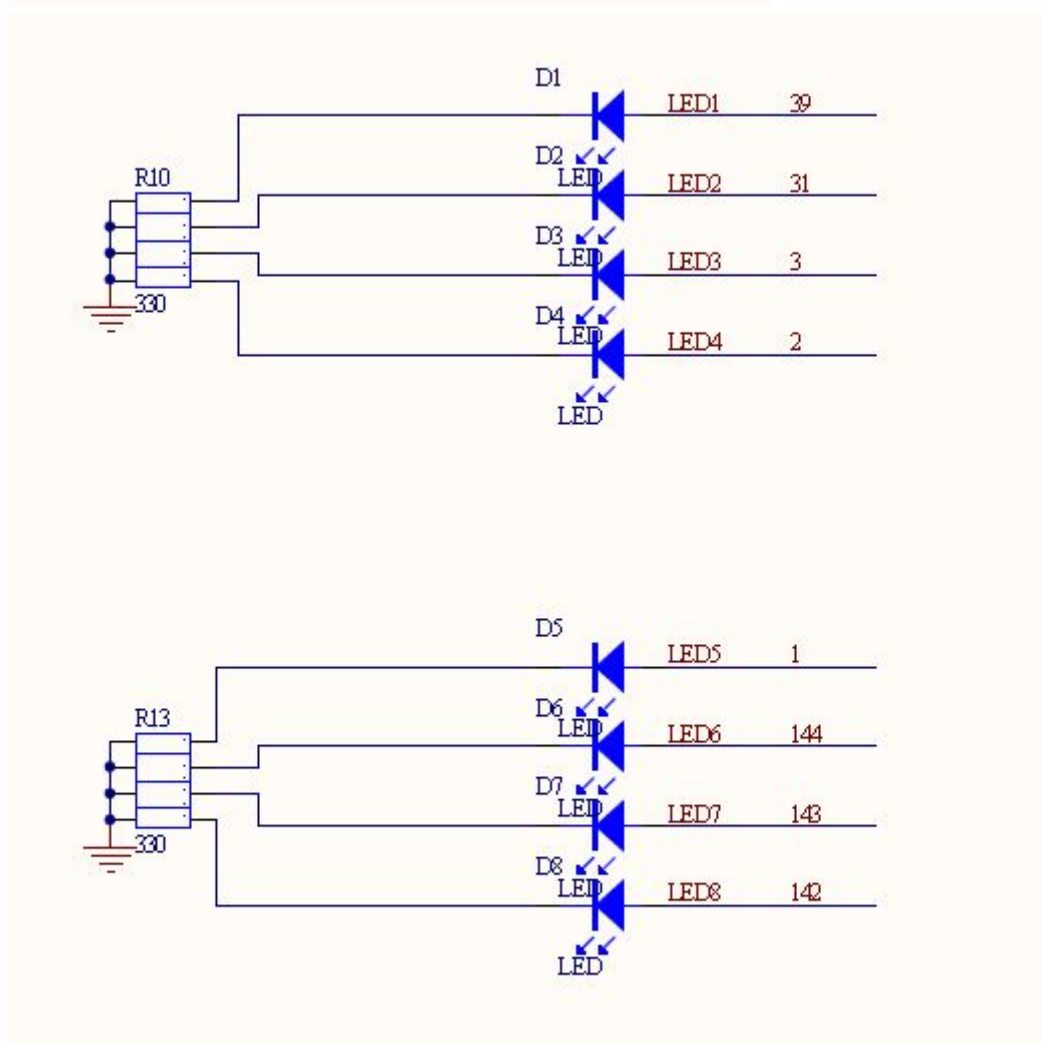
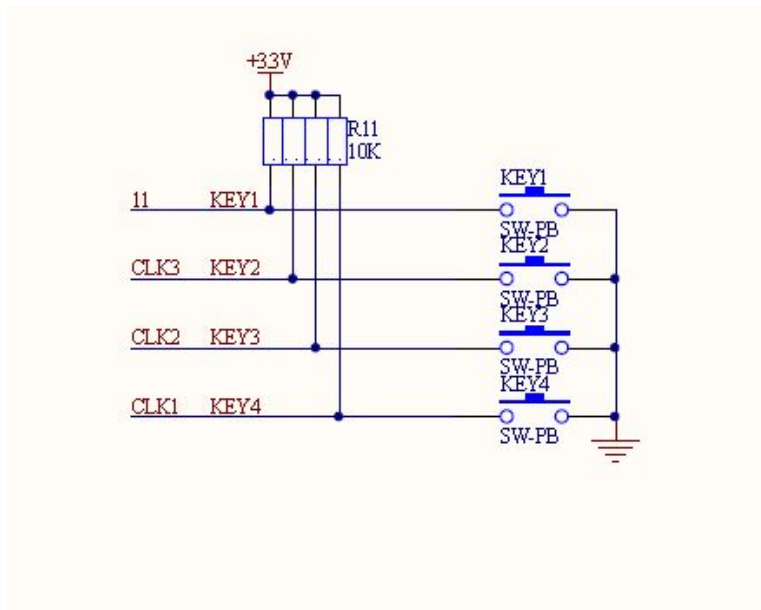


本节实验使用按键控制发光二极管。按键默认不按下时由于上拉电阻原因输出高电平，发光二极管输入为高时发光，所以需要按键输入取反后赋值给发光二极管输入端，即可控制二极管的发光。按键不按下时候 LED 处于熄灭状态，按键按下时候 LED 处于点亮状态。

3.3.2 实验任务

一个按键控制两个发光二极管，按下按键可以使两个 LED 发光，不按下时两个 LED 不发光。

3.3.3 硬件设计



3.3.4 程序设计

设计思路：

输入为四个按键，输出为 8 个 LED，按键按下时按键输入为低电平，而 LED 需要驱动为高电平才能点亮，所以需要对接键输入进行取反作为 LED 的输入。

源代码：

```
module KeyToLED(
//input
input      [3:0]      key ,    //开发板上的四个按键控制 8 个 LED
//output
output wire  [7:0]      LED
// 特别说明：什么时候用 wire 类型，什么时候用 reg 类型？
// 判断原则：在使用 always 时，不管有没有时钟，都必须使用 reg 定义（一种是带时钟的 always 块，一种是不带时钟的 always 块，不带时钟时使用阻塞“=”赋值，带时钟使用非阻塞赋值（<=））；
// assign 使用 wire 类型定义。
);

//reg define

//parameter define

/*****
**                               Main Program
**
**
*****/

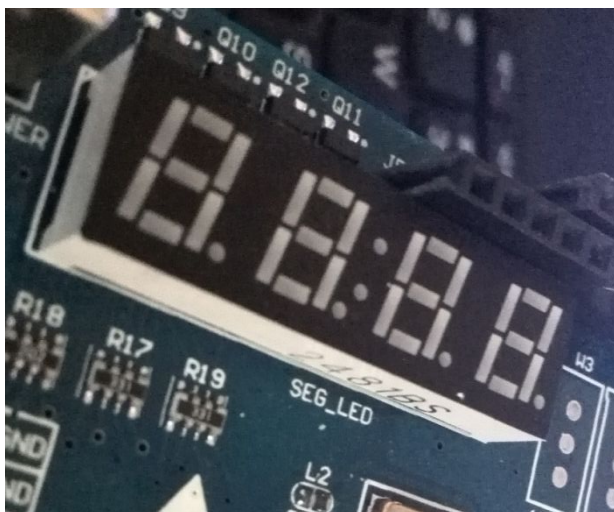
// key 0 ctrl LED 0 & LED 4
// key 1 ctrl LED 1 & LED 5
// key 2 ctrl LED 2 & LED 6
// key 3 ctrl LED 3 & LED 7

// when key is pressed , key input low level, else key input high level
assign LED = ~{ key, key } ;           //key value display in beep
//即 assign LED=~{key[3],key[2],key[1],key[0],key[3],key[2],key[1],key[0],};

endmodule
```

3.3.5 实验现象

如下图所示，将程序下载到开发板上，按下 key3, 则相对应的 D3 和 D7 亮



七段数码管原理：

七段数码管由 7 个发光二极管组成，此外，还有一个圆点型发光二极管（在图中以 dp 表示），用于显示小数点。通过七段发光二极管亮暗的不同组合，可以显示多种数字、字母以及其它符号。

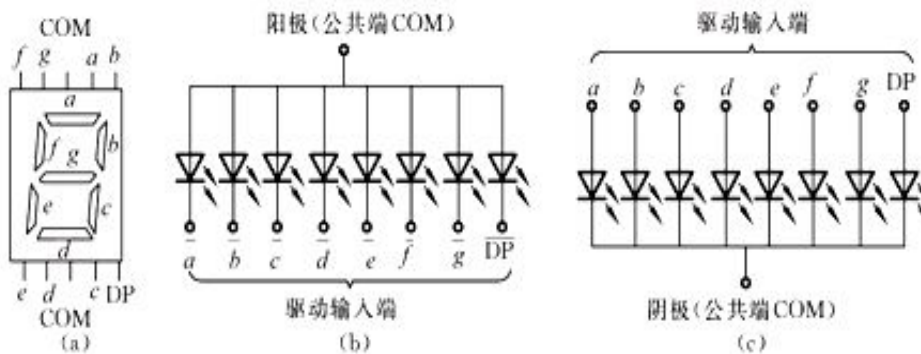
LED 数码管分类：

1) 共阴极接法：叫共阴数码管

把发光二极管的阴极连在一起构成公共阴极。使用时公共阴极接地，这样阳极端输入高电平的段发光二极管就导通点亮，而输入低电平的则不点亮。

2) 共阳极接法：叫共阳数码管

把发光二极管的阳极连在一起构成公共阳极。使用时公共阳极接 +5V。这样阴极端输入低电平的段发光二极管就导通点亮，而输入高电平的则不点亮。



本次实验使用的是共阳极数码管，例如：假如要显示数字 1，则 b、c 接低电平。

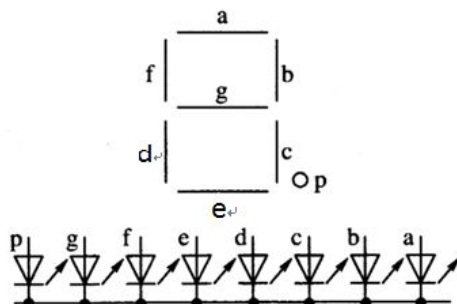
数码管驱动方式：

数码管有直流驱动和动态显示驱动两种驱动方式。直流驱动是指是指每个数码管的每一个段码都由一个单片机或者 FPGA 的 I/O 端口进行驱动，或者使用如 BCD 码二-十进制译码器译码进行驱动。优点是编程简单，显示亮度高，缺点是占用 I/O 端口多。动态显示驱动是将所有数码管通过分时轮流控制各个数码管的 COM 端，就使各个数码管轮流受控显示。将所有数码管的 8 个显示笔划“a, b, c, d, e, f, g, dp”的同名端连在一起，另外为每个数码管的公共极 COM 增加位选通控制电路，位选通由各自独立的 I/O 线控制，当单片机或者 FPGA 的 I/O 口输出字形码时，所有数码管都接收到

手把手教您学习 FPGA

相同的字形码，但究竟是那个数码管会显示出字形，取决于单片机或者 FPGA 对位选通 COM 端电路的控制，所以我们只要将需要显示的数码管的选通控制打开，该位就显示出字形，没有选通的数码管就不会亮。本次实验利用 FPGA 的 IO 口及三极管来驱动数码管，为静态直流驱动方式。

显示举例：



拿显示 8 举例子：

1 1 1 0 1 1 1 1 对应：
e g f dp c d b a

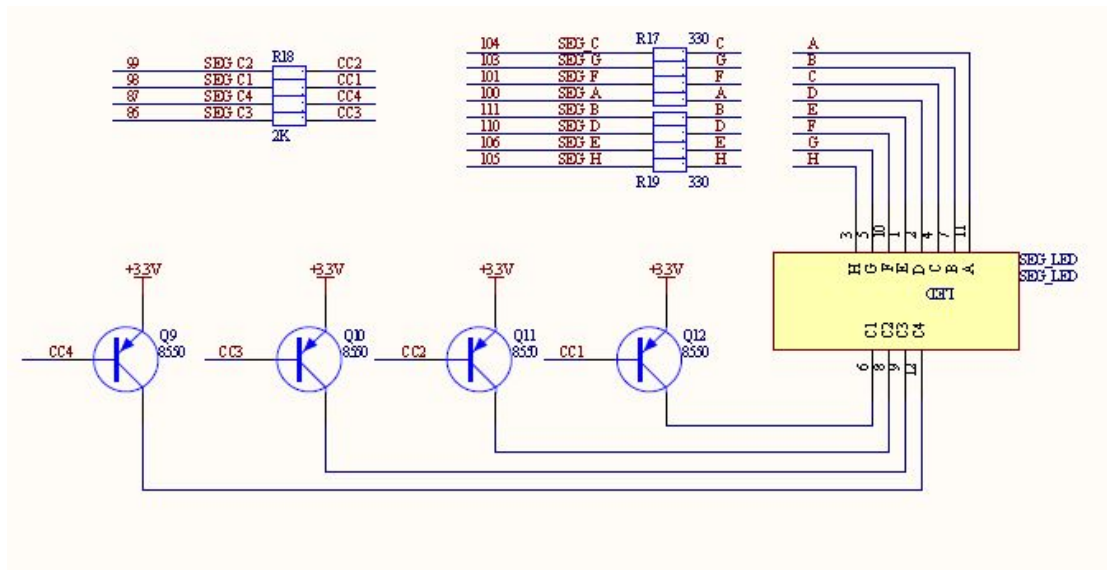
数码管电气特性：

LED 数码管中各段发光二极管的伏安特性与普通二极管类似，只是正向压降与正向电阻较大。在一定范围内，其正向电流与亮度成正比。由于常规数码管的起辉电流比较只有 1~2mA，最大限电流也只有 10~30mA。我们在实验中使用了排阻来限制电流。共阳与共阴与电路接线密切相关，决定了驱动电路的接法，因此在设计电路前要考虑好数码管的选型，否则就会可能不能实现要显示的效果。共阳极的比较好驱动，共阴极比较好编程。

3.4.2 实验任务

编写逻辑使四个数码管从 0~9 循环计数，显示时间间隔为 1 秒。

3.4.3 硬件设计



R17 和 R19 为限流电阻，各位数码管的共阳极由 FPGA 控制 Q9—Q12 来实现 4 位数码管的位输出控制。本次实验数码管是共阳极。

3.4.4 程序设计

设计思路：

逻辑主要有两部分构成：

一个是对时间的控制，以控制数码管跳变的时间，并实现四位数码管从 0~9 的循环，该部分可由计数器实现；

另外一个是对数码管译码的控制，使数码管循环显示 0~9 这十个数字。

源代码：

```
module SegLED (
//input
input          sys_clk      ,
input          sys_rst_n    ,

//output
output wire    seg_c1       ,    //输入低电平使三极管导通
output wire    seg_c2       ,
output wire    seg_c3       ,
output wire    seg_c4       ,

output reg     seg_a         ,
output reg     seg_b         ,
output reg     seg_c         ,
output reg     seg_e         ,
output reg     seg_d         ,
```

手把手教您学习 FPGA

```
output reg          seg_f          ,
output reg          seg_g          ,
output reg          seg_h
);

//parameter define
parameter SIZE = 8;

//reg define
reg    [3:0]        counter        ; //控制逻辑产生 0-9 数字
reg    [SIZE-1:0]   disp_data      ;

reg          disp_clk              ;

reg    [25:0]       clk_cnt        ; //控制时间，2^26=67108864>50000000

reg          segled_a              ; // 数码管段位定义
reg          segled_b              ;
reg          segled_c              ;
reg          segled_e              ;
reg          segled_d              ;
reg          segled_f              ;
reg          segled_g              ;
reg          segled_h              ;

//wire define

/*****
**                               Main Program
**
*****/

//使用计数器控制数码管延时，复位或计数到 1s 时，重新开始计数。1S = 50000000 * 20ns
always @(posedge sys_clk or negedge sys_rst_n) begin
    if (sys_rst_n ==1'b0)
        clk_cnt <= 26'b0;
    else if ( clk_cnt == 26'd50000000 )
        clk_cnt <= 26'b0;
    else
        clk_cnt <= clk_cnt + 26'b1;
end

//利用 clk_cnt 控制 counter 产生，实现 0~9 循环，用来控制数码管显示。RESET 清零。
```

手把手教您学习 FPGA

```
always @(posedge sys_clk or negedge sys_rst_n) begin
    if (sys_rst_n == 1'b0)
        counter <= 4'd0;
    else if ( clk_cnt == 26'd500000000 && counter == 4'd9 )
        counter <= 4'b0;
    else if ( clk_cnt == 26'd500000000 )
        counter <= counter + 4'b1;
    else ;
end
```

//对位段进行译码，显示 0~9

// control SEGLED disp 0-9

```
always @(*) begin
    case (counter)
        9          :
            begin
                segled_a = 1 ;
                segled_b = 1 ;
                segled_c = 1 ;
                segled_e = 0 ;
                segled_d = 0 ;
                segled_f = 1 ;
                segled_g = 1 ;
                segled_h = 0 ;
            end
        8          :
            begin
                segled_a = 1 ;
                segled_b = 1 ;
                segled_c = 1 ;
                segled_e = 1 ;
                segled_d = 1 ;
                segled_f = 1 ;
                segled_g = 1 ;
                segled_h = 1 ;
            end
        7          :
            begin
                segled_a = 1 ;
                segled_b = 1 ;
                segled_c = 1 ;
                segled_e = 0 ;
                segled_d = 0 ;
```

```
        segled_f = 0 ;
        segled_g = 0 ;
        segled_h = 0 ;
    end
6      :
    begin
        segled_a = 1 ;
        segled_b = 0 ;
        segled_c = 1 ;
        segled_e = 1 ;
        segled_d = 1 ;
        segled_f = 1 ;
        segled_g = 1 ;
        segled_h = 0 ;
    end
5      :
    begin
        segled_a = 1 ;
        segled_b = 0 ;
        segled_c = 1 ;
        segled_e = 0 ;
        segled_d = 1 ;
        segled_f = 1 ;
        segled_g = 1 ;
        segled_h = 0 ;
    end
4      :
    begin
        segled_a = 0 ;
        segled_b = 1 ;
        segled_c = 1 ;
        segled_e = 0 ;
        segled_d = 0 ;
        segled_f = 1 ;
        segled_g = 1 ;
        segled_h = 0 ;
    end
3      :
    begin
        segled_a = 1 ;
        segled_b = 1 ;
        segled_c = 1 ;
        segled_e = 0 ;
```

```
        segled_d =1 ;
        segled_f = 0 ;
        segled_g = 1 ;
        segled_h = 0 ;
    end

    2      :
    begin
        segled_a = 1 ;
        segled_b = 1 ;
        segled_c = 0 ;
        segled_e = 1 ;
        segled_d =1 ;
        segled_f = 0 ;
        segled_g = 1 ;
        segled_h = 0 ;
    end

    1      :
    begin
        segled_a = 0 ;
        segled_b = 1 ;
        segled_c = 1 ;
        segled_e = 0 ;
        segled_d =0 ;
        segled_f = 0 ;
        segled_g = 0 ;
        segled_h = 0 ;
    end

    0      :
    begin
        segled_a = 1 ;
        segled_b = 1 ;
        segled_c = 1 ;
        segled_e = 1 ;
        segled_d = 1 ;
        segled_f = 1 ;
        segled_g = 0 ;
        segled_h = 1 ;
    end

    default :
    begin
        segled_a = 0 ;
        segled_b = 0 ;
        segled_c = 0 ;
```



```

                                segled_e = 0 ;
                                segled_d = 0 ;
                                segled_f = 0 ;
                                segled_g = 0 ;
                                segled_h = 0 ;
                                end
                                endcase
                                end

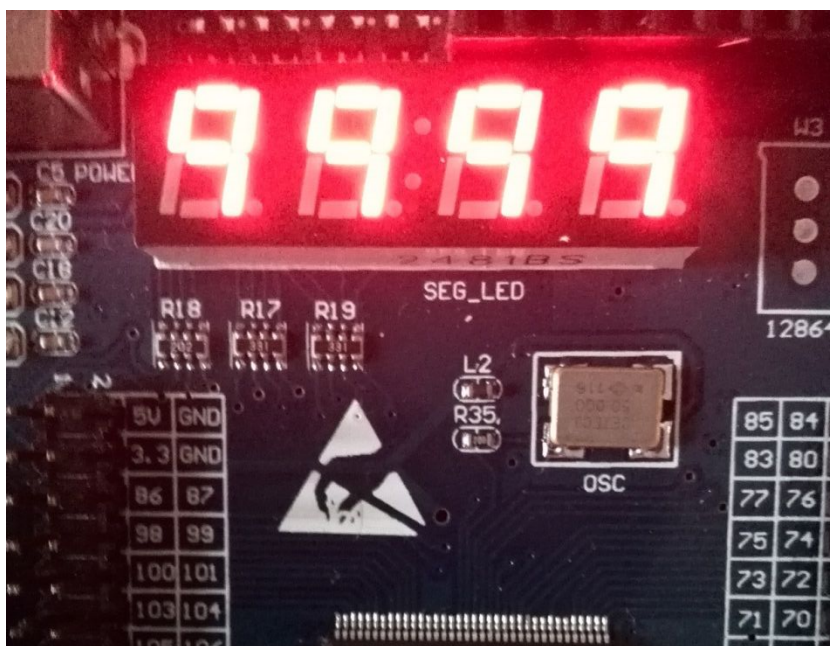
//组合逻辑进行取反，数码管为共阳极，对译码段位取反，低电平有效
always @(*) begin
    seg_a = ~segled_a ;
    seg_b = ~segled_b ;
    seg_c = ~segled_c ;
    seg_e = ~segled_e ;
    seg_d = ~segled_d ;
    seg_f = ~segled_f ;
    seg_g = ~segled_g ;
    seg_h = ~segled_h ;
end

// 四个片选全部选通，四个位选输出位 0，控制三极管，驱动数码管的四个片选全部为 1
assign seg_c1 = 1'b0;
assign seg_c2 = 1'b0;
assign seg_c3 = 1'b0;
assign seg_c4 = 1'b0;

endmodule
//end of RTL code
```

3.4.5 实验现象

程序下载到开发板后，可以看到数码管从 0~9 循环显示。



3.5 七段数码管 动态扫描

3.5.1 动态扫描简介

动态扫描就是轮流向各位数码管送出字形码和响应的位选，只要扫描显示速度足够快，利用发光管的余辉和人眼视觉暂留作用，使人感觉好像各位数码管都在显示。

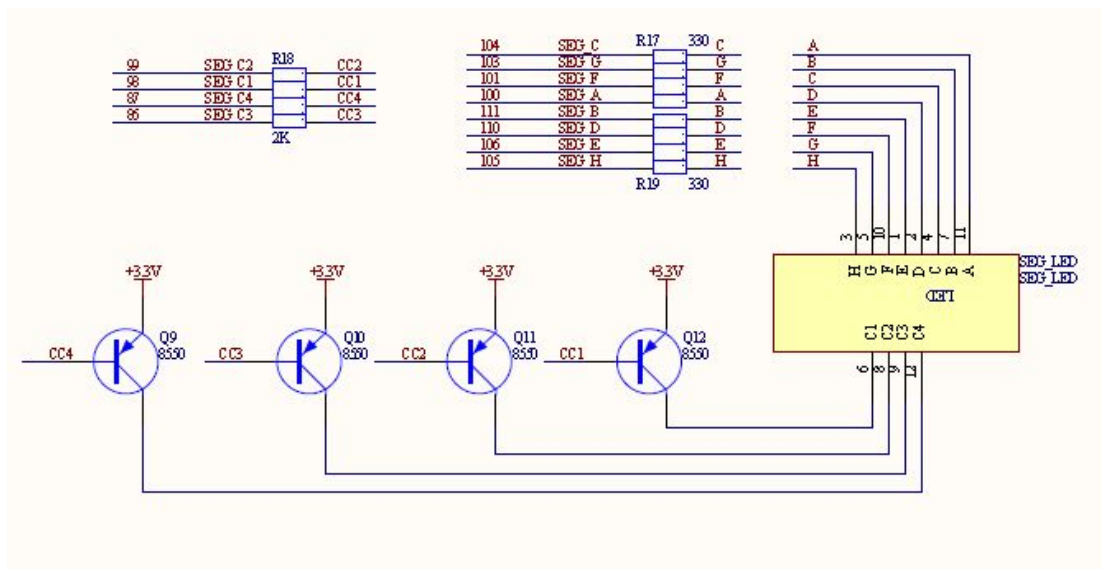
动态显示的特点是将所有位数码管的段选线并联在一起，由位选线控制是哪一位数码管有效。这样一来，就没有必要每一位数码管配一个锁存器，从而大大地简化了硬件电路。选亮数码管采用动态扫描显示。动态显示的亮度比静态显示要差一些，所以在选择限流电阻时应略小于静态显示电路中的。本实验中的限流电阻和静态的一样，影响不是很大。

数码管不同位显示的时间间隔可以通过调整延时程序的延时长短来完成。数码管显示的时间间隔也能够确定数码管显示时的亮度，若显示的时间间隔长，显示时数码管的亮度将亮些，若显示的时间间隔短，显示时数码管的亮度将暗些。若显示的时间间隔过长的话，数码管显示时将产生闪烁现象。所以，在调整显示的时间间隔时，即要考虑到显示时数码管的亮度，又要考虑数码管显示时不闪烁的现象。一般控制在 1ms 左右最佳。本次实验设计时开发板的刷新时间即为 1ms。

3.5.2 实验任务

数码管的四位动态显示每个不同的值。

3.5.3 硬件设计



R17 和 R19 为限流电阻，各位数码管的共阳极由 FPGA 控制 Q9—Q12 来实现 4 位数码管的位输出控制。本次实验数码管是共阳极。

3.5.4 程序设计

设计思路：

对于数码管动态扫描显示需要几个计数器来控制：

- 1、scan_cnt 扫描计数器，用来动态选通四个位选信号，形成动态显示效果。
- 2、clk_cnt 计数器，控制每位数码管的显示延迟。
- 3、counter 计数器，产生 0-9 个数字，控制数码管的显示数字。

源代码：

```
module SegLed_DynamDisp(
//input
input          sys_clk          ,
input          sys_rst_n        ,

//output
output wire    seg_c1           ,
output wire    seg_c2           ,
output wire    seg_c3           ,
output wire    seg_c4           ,

output reg     seg_a             ,
output reg     seg_b             ,
output reg     seg_c             ,
output reg     seg_e             ,
output reg     seg_d             ,
```

手把手教您学习 FPGA

```
output reg          seg_f          ,
output reg          seg_g          ,
output reg          seg_h
);

//parameter define
parameter SIZE = 8;

//reg define
reg    [3:0]        counter        ;
reg    [ 3:0]        disp_data      ;

reg          disp_clk              ;

reg    [25:0]        clk_cnt        ;

reg    [15:0]        scan_cnt        ;
reg    [ 3:0]        segled_bit_sel  ;

reg          segled_a              ;
reg          segled_b              ;
reg          segled_c              ;
reg          segled_e              ;
reg          segled_d              ;
reg          segled_f              ;
reg          segled_g              ;
reg          segled_h              ;

//wire define

/*****
**                               Main Program
**
*****/
// 产生一个 1ms 左右的计数，作为动态选通四个位选信号
always @(posedge sys_clk or negedge sys_rst_n) begin
    if (sys_rst_n == 1'b0)
        scan_cnt <= 16'b0;
    else
        scan_cnt <= scan_cnt + 16'b1;          //20ns* 2^16= 1310720ns 约等于 1.3ms
end

// 产生数码管位选，使用 scan_cnt 高 2 bit 进行控制
```

手把手教您学习 FPGA

```
always @(posedge sys_clk or negedge sys_rst_n) begin
    if (sys_rst_n == 1'b0)                                //复位时从数码管低位开始扫描
        segled_bit_sel <= 4'b0001;
    else if ( scan_cnt[15:14] == 2'b00 )
        //扫描计数 0000 0000 0000 0000~1111 1111 1111 1100, 即每个扫描周期从 0 至 0.325ms 时, 点
        //亮第一个数码管。
        segled_bit_sel <= 4'b0001;
    else if ( scan_cnt[15:14] == 2'b01 )
        //0.325ms~0.65ms 时, 点亮第二个数码管
        segled_bit_sel <= 4'b0010;
    else if ( scan_cnt[15:14] == 2'b10 )
        //0.65ms~0.975ms 时, 点亮第三个数码管
        segled_bit_sel <= 4'b0100;
    else if ( scan_cnt[15:14] == 2'b11 )
        //0.975ms 至 1.3ms 时, 点亮第四个数码管
        segled_bit_sel <= 4'b1000;
    else ;
end

//使用计数器控制数码管延时, 复位或计数到 1s 时, 重新开始计数。1S = 50000000 * 20ns
always @(posedge sys_clk or negedge sys_rst_n) begin
    if (sys_rst_n == 1'b0)
        clk_cnt <= 26'b0;
    else if ( clk_cnt == 26'd50000000 )
        clk_cnt <= 26'b0;
    else
        clk_cnt <= clk_cnt + 26'b1;
end

//利用 clk_cnt 控制 counter 产生, 实现 0~9 循环, 用来控制数码管显示。RESET 清零。
always @(posedge sys_clk or negedge sys_rst_n) begin
    if (sys_rst_n == 1'b0)
        counter <= 4'd0;
    else if ( clk_cnt == 26'd50000000 && counter == 4'd9 )
        counter <= 4'b0;
    else if ( clk_cnt == 26'd50000000 )
        counter <= counter + 4'b1;
    else ;
end

// 让数码管四个位同时输出不同的数字
always @(*) begin
    if ( segled_bit_sel == 4'b0001 )
```

手把手教您学习 FPGA

```
        disp_data = counter + 4'd1 ;           //低 1 位接收到高电平时则加 1；
    else if ( segled_bit_sel == 4'b0010 )
        disp_data = counter + 4'd2 ;           //低 2 位数码管接收到高电平时加 2；
    else if (segled_bit_sel == 4'b0100 )
        disp_data = counter + 4'd3 ;           //低 3 位数码管接收到高电平时加 3；
    else
        disp_data = counter + 4'd4 ;           //低四位数码管接收到高电平时加 4；
end

//对位段进行译码，显示 0~9
// control  SEGLED disp 0-9
always @(*) begin
    case (counter)
        9          :
            begin
                segled_a = 1 ;
                segled_b = 1 ;
                segled_c = 1 ;
                segled_e = 0 ;
                segled_d = 0 ;
                segled_f = 1 ;
                segled_g = 1 ;
                segled_h = 0 ;
            end
        8          :
            begin
                segled_a = 1 ;
                segled_b = 1 ;
                segled_c = 1 ;
                segled_e = 1 ;
                segled_d = 1 ;
                segled_f = 1 ;
                segled_g = 1 ;
                segled_h = 1 ;
            end
        7          :
            begin
                segled_a = 1 ;
                segled_b = 1 ;
                segled_c = 1 ;
                segled_e = 0 ;
                segled_d = 0 ;
                segled_f = 0 ;
```

```
        segled_g = 0 ;
        segled_h = 0 ;
    end

    6      :
        begin
            segled_a = 1 ;
            segled_b = 0 ;
            segled_c = 1 ;
            segled_e = 1 ;
            segled_d = 1 ;
            segled_f = 1 ;
            segled_g = 1 ;
            segled_h = 0 ;
        end

    5      :
        begin
            segled_a = 1 ;
            segled_b = 0 ;
            segled_c = 1 ;
            segled_e = 0 ;
            segled_d = 1 ;
            segled_f = 1 ;
            segled_g = 1 ;
            segled_h = 0 ;
        end

    4      :
        begin
            segled_a = 0 ;
            segled_b = 1 ;
            segled_c = 1 ;
            segled_e = 0 ;
            segled_d =0 ;
            segled_f = 1 ;
            segled_g = 1 ;
            segled_h = 0 ;
        end

    3      :
        begin
            segled_a = 1 ;
            segled_b = 1 ;
            segled_c = 1 ;
            segled_e = 0 ;
            segled_d =1 ;
```

```
        segled_f = 0 ;
        segled_g = 1 ;
        segled_h = 0 ;
    end

    2      :
    begin
        segled_a = 1 ;
        segled_b = 1 ;
        segled_c = 0 ;
        segled_e = 1 ;
        segled_d =1 ;
        segled_f = 0 ;
        segled_g = 1 ;
        segled_h = 0 ;
    end

    1      :
    begin
        segled_a = 0 ;
        segled_b = 1 ;
        segled_c = 1 ;
        segled_e = 0 ;
        segled_d =0 ;
        segled_f = 0 ;
        segled_g = 0 ;
        segled_h = 0 ;
    end

    0      :
    begin
        segled_a = 1 ;
        segled_b = 1 ;
        segled_c = 1 ;
        segled_e = 1 ;
        segled_d = 1 ;
        segled_f = 1 ;
        segled_g = 0 ;
        segled_h = 1 ;
    end

    default :
    begin
        segled_a = 0 ;
        segled_b = 0 ;
        segled_c = 0 ;
        segled_e = 0 ;
```



```

        segled_d = 0 ;
        segled_f = 0 ;
        segled_g = 0 ;
        segled_h = 0 ;
    end

endcase
end

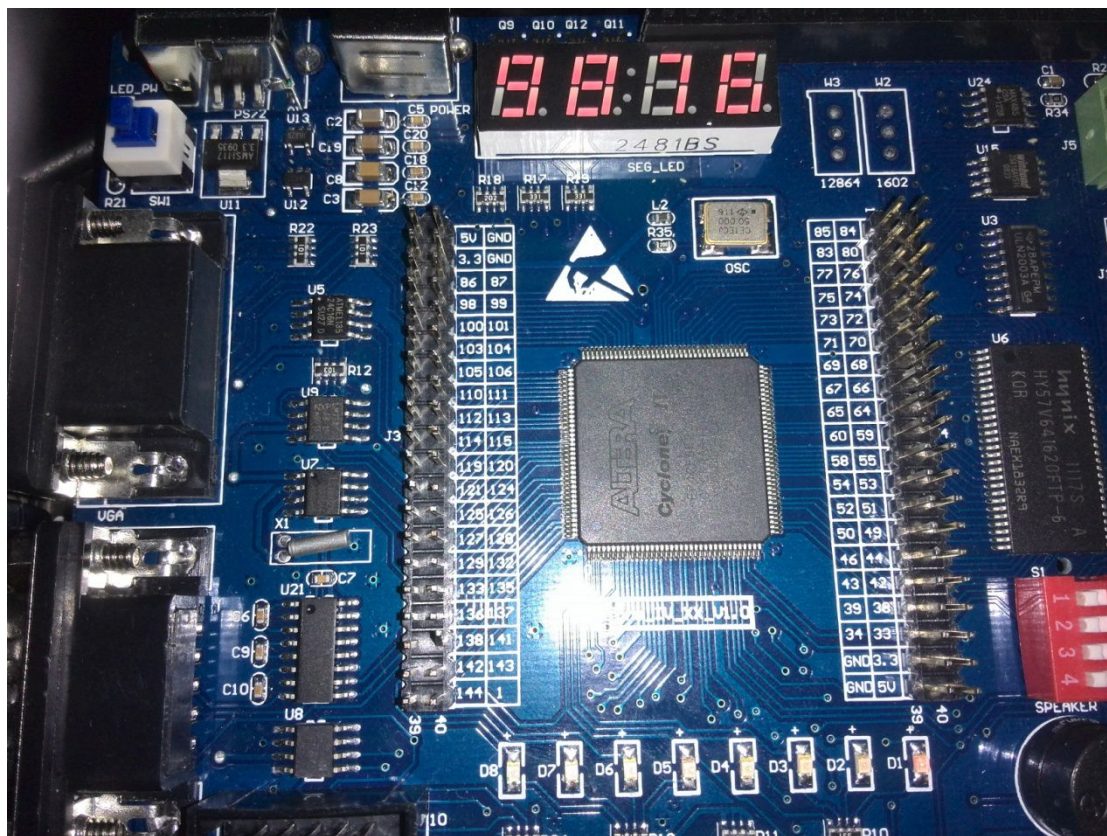
//组合逻辑进行取反，数码管为共阳极，对译码段位取反，低电平有效
always @(*) begin
    seg_a = ~segled_a ;
    seg_b = ~segled_b ;
    seg_c = ~segled_c ;
    seg_e = ~segled_e ;
    seg_d = ~segled_d ;
    seg_f = ~segled_f ;
    seg_g = ~segled_g ;
    seg_h = ~segled_h ;
end

// 四个片选通过 segled_bit_sel 进行选通，输出控制三极管，驱动数码管的四个片选
assign seg_c1 = ~( segled_bit_sel == 4'b0001 );
assign seg_c2 = ~( segled_bit_sel == 4'b0010 );
assign seg_c3 = ~( segled_bit_sel == 4'b0100 );
assign seg_c4 = ~( segled_bit_sel == 4'b1000 );

endmodule
//end of RTL code
```

3.5.5 实验现象

将程序下载到发板以后，可以看到数码管动态显示结果。



3.6 串口发送

3.6.1 串口简介

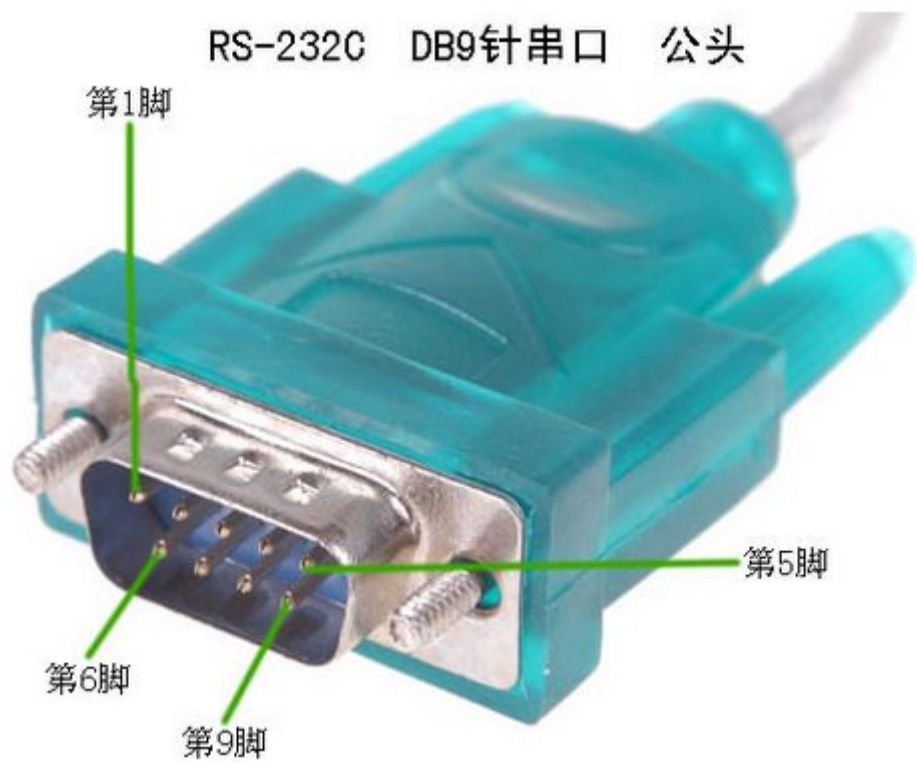
UART 是一种通用串行数据总线，用于异步通信。该总线双向通信，可以实现全双工传输和接收。一般较老的台式电脑机箱会配有 UART 串口，笔记本电脑和较新的台式电脑都取消 UART 串口了。

UART 串口是电子设计用的非常多的一种接口，UART 串口通信实现原理比较简单，基本所有 MCU/ARM/DSP 等芯片都配置有串口，经常被用来作为调试接口，还可以作为 PC 和 CPU 通信的低速接口。

电脑里面的台式机 DB9 串口如下：



由于很多电脑都没有 UART 串口，所以串口通信可以使用 USB 转串口线（一端是 USB 口，一端是 DB9 串口，方便没有 DB9 串口的电脑使用 USB 接口实现 DB9 串口功能）实现，USB 转串口接口和 PC 的串口使用起来是一样的，接口如下图所示：



DB9 的串口定义如下：一般只使用引脚 2 RXD 和 3 TXD。

9针RS-232串口（DB9）		
引脚	简写	功能说明
1	CD	载波侦测（Carrier Detect）
2	RXD	接收数据（Receive）
3	TXD	发送数据（Transmit）
4	DTR	数据终端准备（Data Terminal Ready）
5	GND	地线（Ground）

波特率：

串口有波特率的概念，波特率是表示串口发送速率的一个单位，一般串口的波特率为 3200、6400、9600、15200 等等，9600 波特率代表每秒传输 9600 个 bit 位数据，即 9600bps。

名词解释：

UART(Universal Asynchronous Receiver and Transmit)是通用异步收发器，是通信上的传输模式。

RS-232 是一种异步串行通信协议标准，最初是为远程通信连接数据终端设备 DTE(Data Terminal Equipment)与数据通信设备 DCE（Data Communication Equipment)而制定的。

RS-232 标准规定了连接电缆和机械、电气特性、信号功能及传送过程，但是 RS-232 并未定义

连接器的物理特性，因此存在 DB-25、DB-15 和 DB-9 各种类型的连接器，其引脚的定义也是各不相同的。

DB-9 是具体的物理电气接口器(connector)，我们一般使用的 RS-232 的电气连接器都是 DB-9 的。

3.6.2 实验任务

实现 9600 波特率的串口发送，按下按键发送一次数据（8bit，一个字节）。上位机（PC）串口软件可以接收到发送的数据。

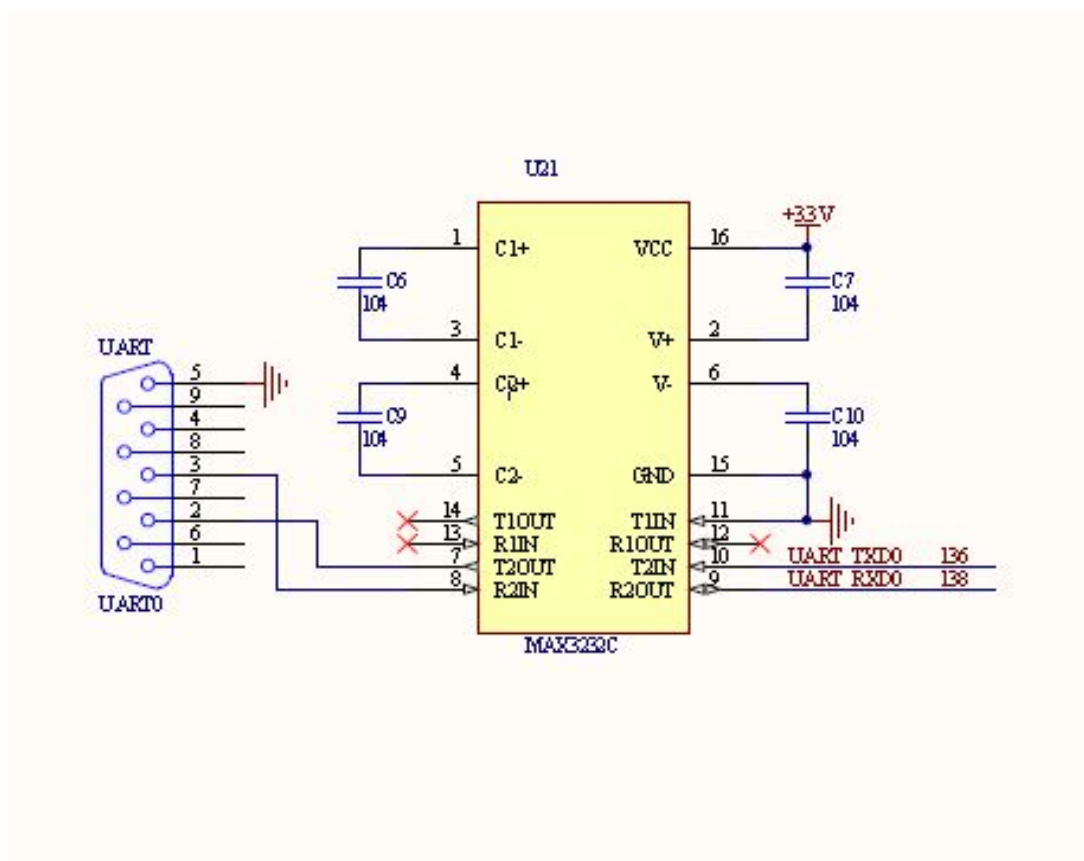
3.6.3 硬件设计

本次实验主要使用 DB9 串口的 2、3 接口，两个串口连接时，接收数据引脚与发送数据针脚相连，彼此交叉，信号对应接地即可。

RS232C 在电气特性上表现为传送的数字量采用负逻辑，且与地对称。所以和单片机或者 FPGA 开发板连接时常常需要加入电平转换芯片 MAX3232C。

MAX3232C 为电平转换芯片，将 TTL 电平转换为 RS-232 电平。MAX3232 等收发器是采用专有的低压差发送器输出级，利用双电荷泵在 3.0V 至 5.5V 电源供电。能够实现真正的 RS-232 性能，MAX3232C 供电电压 3.3V。

MAX3232C 的电路图如下：



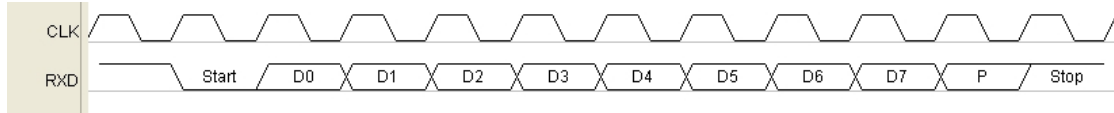
3.6.4 程序设计

设计思路：

以波特率 9600 为例子说明，波特率 9600 接收一个 bit 的时间为 $1s/9600=104\mu s$ （微秒），即

每隔 104us 发送一个数据。

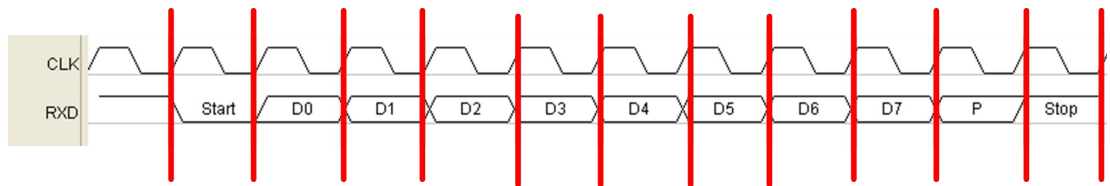
下图是串口的一数据帧（发送 8bit 有效数据）时序图，串口都是一个字节一个字节发送数据的，每个字节都需要占用一次传输，包括起始位（1bit）、数据位（8bit）、校验位（校验数据正确性，1bit）、停止位（1bit）。



下面我们需要分析把数据按照开发板的系统时钟 50Mhz 通过串口发送出去。

104us = 104000ns, 50M 时钟的一个周期时间为 20ns ($1/50\text{Mhz}=20\text{ns}$)，一个时钟周期时间远小于 104000ns，所以可以用 50M 时钟产生的计数器来计数，产生串口的发送时序，然后根据计数器特定的值来发送数据。

下图红色线为各个位可以发送数据的起始，总共有 11 个数据位（1 个起始+8 个数据+1 个校验+1 个停止位）要发送。



我们新建一个计数器，由于一次完整数据接收需要有 1144000ns (104000×11) 时间，所以计数器计到大于 1144000ns 时间即可。那么计数 1144000ns 需要多少个 50Mhz 时钟周期呢？

需要 $1144000\text{ns}/20\text{ns} = 57200$ 个时钟周期，所以需要 16 位计数器，16 位最大计数到 $2^{16}=65536$ 个时钟周期。

如果 15 bit 计数器计数够用吗？ $2^{15}=32768 < 57200$ ，所以 15bit 计数器位数不够。

下面计算各个串口数据位应该在哪个计数器时刻发送。

Startbit (0) 占据计数器的 0 — $104000\text{ns}/20\text{ns} = 5200$

D0 占据计数器的 5200— $208000\text{ns}/20\text{ns} = 10400$

D1 占据计数器的 10400— $312000\text{ns}/20\text{ns} = 15600$

D2 占据计数器的 15600— $416000\text{ns}/20\text{ns} = 20800$

D3 占据计数器的 20800— $520000\text{ns}/20\text{ns} = 26000$

D4 占据计数器的 26000— $624000\text{ns}/20\text{ns} = 31200$

D5 占据计数器的 31200— $728000\text{ns}/20\text{ns} = 36400$

D6 占据计数器的 36400— $832000\text{ns}/20\text{ns} = 41600$

D7 占据计数器的 41600— $936000\text{ns}/20\text{ns} = 46800$

校验位 占据计数器的 46800— $1040000\text{ns}/20\text{ns} = 52000$

Endbit (1) 占据计数器的 52000— $1144000\text{ns}/20\text{ns} = 57200$

根据上面的计数间隔发送对应的串口数据就可以了，就是 9600 波特率的数据发送时序。

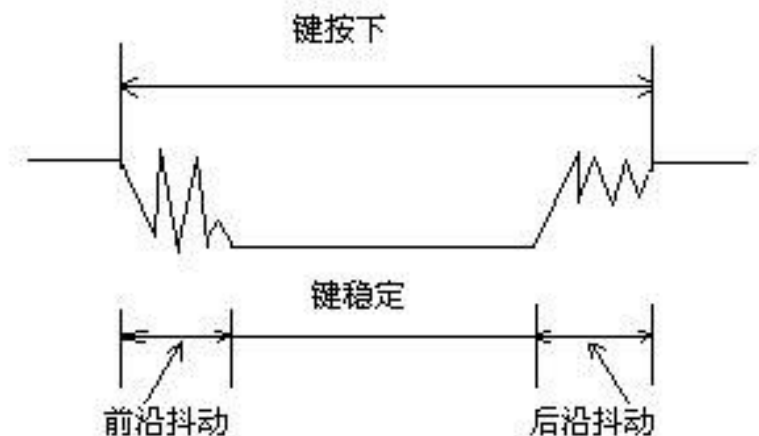
按键去抖：

该实验中我们通过按键发送数据，由于按键所用开关为机械弹性开关，当机械触点断开、闭合时，由于机械触点的弹性作用，一个按键开关在闭合时不会马上稳定地接通，在断开时也不会一下子断开。因而在闭合及断开的瞬间均伴随有一连串的抖动，会使我们的输入产生毛刺，导致发送值不稳定。为了不产生这种现象而作的措施就是按键消抖。

抖动时间的长短由按键的机械特性决定，一般为 5ms~10ms。这是一个很重要的时间参数，在

很多场合都要用到。按键稳定闭合时间的长短则是由操作人员的按键动作决定的，一般为零点几秒至数秒。键抖动会引起一次按键被误读多次。为确保 CPU/FPGA 对键的一次闭合仅作一次处理，必须去除键抖动。在键闭合稳定时读取键的状态，并且必须判别到键释放稳定后再作处理。

通常按键去抖时有软件去抖和硬件去抖两种方式，本次实验我们使用软件去抖。即使按键处于稳定状态时，才开始发送数据。因为发送数据只需计数器计数到 57200，这个时间还是相当短的，可以做到数据在去抖后马上发送完，不会受到按键后沿的影响，所以不需要考虑后沿抖动。



源代码:

```
module UartSend (
input  wire          sys_clk      , //system clock;
input  wire          sys_rst_n   , //system reset, low is active
input  [3:0]         key_data    , //KEY data in 4bit

output wire [7:0]     LED        , //led output
output reg           uart_txd

)

//parameter define
parameter WIDTH = 8 ;
parameter SIZE  = 16 ;
parameter DELAY_CNT = 10000000 ;

//reg define
reg [WIDTH-1:0] buff ;
reg [WIDTH-1:0] data_out ;

reg          uart_txd ;
reg [WIDTH-1:0] txd ; //temp txd signal;
```

手把手教您学习 FPGA

```
reg [SIZE-1:0]      counter      ;
reg [23:0]          delay_count  ;

//wire define
wire               sys_clk       ;
wire               sys_rst_n     ;

/*****
**                               Main Code
**
*****/
//将按键值赋给 buff，由于 buff 是 8 位，key_dada 是四位，这里相当于将 key_data 拼接成 8 位即
//{key_data[3],key_data[2],key_data[1],key_data[0],key_data[3],key_data[2],key_data[1],k
//ey_data[0]}
always @(posedge sys_clk or negedge sys_rst_n) begin
    if (sys_rst_n == 1'b0)          //初始化 buff, 复位时全部清零
        buff <= 8'b0;
    else
        buff <= { key_data , key_data } ;
end

//将 buff 赋值给 LED，使数值在 LED 输出显示。
assign LED = buff;

//使用软件去抖
always @(posedge sys_clk or negedge sys_rst_n) begin
    if ( sys_rst_n == 1'b0)
        delay_count <= 24'b0;
    else if (key_data != 4'hf && delay_count <= DELAY_CNT )
        //去抖时间 DELAT_CNT=10000000*20ns=0.2 秒，需要 24 位计数器。任意按键按下且在 0.2 秒之内，
        //开始计数/
        delay_count <= delay_count+ 24'b1;
    else
        delay_count <= 24'b0;
end

always @(posedge sys_clk or negedge sys_rst_n ) begin
    if ( sys_rst_n ==1'b0 )
        counter <= 16'b0;
    else if (delay_count == DELAY_CNT)
        //counter 在 delay_count 计数到 DELAY_CNT 时开始发送数据
        counter <= 16'b0;
    else if (counter <= 57200)          //完成一次发送数据
```

手把手教您学习 FPGA

```
        counter <= counter + 16'b1;
    else ;
end

always @(*) begin
    if ((counter > 0)      && (counter <= 5200 ))      //发送 start
        txd = 1'b0 ;
    else if ((counter > 5200) && (counter <= 10400))    //发送 D0
        txd = buff[0] ;
    else if ((counter > 10400) && (counter <= 15600))   //发送 D1
        txd = buff[1] ;
    else if ((counter > 15600) && (counter <= 20800))   //发送 D2
        txd = buff[2] ;
    else if ((counter > 20800) && (counter <= 26000))   //发送 D3
        txd = buff[3] ;
    else if ((counter > 26000) && (counter <= 31200))   //发送 D4
        txd = buff[4] ;
    else if ((counter > 31200) && (counter <= 36400))   //发送 D5
        txd = buff[5] ;
    else if ((counter > 36400) && (counter <= 41600))   //发送 D6
        txd = buff[6] ;
    else if ((counter > 41600) && (counter <= 46800))   //发送 D7
        txd = buff[7] ;
    else if ((counter > 46800) && (counter <= 52000))   //校验位
        txd = 1'b1 ;
    else if ((counter > 52000) && (counter <= 57200))   //停止位
        txd = 1'b1 ;
    else
        txd = 1'b1 ;
end

always @(posedge sys_clk or negedge sys_rst_n) begin
    if (sys_rst_n == 1'b0)
        uart_txd <= 1'b1;
    else
        uart_txd <= txd;      //寄存器输出，打拍处理，防止时序毛刺产生
end

endmodule
//end of RTL code
```


3.6.5 实验现象

打开下图的串口软件（光盘中会配送，仅作学习使用），将程序下载到开发板，连接好普通串口线或者 USB 串口线后（如果是 USB 串口，那么下图串口软件的串口号：可能为 COM3 或者 COM4），并勾选 HEX 显示。按下开发板的 key1 至 key4 后会显示响应的值，如下图所示。



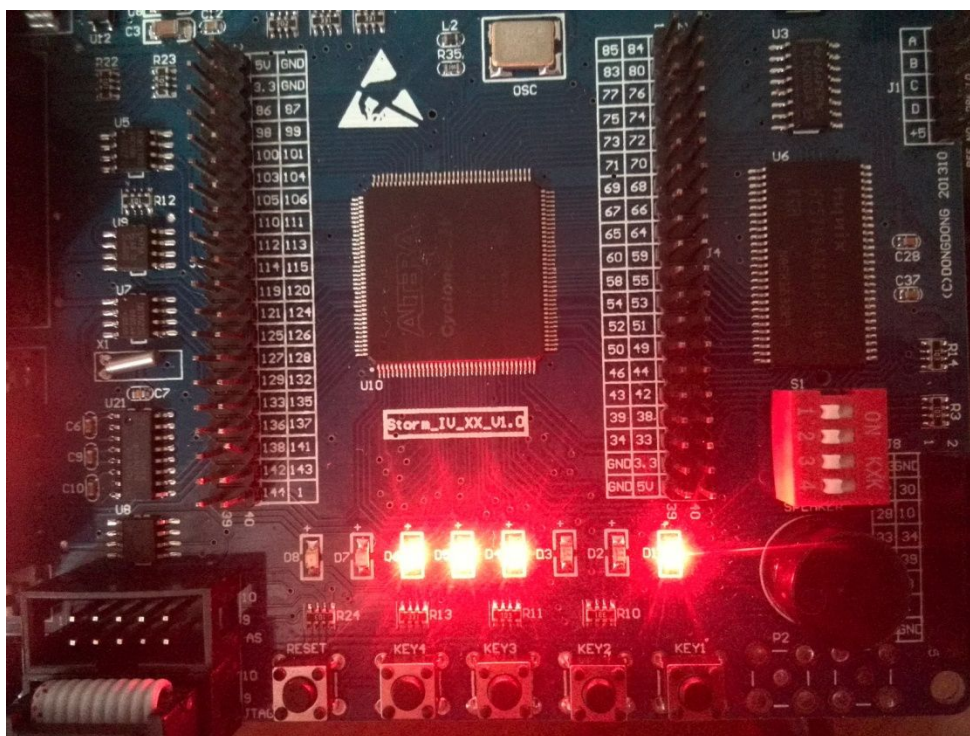
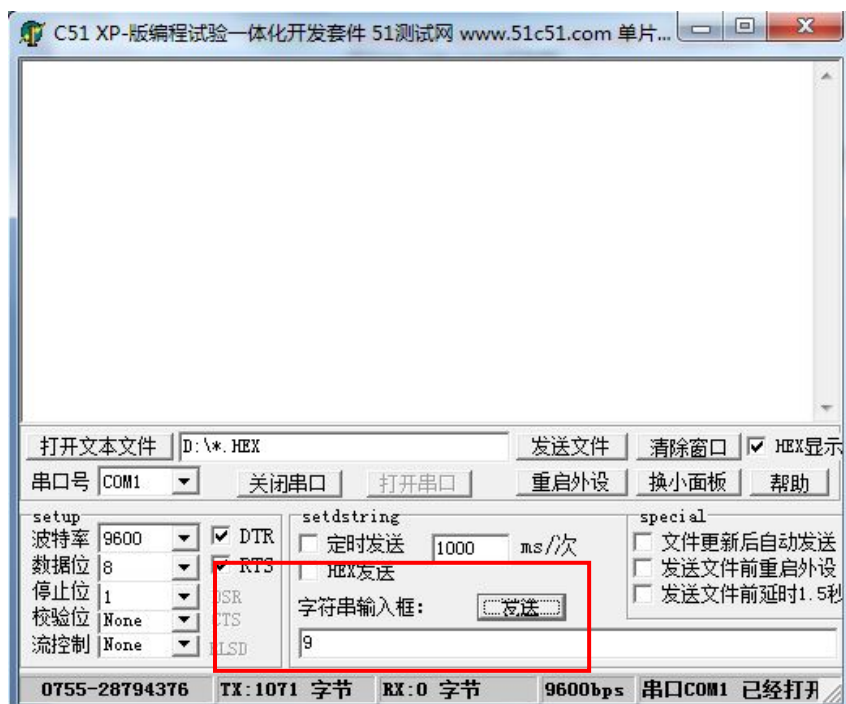
3.7 串口接收

预览版本不提供该部分。

3.7.5 实验现象

将开发板与 PC 相连，程序下载到开发板后，再通过串口线或者 USB 串口线与 PC 相连，在上位机软件的图示方框输入字符串，则在开发板可以看到 LED 表示的相应的 ASCII 值。如输入数字 9（图中红色方框内），点击发送。

9 对应的 ASCII 值为 57，开发板上 LED 显示：00111001，如下图所示：



3.8 同步 FIFO

3.8.1 同步 FIFO 简介

FIFO 是一种先进先出的数据缓存器，在逻辑设计里面用的非常多，FIFO 设计可以说是逻辑设计人员必须掌握的常识性设计。FIFO 一般用在隔离两边读写带宽不一致，或者位宽不一样的地方。

FIFO 包括同步 FIFO 和异步 FIFO 两种，同步 FIFO 有一个时钟信号，读和写逻辑全部使用这一

个时钟信号，异步 FIFO 有两个时钟信号，读和写逻辑用的各种的读写时钟，本节说的全部是同步 FIFO。

FIFO 与普通存储器 RAM 的区别是没有外部读写地址线，使用起来非常简单，但缺点就是只能顺序写入数据，顺序的读出数据，其数据地址由内部读写指针自动加 1 完成，不能像普通存储器那样可以由地址线决定读取或写入某个指定的地址。FIFO 本质上是由 RAM 加读写控制逻辑构成的一种先进先出的数据缓冲器。

FIFO 的常见参数：

- FIFO 的宽度：即 FIFO 一次读写操作的数据位；
- FIFO 的深度：指的是 FIFO 可以存储多少个 N 位的数据（如果宽度为 N）。
- 满标志：FIFO 已满或将要满时由 FIFO 的状态电路送出的一个信号，以阻止 FIFO 的写操作继续向 FIFO 中写数据而造成溢出（overflow）。
- 空标志：FIFO 已空或将要空时由 FIFO 的状态电路送出的一个信号，以阻止 FIFO 的读操作继续从 FIFO 中读出数据而造成无效数据的读出（underflow）。
- 读时钟：读操作所遵循的时钟，在每个时钟沿来临时读数据。
- 写时钟：写操作所遵循的时钟，在每个时钟沿来临时写数据。

读写指针（FIFO 读写地址）的工作原理：

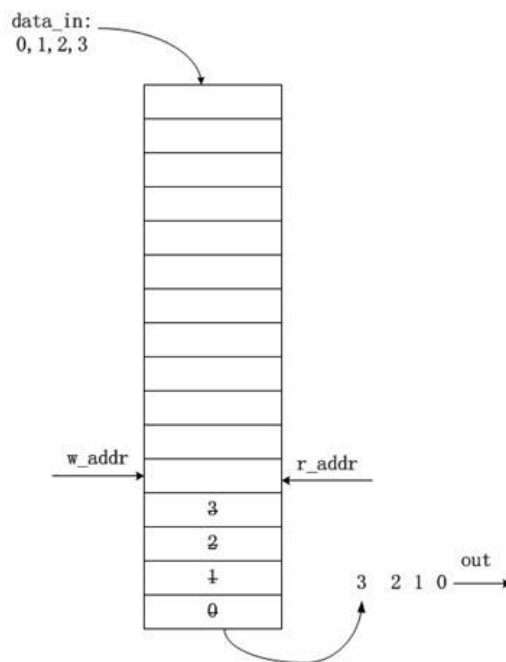
读指针：总是指向下一个将要被写入的单元，复位时，指向第 1 个单元（编号为 0）。

写指针：总是指向当前要被读出的数据，复位时，指向第 1 个单元（编号为 0）

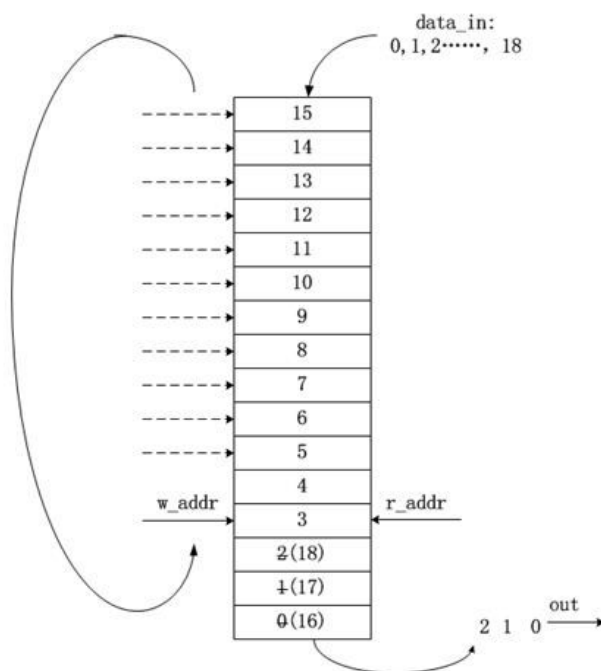
FIFO 的“空” / “满”检测：

FIFO 设计的关键：产生可靠的 FIFO 读写指针和生成 FIFO “空” / “满”状态标志。

当读写指针相等时，表明 FIFO 为空，这种情况发生在复位操作时，或者当读指针读出 FIFO 中最后一个字后，追赶上了写指针时，如下图所示：



当读写指针再次相等时，表明 FIFO 为满，这种情况发生在，当写指针转了一圈，折回来(wrapped around)又追上了读指针，如下图：



为了区分到底是满状态还是空状态，可以采用以下方法：

在指针中添加一个额外的位(extra bit)，当写指针增加并越过最后一个 FIFO 地址时，就将写指针这个未用的 MSB 加 1，其它位回零。对读指针也进行同样的操作。此时，对于深度为 $2n$ 的 FIFO，需要的读/写指针位宽为 $(n+1)$ 位，如对于深度为 8 的 FIFO，需要采用 4bit 的计数器，0000~1000、1001~1111，MSB 作为折回标志位，而低 3 位作为地址指针。

如果两个指针的 MSB 不同，说明写指针比读指针多折回了一次；如 $r_addr=0000$ ，而 $w_addr = 1000$ ，为满。

如果两个指针的 MSB 相同，则说明两个指针折回的次数相等。其余位相等，说明 FIFO 为空；

下面设计一个 16×8 （16 是深度，8 是位宽）的 FIFO：

1. 功能定义：

用 16×8 （16 是深度，8 是位宽）RAM 实现一个同步先进先出（FIFO）队列设计。由写使能端控制数据流写入 FIFO，并由读使能控制 FIFO 中数据的读出。写入和读出的操作由时钟的上升沿触发。当 FIFO 的数据满和空的时候分别设置相应的高电平加以指示。

2. FIFO 信号定义：

信号名称	I/O	功能描述	备注
Rst	In	全局复位（低有效）	
Clk	In	全局时钟	频率 10Mhz；
Wr_en	In	低有效写使能	
Rd_en	In	低有效读使能	
Data_in [7: 0]	In	数据输入端	
Data_out [7: 0]	Out	数据输出端	
Empty	Out	空指示信号	为高时表示 fifo 空
Full	Out	满指示信号	为高时表示 fifo 满

3. 顶层模块划分及功能实现

该同步 FIFO 可划分为如下四个模块，如图 1 所示：

- ① 存储器模块 (RAM) —— 用于存储数据；
- ② 读地址模块 (rd_addr) —— 用于读地址的产生；
- ③ 写地址模块 (wr_addr) —— 用于写地址的产生
- ④ 标志模块 (flag_gen) —— 用于产生 FIFO 当前空满状态。

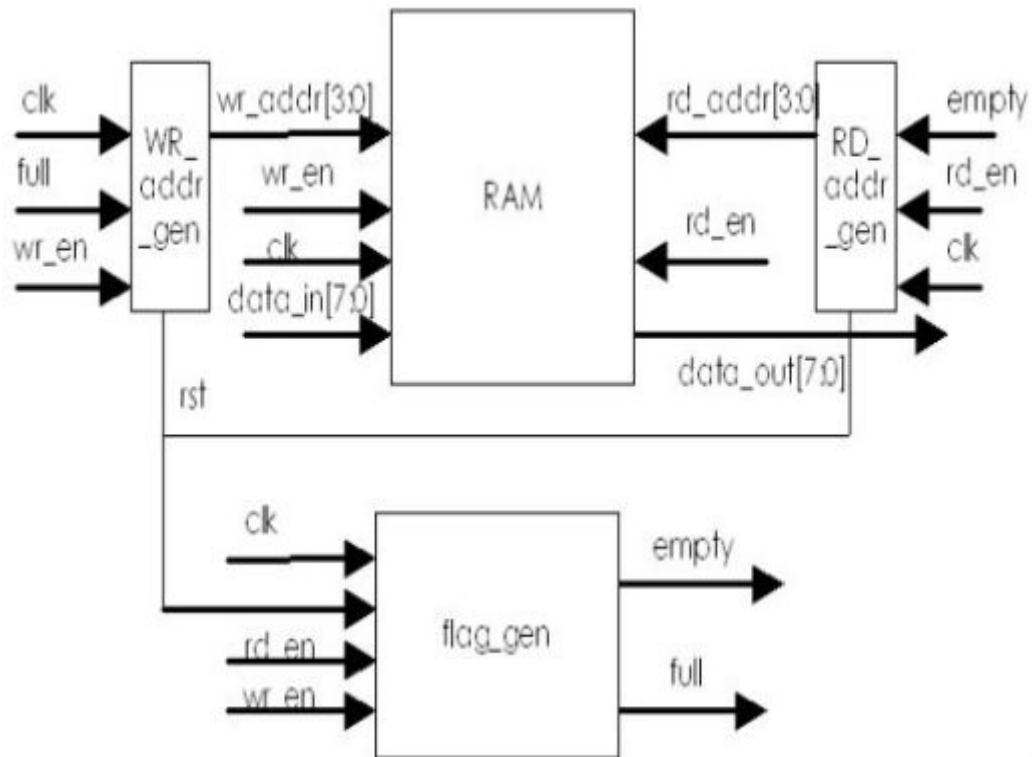


图 1 同步 FIFO 的模块划分

1) RAM 模块

本设计中的 FIFO 采用采用 16*8 双口（一个写口，一个读口）RAM，以循环读写的方式实现；

读地址：根据 RD_addr_gen 模块产生的读地址，在读使能 (rd_en) 为高电平的时候，将 RAM 中 rd_addr[3:0] 地址中的对应单元的数据在时钟上升沿到来的时候，读出到 data_out[7:0] 中。

写地址：根据 WR_addr_gen 产生的写地址，在写使能 (wr_en) 为高电平的时候，将输入数据 (data_in[7:0]) 在时钟上升沿到来的时候，写入 wr_addr[3:0] 地址对应的单元。

2) WR_addr_gen:

该模块用于产生 FIFO 写数据时所用的地址。由于 16 深度的 RAM 单元可以用 4 位地址线寻址。

本模块用 4 位计数器 (wr_addr[3:0]) 实现写地址的产生。

在复位信号为 0（复位有效）时，写地址值为 0。

如果 FIFO 未滿 (full=0, FIFO 满了如果还继续写入，会导致新数据覆盖掉原来的旧数据) 且有写使能 (wr_en) 有效，则 wr_addr[3:0] 加 1；否则写地址不变。

3) RD_addr_gen:

该模块用于产生 FIFO 读数据时所用的地址。由于 16 个 RAM 单元可以用 4 位地址线寻址。本模块用 4 位计数器 (rd_addr[3:0]) 实现读地址的产生。

手把手教您学习 FPGA

在复位信号为 0（复位有效）时，读地址值为 0。

如果 FIFO 未空（empty=0，FIFO 空了如果还继续读，会导致 FIFO 读出的数据不是期望的数据或者不确定的数据）且有读使能（rd_en）有效，则 rd_addr[3:0]加 1；否则读地址不变。

4) flag_gen 模块

flag_gen 模块产生 FIFO 空满标志。本模块设计没有使用读写地址判定 FIFO 是否空满（也可以使用读写地址做判断），而是设计一个计数器，该计数器(ptr_cnt)用于指示当前 FIFO 中数据的个数。由于 FIFO 中最多只有 16 个数据，因此采用 5 位计数器来指示 FIFO 中数据个数。具体计算如下：

复位的时候，ptr_cnt=0；

如果 wr_en 和 rd_en 同时有效的时候，ptr_cnt 不加也不减；表示同时对 FIFO 进行读写操作的时候，FIFO 中的数据个数不变。

如果 wr_en 有效且 full=0，则 ptr_cnt+1；表示写操作且 FIFO 未满足时候，FIFO 中的数据个数增加了 1；

如果 rd_en 有效且 empty=0，则 ptr_cnt-1；表示读操作且 FIFO 未满足时候，FIFO 中的数据个数减少了 1；

如果 ptr_cnt=0 的时候，表示 FIFO 空，需要设置 empty=1；如果 ptr_cnt=16 的时候，表示 FIFO 现在已经满，需要设置 full=1。

3.8.2 实验任务

设计一个 FIFO，具有读写控制逻辑、空满指示信号。

3.8.3 硬件设计

该实验不适合做硬件实现，适合 Modelsim 软件仿真，不涉及硬件。

3.8.4 程序设计

```
module sync_fifo
(
    input                clk          ,
    input                rst          ,
    input                wr_en        ,
    input                rd_en        ,
    input    [7:0]       data_in      ,
    output reg [7:0]     data_out     ,
    output reg           empty        ,
    output reg           full         ,
);

reg [3:0] wr_addr          ;
reg [3:0] rd_addr          ;
reg [4:0] count            ;
```

手把手教您学习 FPGA

```
parameter max_count = 5'b10000 ;
parameter max1_count = 5'b01111 ;
reg [7:0] fifo [0:max_count] ; // 定义一个二维的 RAM，实际设计中根据 RAM 深度和宽度可能会采用 Vedor 提供的 RAM
```

//读操作

```
always @ (posedge clk or negedge rst) begin
    if (rst == 1'b0)
        data_out <=0;
    else if (rd_en && empty==0)
        data_out<=fifo[rd_addr];
end
```

//写操作

```
always @ (posedge clk ) begin
    if (wr_en==1&&full==0)
        fifo[wr_addr]<=data_in;
end
```

//更新读地址

```
always @ (posedge clk or negedge rst) begin
    if (rst == 1'b0)
        rd_addr<=4'b0000;
    else if (empty==0&&rd_en==1)
        rd_addr<=rd_addr+1;
end
```

//更新写地址

```
always @ (posedge clk or negedge rst) begin
    if (rst == 1'b0)
        wr_addr<=4'b0000;
    else if (full==0&&wr_en==1)
        wr_addr<=wr_addr+1;
    else
        wr_addr<=4'b0000;
end
```

//更新标志位

手把手教您学习 FPGA

```
always @ (posedge clk or negedge rst) begin
    if (rst == 1'b0)
        count<=0;
    else begin
        case({wr_en, rd_en})
            2'b00:count<=count;
            2'b01:
                if(count!=5'b000000)
                    count<=count-1;
            2'b10:
                if(count!= max1_count)
                    count<=count+1;
            2'b11:count<=count;
        endcase
    end
end

always @(count) begin
    if(count==5'b000000)
        empty<=1;
    else
        empty<=0;
end

always @(count) begin
    if (count== max_count)
        full<=1;
    else
        full<=0;
end
endmodule
```

下面是 TestBench 代码（使用 Modelsim 仿真使用的激励代码）：

手把手教您学习 FPGA

TestBench.v :

```
`timescale 1 ns/ 1 ps
module sync_fifo_vlg_tst();

    reg          clk      ;
    reg [7:0] data_in    ;
    reg    rd_en        ;
    reg    rst          ;
    reg    wr_en        ;
    // wires
    wire [7:0] data_out  ;
    wire      empty     ;
    wire      full      ;
    reg [7:0] dindely    ;
    sync_fifo il (
        .clk      ( clk      ),
        .data_in   ( data_in  ),
        .data_out  ( data_out ),
        .empty     ( empty    ),
        .full      ( full     ),
        .rd_en     ( rd_en    ),
        .rst       ( rst      ),
        .wr_en     ( wr_en    )
    );

    always @(posedge clk) begin
        dindely <= data_in;
    end

    initial begin
        clk=0;
        rst=0;
        rd_en=0;
        wr_en=0;
        data_in=0;
    end
endmodule
```

手把手教您学习 FPGA

```
#40
rst = 1 ;

#35

wr_en=1;

#400

rd_en=1;

end

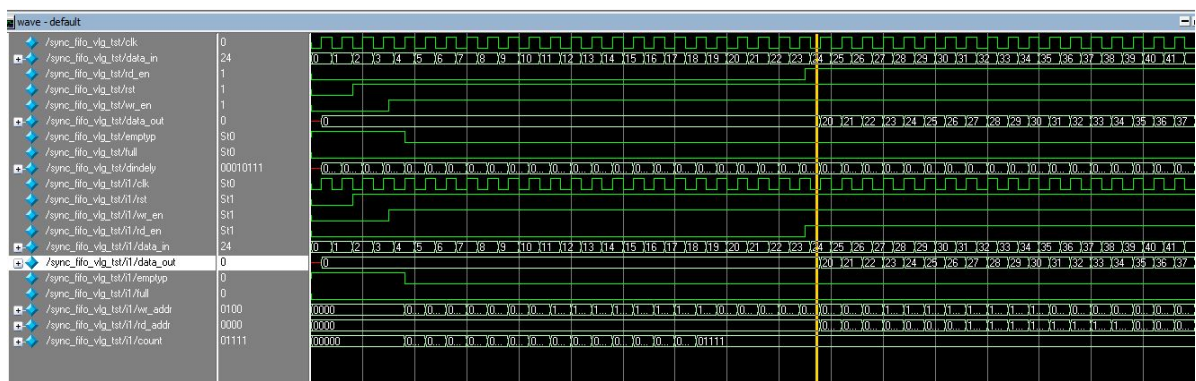
always #20 data_in<=data_in+1;

always #10 clk=~clk;

endmodule
```

3.8.5 实验现象

该实验不适合做硬件实现，适合 Modelsim 软件仿真，仿真波形如下图所示：

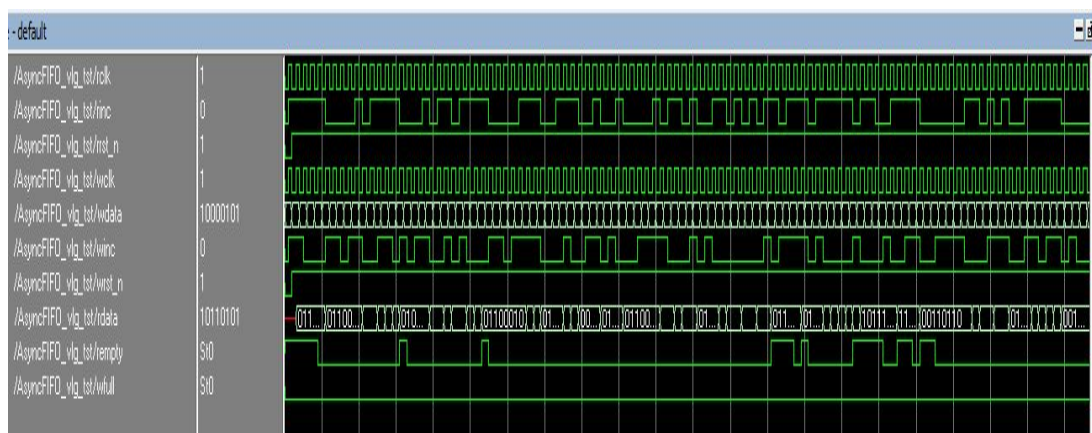


3.8 异步 FIFO

预览版本不提供该部分。

3.8.5 实验现象：

该实验不适合做硬件实现，适合 Modelsim 软件仿真，仿真波形如下图所示：



3.9 状态机

3.9.1 状态机简介

有限状态机英文名字，Finite State Machine，简称状态机，缩写为 FSM。

有限状态机是指输出取决于过去输入部分和当前输入部分的时序逻辑电路。有限状态机又可以认为是组合逻辑和寄存器逻辑的一种组合。状态机特别适合描述那些发生有先后顺序或者有逻辑规律的事情，其实这就是状态机的本质。状态机就是对具有逻辑顺序或时序规律的事件进行描述的一种方法。

根据状态机的输出是否与输入条件相关，可将状态机分为两大类，即摩尔 (Moore) 型状态机和米勒 (Mealy) 型状态机。

- Mealy 状态机：时序逻辑的输出不仅取决于当前状态，还取决于输入。
- Moore 状态机：时序逻辑的输出只取决于当前状态。

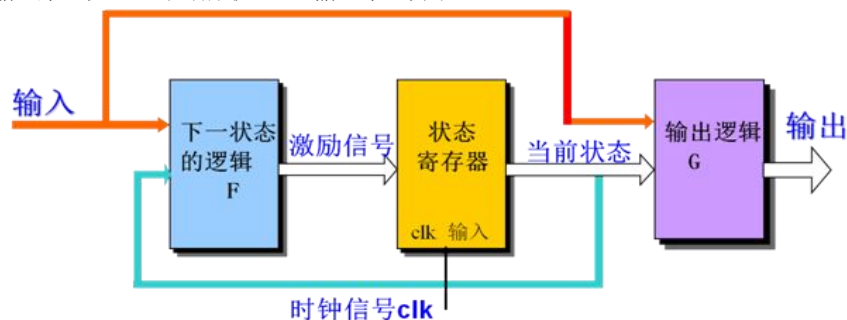
根据实际写法，状态机还可以分为一段式、二段式和三段式状态机。

- 一段式：把整个状态机 写在一个 always 模块中，并且这个模块既包含状态转移，又含有组合逻辑输入/输出。
- 二段式：状态切换用时序逻辑，次态输出和信号输出用组合逻辑。
- 三段式：状态切换用时序逻辑，次态输出用组合逻辑，信号输出用时序逻辑。

Mealy 状态机：

下一个状态 = F(当前状态，输入信号)；

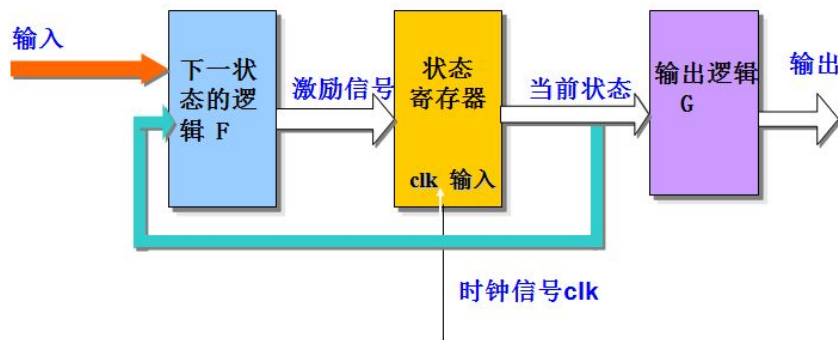
输出信号 = G(当前状态，输入信号)；



Moore 状态机：

下一个状态 = F(当前状态，输入信号)；

输出信号 = G(当前状态)；



逻辑设计里面，最常用的是二段式和三段式的状态机，三段式状态机比二段式状态机更好，下面重点介绍三段式状态机。

三段式状态机：

两段式直接采用组合逻辑输出，而三段式则通过在组合逻辑后再增加一级寄存器来实现时序逻辑输出。这样做的好处是可以有效地滤去组合逻辑输出的毛刺，同时可以有效地进行时序计算与约束，另外对于总线形式的输出信号来说，容易使总线数据对齐，从而减小总线数据间的偏移，减小接收端数据采样出错的频率。

三段式状态机的基本格式是：

- 第一个 always 语句实现同步状态跳转；
- 第二个 always 语句实现组合逻辑；
- 第三个 always 语句实现同步输出。

3.9.2 实验任务

使用状态机设计一个 7 分频的分频器。

3.9.3 硬件设计

该实验不适合做硬件实现，适合 Modelsim 软件仿真，不涉及硬件。

3.9.4 程序设计

设计思路：

描述状态机需要注意的事项：

- 定义模块名和输入输出端口；
- 定义输入、输出变量或寄存器；
- 定义时钟和复位信号；
- 定义状态变量和状态寄存器；
- 用时钟沿触发的 always 块表示状态转移过程；
- 在复位信号有效时给状态寄存器赋初始值；

- 描述状态的转换过程：符合条件，从一个状态到另外一个状态，否则留在原状态；
- 验证状态转移的正确性，必须完整和全面。

方案说明：

1、本状态机采用独热码设计，简称 one-hot code，独热码编码的最大优势在于状态比较时仅需要比较一个位，从而一定程度上简化了译码逻辑。

2、一般状态机状态编码使用二进制编码、格雷码、独热码。

各种编码比较：

二进制编码、格雷码编码使用最少的触发器，消耗较多的组合逻辑，而独热码编码反之。独热码编码的最大优势在于状态比较时仅需要比较一个位，从而一定程度上简化了译码逻辑。虽然在需要表示同样的状态数时，独热编码占用较多的位，也就是消耗较多的触发器，但这些额外触发器占用的面积可与译码电路省下来的面积相抵消。

Binary（二进制编码）、gray-code（格雷码）编码使用最少的触发器，较多的组合逻辑，而 one-hot（独热码）编码反之。one-hot 编码的最大优势在于状态比较时仅需要比较一个 bit，一定程度上从而简化了比较逻辑，减少了毛刺产生的概率。另一方面，对于小型设计使用 gray-code 和 binary 编码更有效，而大型状态机使用 one-hot 更高效。

源代码：

```
module divider7_fsm (
//input
input      sys_clk      ,      // system clock;
input      sys_rst_n    ,      // system reset, low is active;

//output
output reg  clk_divide_7      // output divide 7 clk
);

//reg define
reg [6:0]    curr_st      ;      // FSM current state
reg [6:0]    next_st      ;      // FSM next state
reg          clk_divide_7 ;      // generated clock, divide by 7

//wire define

//parameter define

//one hot code design
parameter S0 = 7'b0000000;
parameter S1 = 7'b0000001;
parameter S2 = 7'b0000010;
parameter S3 = 7'b0000100;
parameter S4 = 7'b0001000;
parameter S5 = 7'b0010000;
```

手把手教您学习 FPGA

```
parameter S6 = 7'b0100000;

/*****
**          Main Program
**
*****/
//generate FSM next state
always @(posedge sys_clk or negedge sys_rst_n) begin
    if (sys_rst_n ==1'b0) begin
        curr_st <= 7'b0;
    end
    else begin
        curr_st <= next_st;
    end
end

//FSM state logic
always @(*) begin
    case (curr_st)
        S0: begin
            next_st = S1;
        end
        S1: begin
            next_st = S2;
        end
        S2: begin
            next_st = S3;
        end
        S3: begin
            next_st = S4;
        end
        S4: begin
            next_st = S5;
        end
        S5: begin
            next_st = S6;
        end
        S6: begin
            next_st = S0;
        end
        default: next_st = S0;
    endcase
end
```


手把手教您学习 FPGA

AT24C01/02/04/08/16 是一个 1K/2K/4K/8K/16K 位串行 CMOS E2PROM 内部含有 128/256/512/1024/2048 个 8 位字节。

AT24C01 有一个 8 字节页写缓冲器，AT24C02/04/08/16 有一个 16 字节页写缓冲器该器件通过 I2C 总线接口进行操作有一个专门的写保护功能。

AT24CXX 系列的操作时序都是一样的，本文以 AT24C16 为例子进行设计。

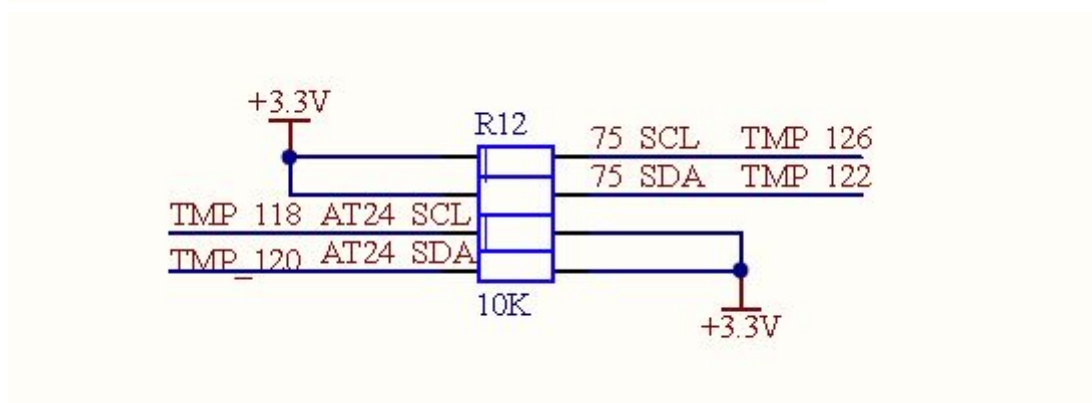
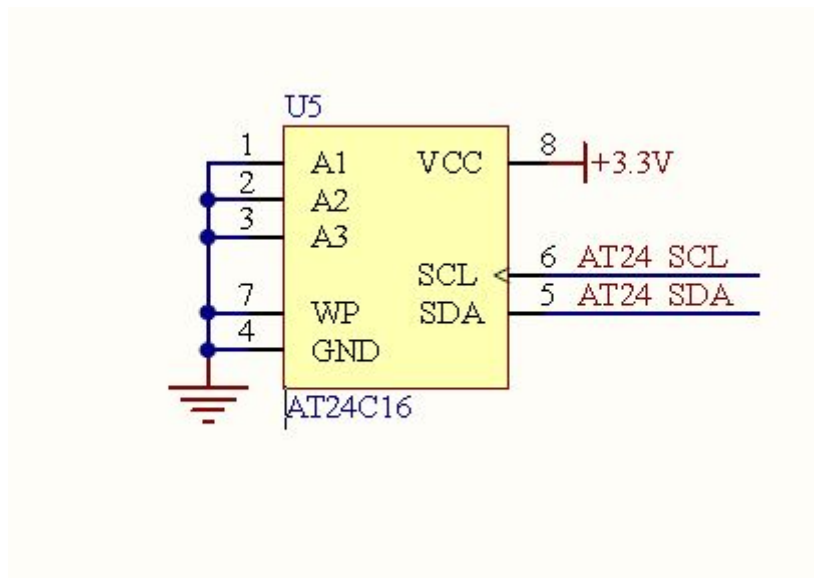
3.10.2 实验任务

在 E2PROM AT24C16 的 0x02 地址写入 0x55 数据。

3.10.3 硬件设计

A1/A2/A3 地址代表每个器件的编号，开发板上三个地址位全部接地，因此 `device_addr = 3'b000` (程序中会用到)。

WP 接地，处于写保护关闭状态。



I2C 接口需要接上拉电阻，原因是 I2C 接口是漏极开路模式，需要接上拉电阻拉到确定的电平上。

3.10.4 程序设计

设计思路：

本次实验也是主要根据 AT24CXX 的字节写时序要求以及 I2C 总线协议的规定来编写程序。

AT24CXX 时钟频率如下图所示：

读写周期范围						
符号	参数	1.8 V, 2.5 V		4.5V~5.5V		单位
		最小	最大	最小	最大	
F_{SCL}	时钟频率		100		400	KHz
T_1	SCL, SDA 输入的噪声抑制时间		200		200	ns

由于系统时钟为 50MHz，为了分频方便，此处选用 50KHz 的频率作为 AT24C16 时钟频率。

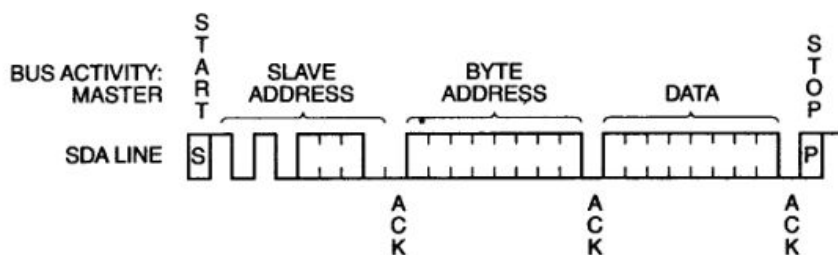
分频计数器最大需要计数到 $50\text{MHz}/50\text{KHz} = 50000000/50000 = 1000$ 。

AT24CXX 的写操作包括：字节写和页写，本文采用字节写模式。

字节写：

在字节写模式下主器件发送起始命令和从器件地址信息 R/W 位置零给从器件，在从器件产生应答信号后，主器件发送 AT24CXX 的字节地址，主器件在收到从器件的另一个应答信号后，再发送数据到被寻址的存储单元，AT24CXX 再次应答，并在主器件产生停止信号后开始内部数据的擦写。在内部擦写过程中 AT24CXX 不再应答主器件的任何请求。

字节写时序：



总共 29 个 I2C 时钟周期完成一次写操作，本设计使用 SCL 的四倍频率来控制计数器， 4×29 小于 256，所以计数器宽度为 8 位宽即可。本设计使用计数器控制 SDA 相对于 SCL 的相位关系（开始和结束就是通过相位调整来得到的）。

I2C 总线数据通信协议规范（必须遵守）：

主机和 DEVICE（设备）之间的通信必须严格遵循 I2C 总线管理定义的规则。DEVICE（设备）寄存器读/写操作的协议通过下列描述的步骤来说明：

1、通信开始之前，I2C 总线必须空闲或者不忙。这就意味着总线上的所有器件都必须释放 SCL 和 SDA 线，SCL 和 SDA 线被总线的上拉电阻拉高。

2、由主机来提供通信所需的 SCL 时钟脉冲。在连续的 9 个 SCL 时钟脉冲作用下，数据（8 位的数据字节以及紧跟其后的 1 个应答状态位）被传输。

3、在数据传输过程中，除起始和停止信号外，SDA 信号必须保持稳定，而 SCL 信号必须为高。这就表明 SDA 信号只能在 SCL 为低时改变。

4、S：起始信号，主机启动一次通信的信号，SCL 为高电平，SDA 从高电平变成低电平。

5、RS：重复起始信号，与起始信号相同，用来启动一个写命令后的读命令。

6、P：停止信号，主机停止一次通信的信号，SCL 为高电平，SDA 从低电平变成高电平。然后总线变成空闲状态。

手把手教您学习 FPGA

7、W：写位，在写命令中写/读位=0。

8、R：读位，在读命令中写/读位=1。

9、A：器件应答位，由 DEVICE（设备） 返回。当器件正确工作时该位为 0，否则为 1。为了使器件获得 SDA 的控制权，这段时间内主机必须释放 SDA 线。

10、A'：主机应答位，不是由器件返回，而是在读 2 字节的数据时由主控制器或主机设置的。在这个时钟周期内，为了告知器件的第一个字节已经读走并要求器件将第二个字节放到总线上，主机必须将 SDA 线设为低电平。

11、NA：非应答位。在这个时钟周期内，数据传输结束时器件和主机都必须释放 SDA 线，然后由主机产生停止信号。

12、在写操作协议中，数据从主机发送到器件，由主机控制 SDA 线，但在器件将应答信号发送到总线的时钟周期内除外。

13、在读操作协议中，数据由器件发送到总线上，在器件正在将数据发送到总线和控制 SDA 线的这段时间内，主机必须释放 SDA 线，但在主器件将应答信号发送到总线的时间周期内除外。

```
module e2prom_write (
input          sys_clk          ,    //system clock;
input          sys_rst_n        ,    //system reset, low is
active;

inout          i2c_sda          ,

//output ports
output         i2c_sclk         ,

output reg [7:0]                LED
);

//parameter define
parameter WIDTH = 8;

//reg define
reg  [WIDTH-1:0]    counter      ;
reg  [9:0]          counter_div  ;

reg                clk_50k       ;
reg                clk_200k      ;

reg                i2c_sclk      ;
reg                sda           ;

reg                enable        ;
reg  [WIDTH-1:0]    data_out     ;
```

手把手教您学习 FPGA

```
reg    [31:0]          counter_init      ;
//wire define

wire   [2:0]           device_addr       ;
wire   [7:0]           memory_addr       ;

wire                               sda_input      ;
wire   [10:0]          buff              ;

/*****
**                               Main Program
**
*****/

//AT24C16 device address is 000, this value is no care ;
assign device_addr =3'b000;           //device address is 000;

// AT24C16 write addr
assign memory_addr =8'h02;           //memeory address is 02;

// AT24C16 write data
assign buff        =8'h55;           //memeory data is 8'h55 ;

// counter for gen a clk_50k : need count to 1000, for 50M/1000 = 50K hz
always @(posedge sys_clk or negedge sys_rst_n) begin
    if (sys_rst_n ==1'b0)
        counter_div <= 10'b0;
    else if (counter_div >= 10'd999)
        counter_div <= 10'b0;
    else
        counter_div <= counter_div + 10'b1;
end

// gen a clk_50k use counter_div : not use counter_div 0 - 500 is for i2c bus
request start timing
always @(posedge sys_clk or negedge sys_rst_n) begin
    if (sys_rst_n ==1'b0)
        clk_50k <= 10'b0;
    else if ((counter_div >= 375) && (counter_div < 875))
        clk_50k <= 10'b1;
    else
        clk_50k <= 10'b0;
```

end

```
// counter for init for AT24C16
//延迟一定时间(时间具体值没有意义), E2PROM 内部初始化
always @(posedge sys_clk or negedge sys_rst_n) begin
    if (sys_rst_n == 1'b0)
        counter_init <= 32'h0;
    else if ( counter_init < 32'h5f5e100 )
        counter_init <= counter_init + 32'b1;
    else ;
end
```

```
// gen a 200K CLK for work counter count
always @(posedge sys_clk or negedge sys_rst_n) begin
    if (sys_rst_n == 1'b0)
        clk_200k <= 10'b0;
    else if ((counter_div >= 0 ) && (counter_div < 125))
        clk_200k <= 10'b0;
    else if ((counter_div >= 125) && (counter_div < 250))
        clk_200k <= 10'b1;
    else if ((counter_div >= 250) && (counter_div < 375))
        clk_200k <= 10'b0;
    else if ((counter_div >= 375) && (counter_div < 500))
        clk_200k <= 10'b1;
    else if ((counter_div >= 500) && (counter_div < 625))
        clk_200k <= 10'b0;
    else if ((counter_div >= 625) && (counter_div < 750))
        clk_200k <= 10'b1;
    else if ((counter_div >= 750) && (counter_div < 875))
        clk_200k <= 10'b0;
    else if ((counter_div >= 875) && (counter_div < 1000))
        clk_200k <= 10'b1;
    else ;
end
```

end

```
// when AT24C16 init finish, work counter start to add
always @(posedge clk_200k or negedge sys_rst_n) begin
    if (sys_rst_n == 1'b0)
        counter <= 8'h0;
    else if ( counter_init == 32'h5f5e100 && counter < 8'hff )
        counter <= counter + 8'b1;
    else ;
end
```

end

```
//generate real clk for SCLK ,when the i2c bus is idle, make the clk wire high
level
always @(*) begin
    if ( counter >= 2 && counter <= 118 )
        i2c_sclk = clk_50k;
    else
        i2c_sclk = 1'b1;
end

// output SDA data with AT24C16 data sheet request
// 使用四倍频方便控制 I2C 的时序高低变化，四个计数代表一个 bit 操作
// 下面全部是按照 E2PROM 的时序要求写特定 bit
always @(*) begin
    case (counter)
        0 : sda = 1'b1 ;
        1 : sda = 1'b1 ;
        2 : sda = 1'b1 ;
        3 : sda = 1'b0 ;
        4 : sda = 1'b0 ;                //SOP 开始时序

        5 : sda = 1'b1 ;
        6 : sda = 1'b1 ;
        7 : sda = 1'b1 ;
        8 : sda = 1'b1 ;                // 1

        9 : sda = 1'b0 ;
        10 : sda = 1'b0 ;
        11 : sda = 1'b0 ;
        12 : sda = 1'b0 ;              //0

        13 : sda = 1'b1 ;
        14 : sda = 1'b1 ;
        15 : sda = 1'b1 ;
        16 : sda = 1'b1 ;              //1

        17 : sda = 1'b0 ;
        18 : sda = 1'b0 ;
        19 : sda = 1'b0 ;
        20 : sda = 1'b0 ;              //0

        21 : sda = device_addr[2] ;
        22 : sda = device_addr[2] ;
```

```
23 : sda = device_addr[2] ;
24 : sda = device_addr[2] ;

25 : sda = device_addr[1] ;
26 : sda = device_addr[1] ;
27 : sda = device_addr[1] ;
28 : sda = device_addr[1] ;

29 : sda = device_addr[0] ;
30 : sda = device_addr[0] ;
31 : sda = device_addr[0] ;
32 : sda = device_addr[0] ;

33 : sda = 1'b0 ;
34 : sda = 1'b0 ;
35 : sda = 1'b0 ;
36 : sda = 1'b0 ;                                // write ,bit[0] = 0

37 : sda = 1'bz ;
38 : sda = 1'bz ;
39 : sda = 1'bz ;
40 : sda = 1'bz ;                                //ACK ,sda should be input

41 : sda = memory_addr[7] ;
42 : sda = memory_addr[7] ;
43 : sda = memory_addr[7] ;
44 : sda = memory_addr[7] ;

45: sda = memory_addr[6] ;
46: sda = memory_addr[6] ;
47: sda = memory_addr[6] ;
48: sda = memory_addr[6] ;

49: sda = memory_addr[5] ;
50: sda = memory_addr[5] ;
51: sda = memory_addr[5] ;
52: sda = memory_addr[5] ;

53: sda = memory_addr[4] ;
54: sda = memory_addr[4] ;
55: sda = memory_addr[4] ;
56: sda = memory_addr[4] ;
```

```
57: sda = memory_addr[3] ;
58: sda = memory_addr[3] ;
59: sda = memory_addr[3] ;
60: sda = memory_addr[3] ;

61: sda = memory_addr[2] ;
62: sda = memory_addr[2] ;
63: sda = memory_addr[2] ;
64: sda = memory_addr[2] ;

65: sda = memory_addr[1] ;
67: sda = memory_addr[1] ;
67: sda = memory_addr[1] ;
68: sda = memory_addr[1] ;

69: sda = memory_addr[0] ;
70: sda = memory_addr[0] ;
71: sda = memory_addr[0] ;
72: sda = memory_addr[0] ;

73: sda = 1'bz ;
74: sda = 1'bz ;
75: sda = 1'bz ;
76: sda = 1'bz ;                                //ACK

77: sda = buff[7] ;
78: sda = buff[7] ;
79: sda = buff[7] ;
80: sda = buff[7] ;                                //wirte data, high bit 7

81: sda = buff[6] ;
82: sda = buff[6] ;
83: sda = buff[6] ;
84: sda = buff[6] ;                                //wirte data, high bit 6

85: sda = buff[5] ;
86: sda = buff[5] ;
87: sda = buff[5] ;
88: sda = buff[5] ;                                //wirte data, high bit 5

89: sda = buff[4] ;
90: sda = buff[4] ;
91: sda = buff[4] ;
```

```

    92: sda = buff[4] ;                                //wirte data, high bit 4

    93: sda = buff[3] ;
    94: sda = buff[3] ;
    95: sda = buff[3] ;
    96: sda = buff[3] ;                                //wirte data, high bit 3

    97: sda = buff[2] ;
    98: sda = buff[2] ;
    99: sda = buff[2] ;
    100: sda = buff[2] ;                               //wirte data, high bit 2

    101: sda = buff[1] ;
    102: sda = buff[1] ;
    103: sda = buff[1] ;
    104: sda = buff[1] ;                               //wirte data, high bit 1

    105: sda = buff[0] ;
    106: sda = buff[0] ;
    107: sda = buff[0] ;
    108: sda = buff[0] ;                               //wirte data, high bit 0

    109: sda = 1'bz ;
    110: sda = 1'bz ;
    111: sda = 1'bz ;
    112: sda = 1'bz ;                                //ACK

    113: sda = 1'b0 ;
    114: sda = 1'b0 ;

    115: sda = 1'b1 ;
    116: sda = 1'b1 ;
    117: sda = 1'b1 ;                                //EOP

    default : sda = 1'b1 ;
endcase
end

// output SDA data enable with AT24C16 data sheet request
// 使能信号控制 SDA 是否输出
always @(*) begin
    case (counter)
        // 0 : enable = 1'b1 ;                        //when the bus is idle, the bus should

```



```
be release;
    1 : enable = 1'b1 ;
    2 : enable = 1'b1 ;
    3 : enable = 1'b1 ;
    4 : enable = 1'b1 ;           //SOP

    5 :enable = 1'b1 ;
    6 :enable = 1'b1 ;
    7 :enable = 1'b1 ;
    8 :enable = 1'b1 ;           // 1

    9 :enable = 1'b1 ;
   10 :enable = 1'b1 ;
   11 :enable = 1'b1 ;
   12 :enable = 1'b1 ;           //0

   13 :enable = 1'b1 ;
   14 :enable = 1'b1 ;
   15 :enable = 1'b1 ;
   16 :enable = 1'b1 ;           //1

   17 :enable = 1'b1 ;
   18 :enable = 1'b1 ;
   19 :enable = 1'b1 ;
   20 :enable = 1'b1 ;           //0

   21 :enable = 1'b1 ;
   22 :enable = 1'b1 ;
   23 :enable = 1'b1 ;
   24 :enable = 1'b1 ;

   25 :enable = 1'b1 ;
   26 :enable = 1'b1 ;
   27 :enable = 1'b1 ;
   28 :enable = 1'b1 ;

   29 : enable = 1'b1 ;
   30 : enable = 1'b1 ;
   31 : enable = 1'b1 ;
   32 : enable = 1'b1 ;

   33 :enable = 1'b1 ;
   34 :enable = 1'b1 ;
```

```
35 :enable = 1'b1 ;
36 :enable = 1'b1 ;                                // write , bit[0] = 0

41:enable = 1'b1 ;
42:enable = 1'b1 ;
43:enable = 1'b1 ;
44:enable = 1'b1 ;

45:enable = 1'b1 ;
46:enable = 1'b1 ;
47:enable = 1'b1 ;
48:enable = 1'b1 ;

49:enable = 1'b1 ;
50:enable = 1'b1 ;
51:enable = 1'b1 ;
52:enable = 1'b1 ;

53:enable = 1'b1 ;
54:enable = 1'b1 ;
55:enable = 1'b1 ;
56:enable = 1'b1 ;

57:enable = 1'b1 ;
58:enable = 1'b1 ;
59:enable = 1'b1 ;
60:enable = 1'b1 ;

61:enable = 1'b1 ;
62:enable = 1'b1 ;
63:enable = 1'b1 ;
64:enable = 1'b1 ;

65:enable = 1'b1 ;
67:enable = 1'b1 ;
67:enable = 1'b1 ;
68:enable = 1'b1 ;

69:enable = 1'b1 ;
70:enable = 1'b1 ;
71:enable = 1'b1 ;
72:enable = 1'b1 ;
```

```
77:enable = 1'b1 ;
78:enable = 1'b1 ;
79:enable = 1'b1 ;
80:enable = 1'b1 ;                                //wirte data, hign bit 7

81:enable = 1'b1 ;
82:enable = 1'b1 ;
83:enable = 1'b1 ;
84:enable = 1'b1 ;                                //wirte data, hign bit 6

85:enable = 1'b1 ;
86:enable = 1'b1 ;
87:enable = 1'b1 ;
88:enable = 1'b1 ;                                //wirte data, hign bit 5

89:enable = 1'b1 ;
90:enable = 1'b1 ;
91:enable = 1'b1 ;
92:enable = 1'b1 ;                                //wirte data, hign bit 4

93:enable = 1'b1 ;
94:enable = 1'b1 ;
95:enable = 1'b1 ;
96:enable = 1'b1 ;                                //wirte data, hign bit 3

97:enable = 1'b1 ;
98:enable = 1'b1 ;
99:enable = 1'b1 ;
100:enable = 1'b1 ;                               //wirte data, hign bit 2

101:enable = 1'b1 ;
102:enable = 1'b1 ;
103:enable = 1'b1 ;
104:enable = 1'b1 ;                               //wirte data, hign bit 1

105:enable = 1'b1 ;
106:enable = 1'b1 ;
107:enable = 1'b1 ;
108:enable = 1'b1 ;                               //wirte data, hign bit 0

113:enable = 1'b1 ;
114:enable = 1'b1 ;
```

手把手教您学习 FPGA

```
115:enable = 1'b1 ;
116:enable = 1'b1 ;
117:enable = 1'b1 ;                                //EOP
default : enable = 1'b0 ;

endcase
end

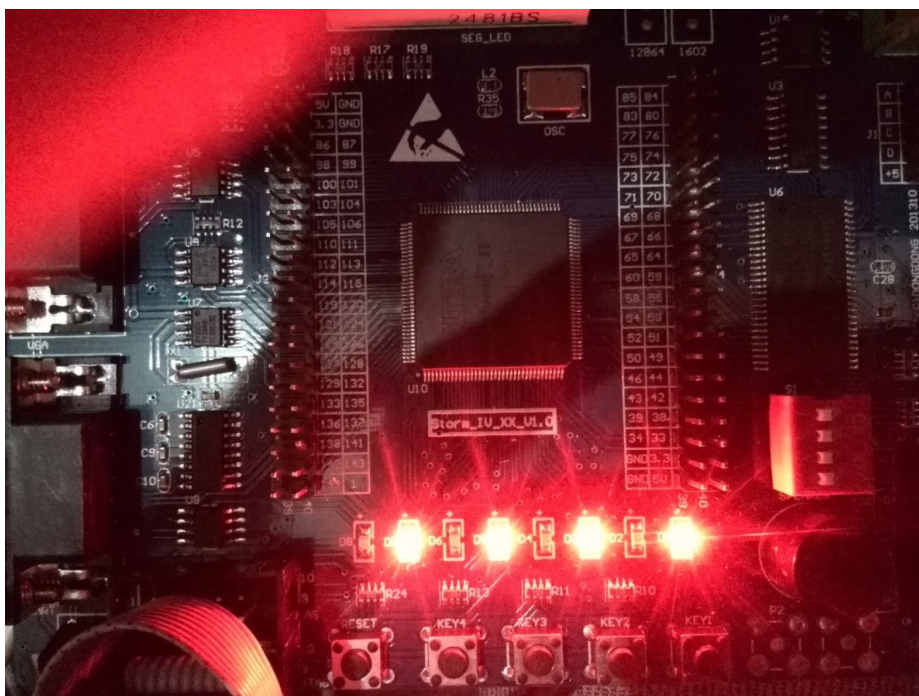
// output sda data when sda enable is 1, else output Z state
// 使用使能信号控制三态接口，应该输出还是释放总线（输出 Z 态）
assign i2c_sda = (enable == 1'b1)? sda : 1'bz;

// disp 0x55 to the led when write end
always @(posedge sys_clk or negedge sys_rst_n) begin
    if (sys_rst_n == 1'b0)
        LED <= 8'b0;
    else if ( counter == 117 )
        LED <= 8'h55 ;
    else ;
end

endmodule

//end of RTL code
```

3. 10. 5 实验现象

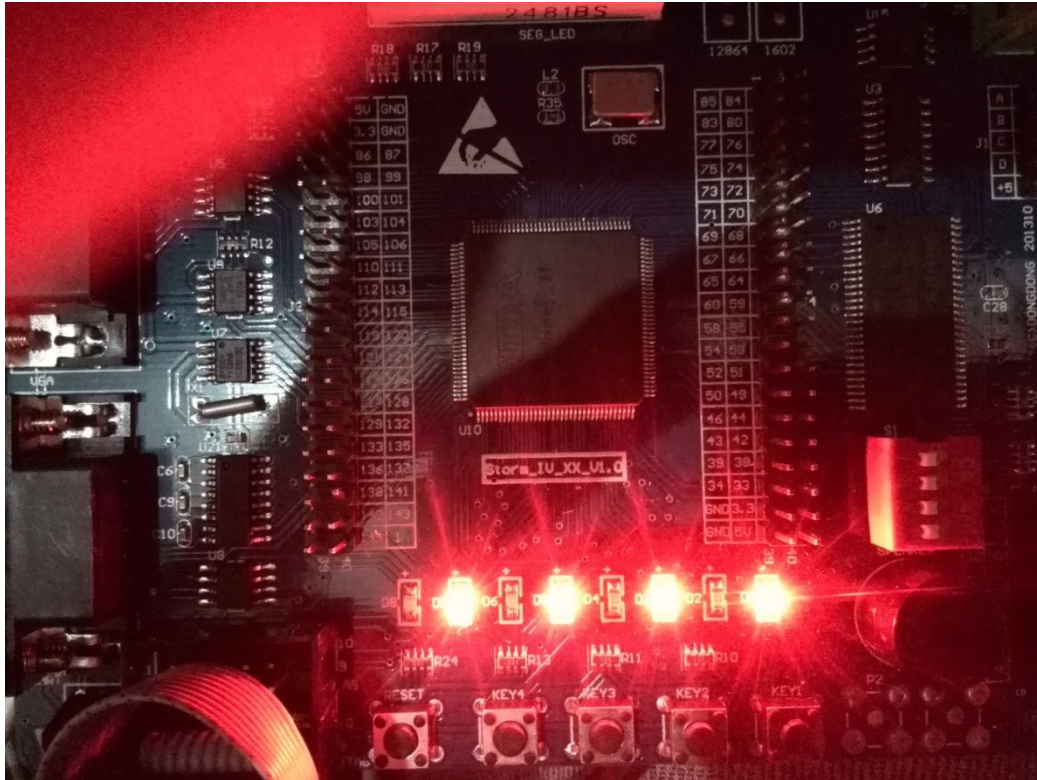


3.11 E2PROM 读操作

预览版本不提供该部分。

3.11.5 实验现象

将程序 sof 文件下载到开发板，可以观察到 LED 显示 01010101，将之前写入的 0x55 数据通过 LED 显示出来。



3.12 PS/2 键盘读操作

3.12.1 PS/2 接口简介

PS/2 是在较早电脑上常见的接口之一，用于鼠标、键盘等设备。一般情况下，PS/2 接口的鼠标为绿色，键盘为紫色。当前的电脑一般都不再配有 PS/2 接口，PS/2 键盘也比较少见了。本文之所以介绍 PS/2 接口，主要是因为 PS/2 接口协议简单，适合入门学习。

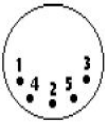
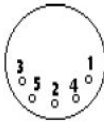


PS/2 接口是输入装置接口，而不是传输接口。所以 PS2 口根本没有传输速率的概念，只有扫描速率。在 Windows 环境下，ps/2 鼠标的采样率默认为 60 次/秒，USB 鼠标的采样率为 120 次/秒。较高的采样率理论上可以提高鼠标的移动精度。

PS/2 接口设备不支持热插拔，强行带电插拔有可能烧毁主板。

早期，在 PS/2 键盘中，包含了一个嵌入式的微控制器(如 InD1，8048 系列)，以用来执行各项工作并减少整个系统中的负担。微控制器所要作的工作就是监测所有的按键，以及当按键被按下或放开时，就回报给主机。

PS/2 有 6 脚的 mini-DIN 或 5 脚的 DIN 连接器，每种引脚的定义如下所示：

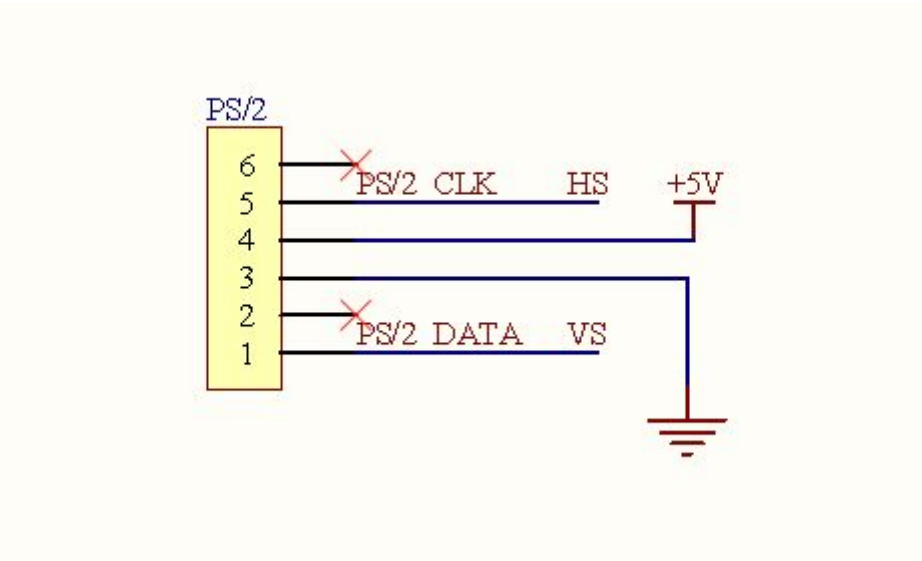
Male 公的	Female 母的	5-pin DIN (AT/XT):	5 脚 DIN(AT/XT)
		1 - Clock	1—时钟
		2 - Data	2—数据
		3 - Not Implemented	3—未实现，保留
		4 - Ground	4—电源地
(Plug) 插头	(Socket) 插座	5 - +5v	5—电源+5V

Male 公的	Female 母的	6-pin Mini-DIN (PS/2):	6 脚 Mini-DIN(PS/2)
		1 - Data	1—数据
		2 - Not Implemented	2—未实现，保留
		3 - Ground	3—电源地
		4 - +5v	4—电源+5V
(Plug) 插头	(Socket) 插座	5 - Clock	5—时钟
		6 - Not Implemented	6—未实现，保留

3. 12. 2 实验任务

PS/2 键盘按键，可以发送键值给 LED 显示（16 进制显示）。

3. 12. 3 硬件设计



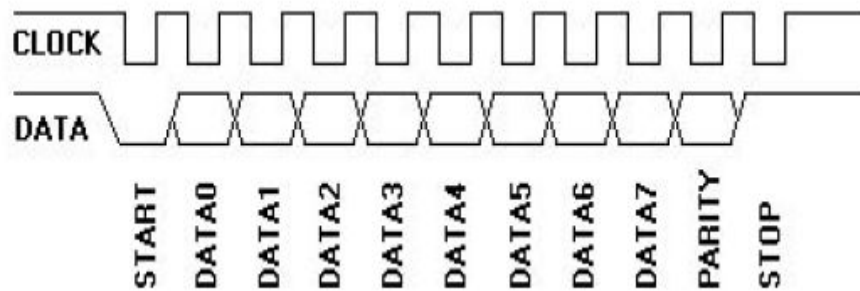
连接器上有四个管脚：电源、地、5V 数据和时钟。
PS/2 鼠标和键盘履行一种双向同步串行协议。换句话说，每次数据线上发送一位数据并且每在时钟线上发一个脉冲就被读入。

3. 12. 4 程序设计

设计思路：
数据和时钟线都是集电极开路结构（正常情况为高电平）。当键盘或鼠标等待发送数据时，它首先检查时钟以确认它是否是高电平。如果不是，那么是主机抑制了通讯，设备必须缓冲任何要发送的数据直到重新获得总线的控制权（键盘有 16 字节的缓冲区，而鼠标的缓冲区仅存储最后一个要发送的数据包）。如果时钟线是高电平，设备就可以开始传送数据。
设备到主机通讯过程中，键盘和鼠标使用一种包含 11 位的串行协议。即数据所有数据安排在字节中，每个字节为一帧，包含 11 位。这些位含义是：

1 start bit. This is always 0.	1 个起始位，总是为 0
8 data bits, least significant bit first.	8 个数据位，低位在前
1 parity bit (odd parity).	1 个校验位，奇校验
1 stop bit. This is always 1.	1 个停止位，总是为 1
1 acknowledge bit (Host-to-device communication only)	1 个应答位（仅在主机对设备的通讯中）

每位在时钟下降沿被读入，如下图所示：



我们只需要在时钟的下降沿采样即可，没有数据时候时钟为高电平，所以可以用时钟沿来控制采样。设置一个 buff 移位寄存器，然后在每个时钟下降沿采样，不需要控制信号，没有按键时候时钟是不会跳变的，所以我们只需要从 buff 中按照顺序取出我们需要的键值即可。每按键一下，就会发送一个键值给上位机。不按键不会发送任何数据。

源代码：

```
module ps2key (
    //input
    input sys_clk, //system clock;
    input sys_rst_n, //system reset, low is
    active;

    input ps2_key_clk, //ps2 keyboard clock;
    input ps2_key_data, //ps2 keyboard data ;
    //output
    output reg [7:0] led // output led,in hex
    format
);

//reg define
reg [10:0] buff ; // for regsiger data

//wire define

/*****
** Main Program
**
*****/
//在时钟的下降沿采样，使用移位寄存器串转并即可
always @(negedge ps2_key_clk ) begin
    buff[0] <= ps2_key_data ;
    buff[1] <= buff[0] ;
    buff[2] <= buff[1] ;
    buff[3] <= buff[2] ;
end
```


手把手教您学习 FPGA

```
        buff[4]  <= buff[3]          ;
        buff[5]  <= buff[4]          ;
        buff[6]  <= buff[5]          ;
        buff[7]  <= buff[6]          ;
        buff[8]  <= buff[7]          ;
        buff[9]  <= buff[8]          ;
        buff[10] <= buff[9]          ;
    end

    // 取有效数据
    always @(*) begin
        data_out[7] <= buff[2];
        data_out[6] <= buff[3];
        data_out[5] <= buff[4];
        data_out[4] <= buff[5];
        data_out[3] <= buff[6];
        data_out[2] <= buff[7];
        data_out[1] <= buff[8];
        data_out[0] <= buff[9];
    end

    //assign led = data_out ;           //led is high acitve

    always @(*) begin
        led <= data_out ;
    end

endmodule
//end of RTL code

/*****键盘上各个键的码表, 16 进制格式*****/

// 0x1C,  <=====>  'a',
// 0x32,  <=====>  'b',
// 0x21,  <=====>  'c',
// 0x23,  <=====>  'd',
// 0x24,  <=====>  'e',
// 0x2B,  <=====>  'f',
// 0x34,  <=====>  'g',
// 0x33,  <=====>  'h',
// 0x43,  <=====>  'i',
// 0x3B,  <=====>  'j',
// 0x42,  <=====>  'k',
```

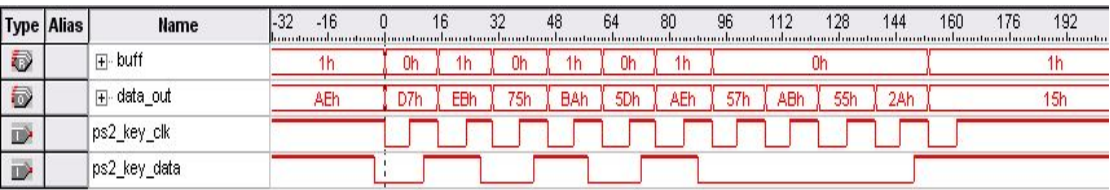
手把手教您学习 FPGA

```
// 0x4B, <=====> 'l',
// 0x3A, <=====> 'm',
// 0x31, <=====> 'n',
// 0x44, <=====> 'o',
// 0x4D, <=====> 'p',
// 0x15, <=====> 'q',
// 0x2D, <=====> 'r',
// 0x1B, <=====> 's',
// 0x2C, <=====> 't',
// 0x3C, <=====> 'u',
// 0x2A, <=====> 'v',
// 0x1D, <=====> 'w',
// 0x22, <=====> 'x',
// 0x35, <=====> 'y',
// 0x1A, <=====> 'z',
// 0x45, <=====> '0',
// 0x16, <=====> '1',
// 0x1E, <=====> '2',
// 0x26, <=====> '3',
// 0x25, <=====> '4',
// 0x2E, <=====> '5',
// 0x36, <=====> '6',
// 0x3D, <=====> '7',
// 0x3E, <=====> '8',
// 0x46, <=====> '9',
// 0x0E, <=====> '`',
// 0x4E, <=====> '-',
// 0x55, <=====> '=',
// 0x5D, <=====> '\\',
// 0x29, <=====> ' ',
// 0x54, <=====> '[',
// 0x5B, <=====> ']',
// 0x4C, <=====> ';',
// 0x52, <=====> '\\',
// 0x41, <=====> ', ',
// 0x49, <=====> '.',
// 0x4A, <=====> '/',
// 0x71, <=====> '...',
// 0x70, <=====> '0',
// 0x69, <=====> '1',
// 0x72, <=====> '2',
// 0x7A, <=====> '3',
// 0x6B, <=====> '4',
```

```
// 0x73, <=====> '5',  
// 0x74, <=====> '6',  
// 0x6C, <=====> '7',  
// 0x75, <=====> '8',  
// 0x7D, <=====> '9',  
// 0x0d, <=====> ' ',  
  
/*****键盘上各个键的码表, 结束*****/
```

3.12.5 实验现象

将 PS/2 键盘插到开发板上，上电，下载 sof 文件到板子上，然后按 Q 键，采样到的真实数据如下：



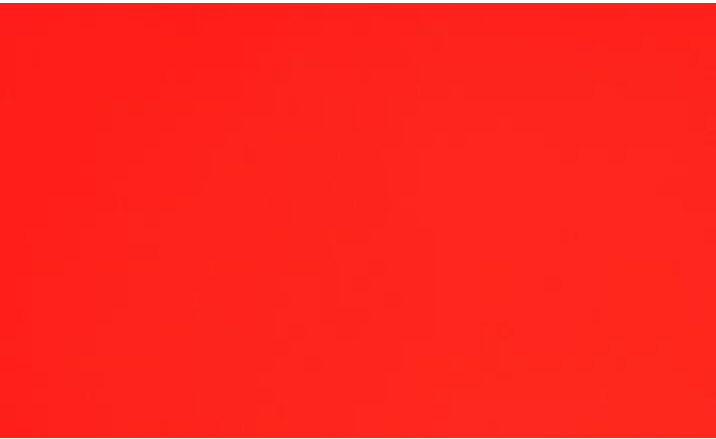
这个是按 Q 键产生的一个键值，和上面键值表比较知道采样是正确的。

3.13 VGA

预览版本不提供该部分。

3.13.5 实验现象

程序下载到 FPGA 开发板上，显示器固定显示红色。

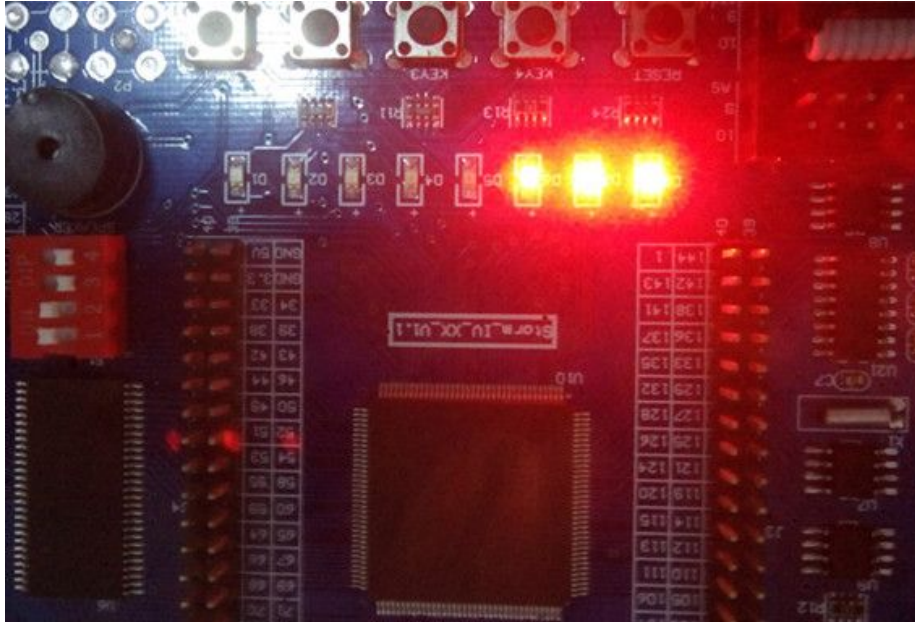


3.14 LCD1602

预览版本不提供该部分。

3.15.5 实验现象

上电，Quartus 全编译完程序后，下载 sof 文件到板子上，然后按下红外遥控器的“一”时，会有三个灯在亮，说明程序运行正确！

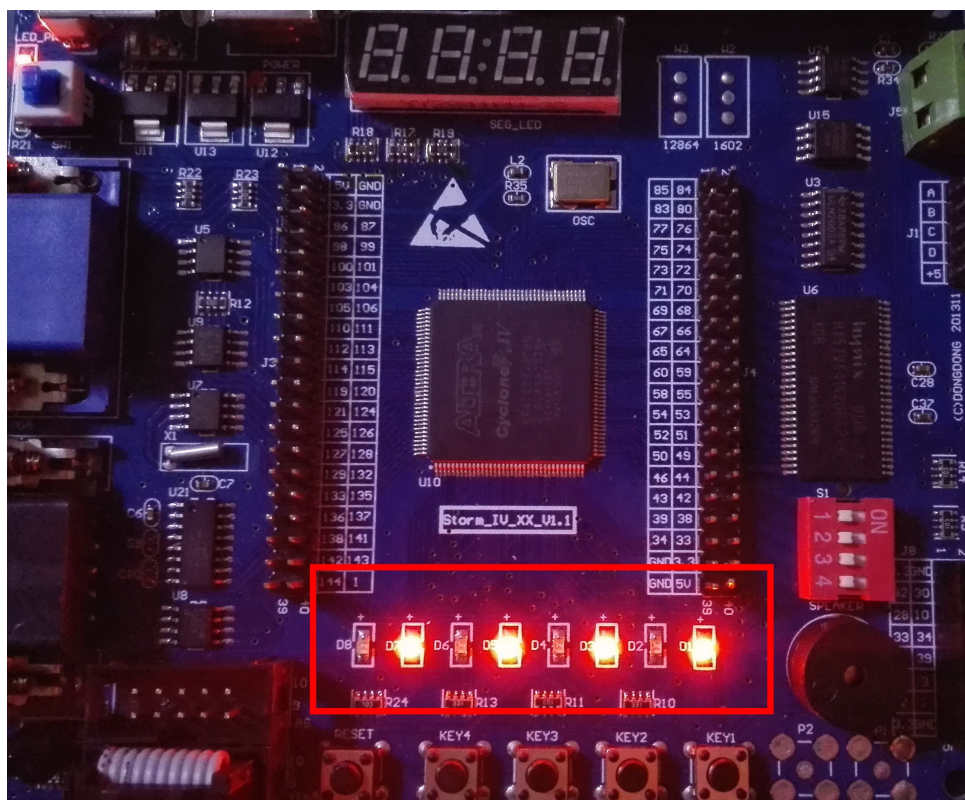


3.16 SDRAM 控制器

预览版本不提供该部分。

3.15.5 实验现象

程序编译后，下载到开发板里面，会有 4 个 LED 亮，说明 SDRAM 读写 OK。



第四章 设计思想和感悟

设计思想主要是有经验的老员工的经验积累，新员工和初学的人最欠缺的就是设计思想，因为这个是需要经过大型项目锤炼才能获得的。

可能很多初学者不理解为什么要有规范，为什么大牛和菜鸟写的代码差异很大，或者菜鸟的 Bug 很多，作为过来人，我建议菜鸟们先不要纠结对错，先按照大牛的规范和思路学习和实践，等菜鸟飞起来的那天，就会觉得之前的规范和思想有多么重要，那些都是前人的精华总结。

下面几点是我接过多个复杂项目后的几个思想总结和感悟。

4.1 代码简单化

很多初学者和没有经过正规项目实践的人可能觉得代码写的越复杂越有水平，笔者最开始学习 Verilog 也是这样想的，每每看到复杂的代码都崇拜不已。

其实不是这样的，当前项目越来越复杂，新写的代码都需要经过多轮检视和后续项目重用，如果您的代码写的复杂无比，那么带来的检视、重用工作量是巨大的，也是项目周期不容许的，所

以最好是使用简单的语句和逻辑实现复杂的功能，越简单越好。能把一个复杂的东西使用简单的语句表达出来，是一种很好的能力。

代码简单化有几点建议：

- 1) 使用最基础和常见的语法，也有利于工具的资源优化。
- 2) 少用复杂的语法，如“^^”、“===”等。
- 3) 复杂功能分析清楚，语句之间越少耦合越好。
- 4) 尽量不要使用状态机，状态机综合的电路较难分析，而且和其他电路相比还需要分析状态机覆盖率；

4.2 注释层次化

代码越复杂，注释的意义越大，注释分层可以提高对代码的理解和方案的划分。

注释层次化有几点建议：

- 1) 使用步骤 1、2、3、4 等编号描述.V 文件的结构划分和功能。
- 2) 每个步骤里面可以包括子步骤 1、2、3、4 等。
- 3) 每个子步骤如果还是很复杂，可以继续划分子子步骤。
- 4) 每个子步骤包括几个 always 或者 assign 块逻辑。

4.3 交互界面清晰化

交互界面清晰化指的是模块间接口或者模块内子模块交互简单、清晰、没有歧义。因为涉及到接口配合的地方一般都不是一个人完成的代码，凡不是一个人完成的事情都需要理解对方的意图，都是容易出问题的地方。

交互界面清晰化有几点建议：

- 1) 接口定义越简单越好，需要避免耦合太多。
- 2) 信号功能描述清晰，没有歧义。
- 3) 信号最好在什么时候代表的都是相同的功能。
- 4) 从系统方案层面划分接口，保证接口耦合性最少

4.4 模块划分最优化

复杂系统一般都是由多个模块一起完成的，如果模块划分不合理，会出现模块间耦合太多，导致系统不够健壮和清晰，很容易出问题，而且系统接受新需求能力不强，后续项目重用也比较麻烦。所以模块划分是非常重要的一个环节。让模块划分最优化是一个系统工程师必备的技能。

模块划分最优化有几点建议：

- 1) 完全理解系统的需求和规格。
- 2) 对系统的实现要做到非常清楚。
- 3) 划分的模块需要和熟悉系统的人讨论，大家都觉得是最优的时候才行。
- 4) 模块间接口简单、清晰、没有歧义。
- 5) 模块功能比较完整，同一个功能最好使用一个模块实现。
- 6) 系统划分也要方便验证。

4.5 方案精细化

设计方案是编写代码的前提，复杂些的项目如果没有方案的话，根本没有法写出代码。另外，很多初学者都希望多点时间写代码，其他的时间能少则少，其实这种思想是错误的，磨刀不误砍柴工，有了详细的设计方案后，写代码可以说是体力活。

我们之前的一个项目，方案时间是 3 个月，代码时间是 1.5 个月，方案时间远大于编码时间。另外方案必须达到能指导编码的阶段才能进行编码，其中包括：功能描述、架构实现、表现描述、模块划分、模块实现细节和时序描述、接口描述等。

有几点建议：

- 1) 设计方案细化到 always 块层次
- 2) 模块实现细节描述清楚。
- 3) 重点电路和复杂电路的时序需要在编码前画出。
- 4) 设计方案需要经过专家评审。
- 5) 设计方案需要有架构和模块划分描述。
- 6) 设计方案需要包括资源占用的评估报告。

4.6 时序流水化

时序设计是代码编写的前提，流水处理包括两种，第一种是通过时隙划分的方式实现流水，第二种是通过寄存器延时的方式进行流水设计。

第一种方式：

经过几个复杂项目，我们发现，如果时序是时隙流水的形式，可以极大的减少设计的复杂度。比如操作一个 RAM，我们使用四个时隙去划分，第一个时隙是逻辑读写，第二个时隙是 CPU 读写，那么这种就不需要做读写冲突处理。四个时隙作为一个流水不停的执行。设计简化非常多。可以进行流水时隙处理需要满足性能要求，如果我们需要每个周期都读写 RAM，那么用四个时隙去划分，第一个时隙是逻辑读写，第二个时隙是 CPU 读写就不可行了，这个时候需要做读写冲突处理。

第二种方式：

还有一种是非时隙流水处理，这种是固定使用寄存器延时进行处理，比如从 FIFO 读出数据的第一个时钟周期进行数据校验，第二个时钟周期进行数据的处理，第三个时钟周期进行数据的拼接，第四个时钟周期进行数据的 CRC 校验。这种处理方式也是流水处理的一种，这种处理方式性能很高。也是非常常见的设计方法之一。

有几点建议：

- 1) 满足性能要求的前提时隙化处理。
- 2) 通过寄存器延时的方式难度大于时隙化处理方式。
- 3) 每个时隙或者延迟周期处理事情要明确、单一。

2016-03-10 (C) 恒创科技 阿东团队